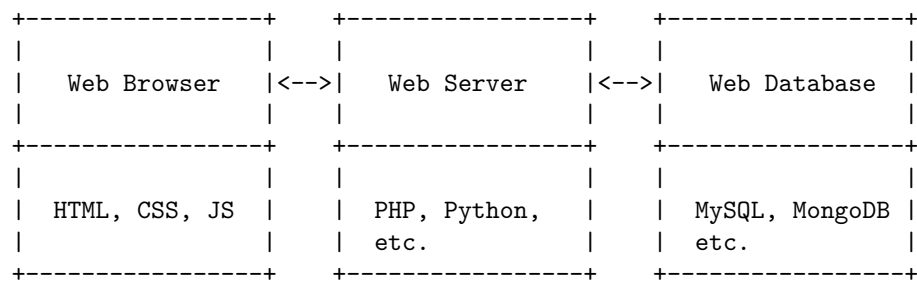


Web technology is the use of various technologies and protocols to create, deliver, and interact with web-based applications and services. Web technology consists of different components that work together to enable web functionality and user experience. Some of the common components of web technology are:

- Web browsers: These are software applications that allow users to access and view web pages and web applications. Examples of web browsers are Google Chrome, Mozilla Firefox, Microsoft Edge, etc.
- Web servers: These are software applications that run on a computer or a cloud platform and respond to requests from web browsers. Web servers host web pages, web applications, and web services. Examples of web servers are Apache, Nginx, IIS, etc.
- Web protocols: These are rules and standards that define how web browsers and web servers communicate and exchange data. Examples of web protocols are HTTP, HTTPS, FTP, SMTP, etc.
- Web languages: These are programming languages and markup languages that are used to create and display web pages and web applications. Examples of web languages are HTML, CSS, JavaScript, PHP, Python, etc.
- Web frameworks: These are software libraries and tools that provide common functionality and features for web development. Web frameworks simplify and speed up the development process by providing reusable code and templates. Examples of web frameworks are Bootstrap, React, Django, Laravel, etc.
- Web databases: These are software applications that store and manage data for web applications and services. Web databases can be relational or non-relational, depending on the data model and structure. Examples of web databases are MySQL, PostgreSQL, MongoDB, etc.

The following diagram illustrates the basic architecture of a web technology stack using ASCII art:

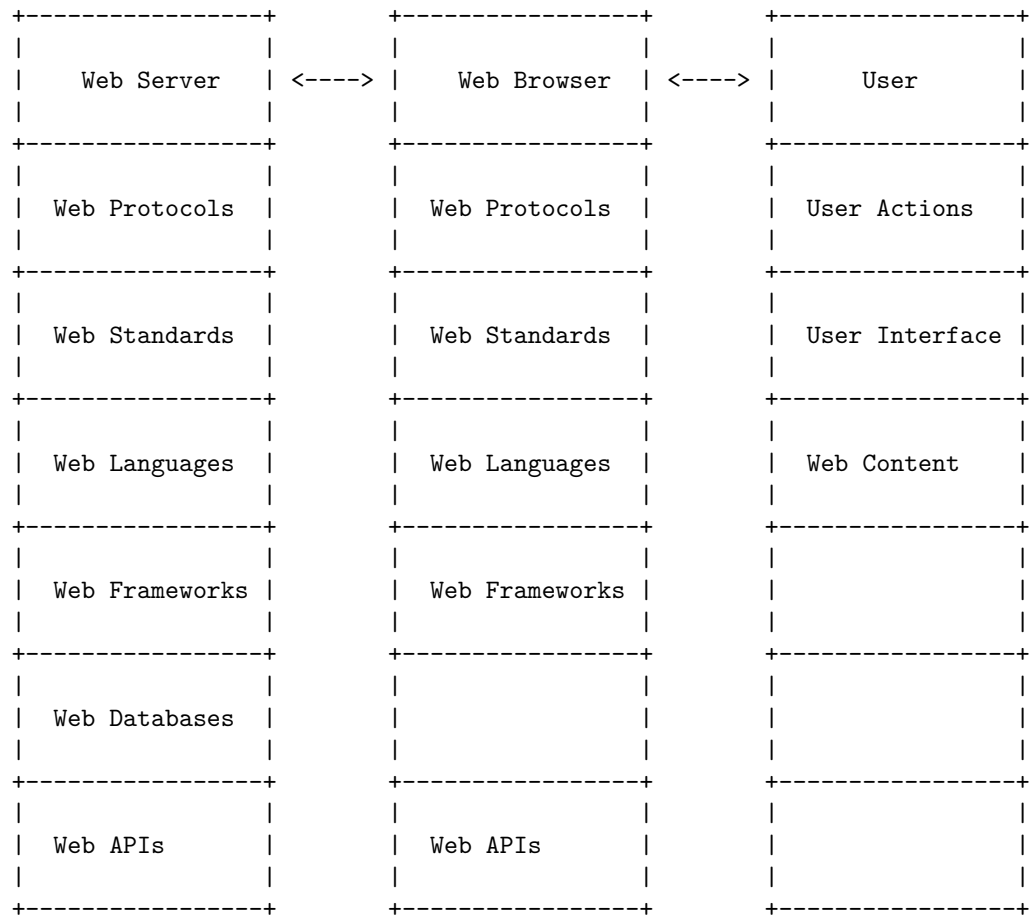


The diagram shows how a web browser sends a request to a web server using a web protocol (such as HTTP). The web server then processes the request using a web language (such as PHP) and interacts with a web database (such as MySQL) to retrieve or store data. The web server then sends a response to the web browser using a web protocol (such as HTTP). The web browser then displays the response using a web language (such as HTML) and a web framework (such as Bootstrap).

Unit 1 - Introduction to Web Technology

Web technology is the use of internet-based computer programs to provide information, services, or interactions to users. Web technology consists of different components that work together to create web applications, such as websites, web servers, web browsers, web protocols, web standards, web languages, web frameworks, web databases, and web APIs.

The following diagram illustrates the basic architecture of a web application using ASCII art:

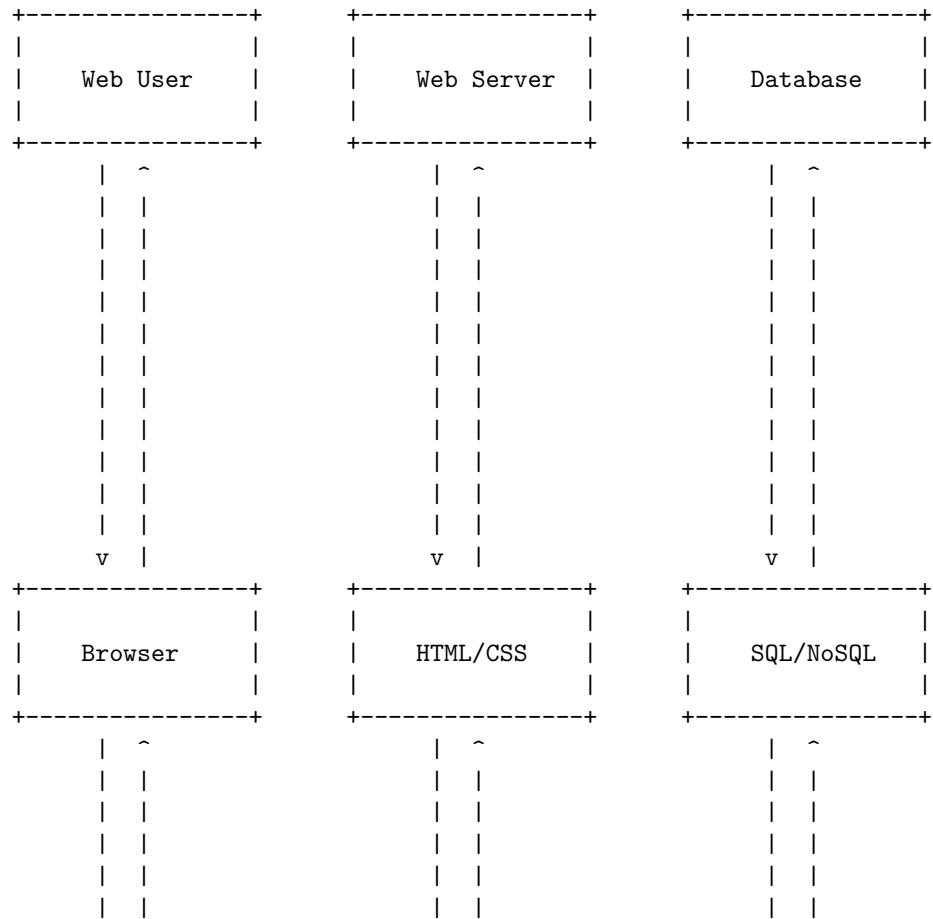


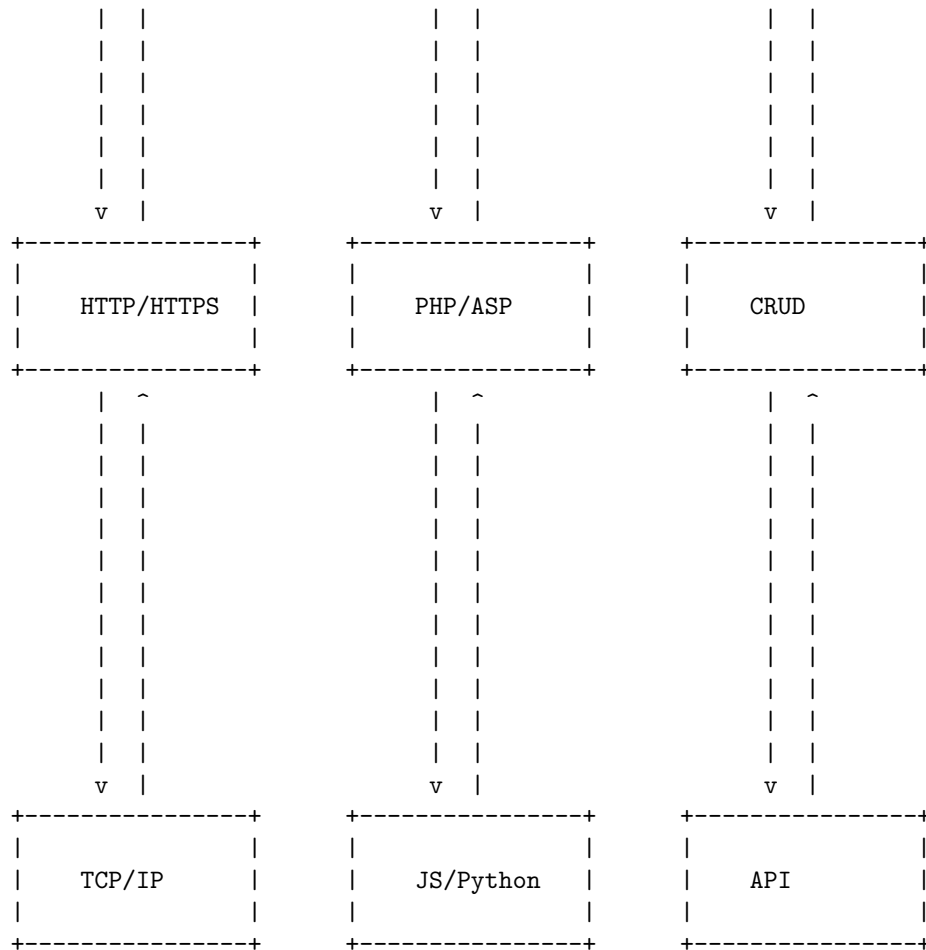
The web server is the computer program that hosts the web application and responds to requests from the web browser. The web browser is the software that allows the user to access and view the web application. The web protocols are the rules and methods that enable the communication between the web server and the web browser, such as HTTP, HTTPS, FTP, etc. The web standards are the specifications and guidelines that define the structure, format, and behavior

of the web application, such as HTML, CSS, JavaScript, XML, JSON, etc. The web languages are the programming or markup languages that are used to create and manipulate the web application, such as PHP, Python, Ruby, Java, C#, etc. The web frameworks are the libraries or tools that provide common functionalities and features for the web application, such as Django, Laravel, Rails, ASP.NET, etc. The web databases are the systems that store and retrieve the data for the web application, such as MySQL, MongoDB, PostgreSQL, etc. The web APIs are the interfaces that allow the web application to interact with other web services or applications, such as Google Maps, Facebook, Twitter, etc.

Web technology is a method by which computers communicate with each other with the help of markup languages and multimedia packages. Web technology involves developing a web site for the Internet (World Wide Web) or an intranet (a private network).

The following diagram illustrates the basic architecture of a web site using web technology:





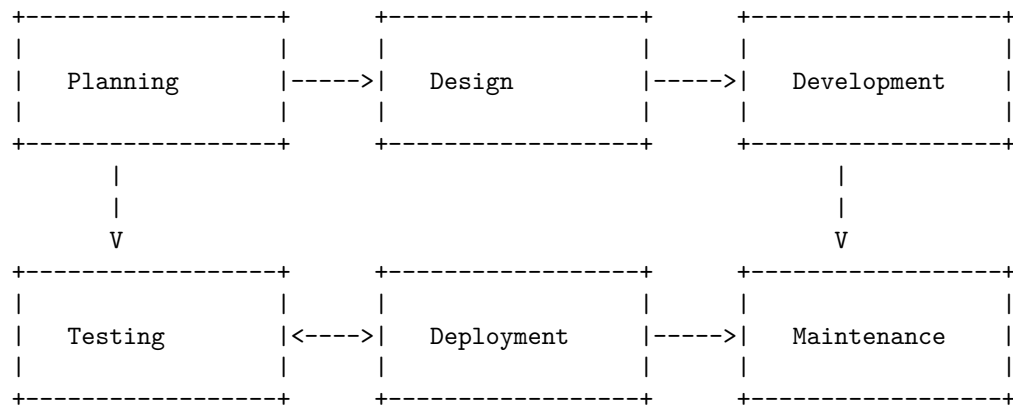
The diagram shows the following components:

- Web User: The person who accesses the web site using a web browser.
- Web Server: The computer that hosts the web site and responds to the requests from the web user.
- Database: The system that stores and retrieves the data for the web site.
- Browser: The software that interprets and displays the web pages on the web user's device.
- HTML/CSS: The markup languages that define the structure and style of the web pages.
- SQL/NoSQL: The languages that query and manipulate the data in the database.
- HTTP/HTTPS: The protocols that enable the communication between the browser and the web server.
- PHP/ASP: The server-side scripting languages that generate dynamic web pages

Web development strategies are the methods and techniques used in the planning, design, development, deployment and management of web applications and systems. The aim of web development strategies is to ensure that these applications and systems are aligned with the business goals of the organisation, and that they meet the needs of the users.

Web Development Strategies

The following diagram illustrates the basic architecture of a web development strategy:



The diagram shows the following steps:

- **Planning:** This is the first stage of web development, where the scope, objectives, requirements, budget, timeline, and stakeholders of the project are defined. The planning stage also involves conducting research, analysis, and brainstorming to identify the best solutions and technologies for the project.
- **Design:** This is the stage where the visual and functional aspects of the web application or system are created. The design stage involves creating wireframes, mockups, prototypes, and user interface elements, as well as defining the user experience, navigation, and content structure of the web application or system.
- **Development:** This is the stage where the web application or system is coded, integrated, and tested. The development stage involves writing the front-end and back-end code, using the appropriate programming languages, frameworks, libraries, and tools, as well as implementing the database, security, and performance features of the web application or system.
- **Deployment:** This is the stage where the web application or system is launched and made available to the users. The deployment stage involves uploading the web application or system files to the web server, configuring the domain name, and ensuring the web application or system is

functioning properly.

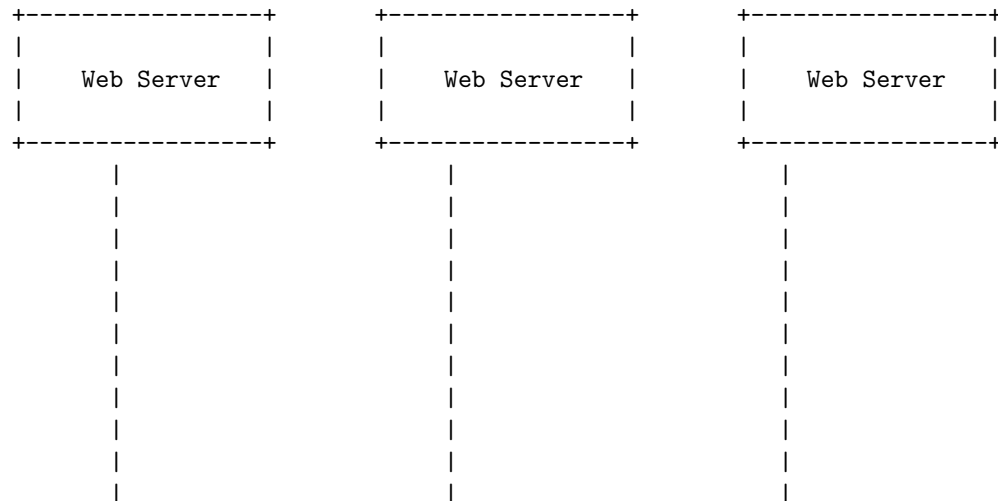
- Maintenance: This is the stage where the web application or system is monitored, updated, and improved. The maintenance stage involves fixing bugs, adding new features, enhancing the user experience, and ensuring the web application or system is compatible with the latest technologies and standards.

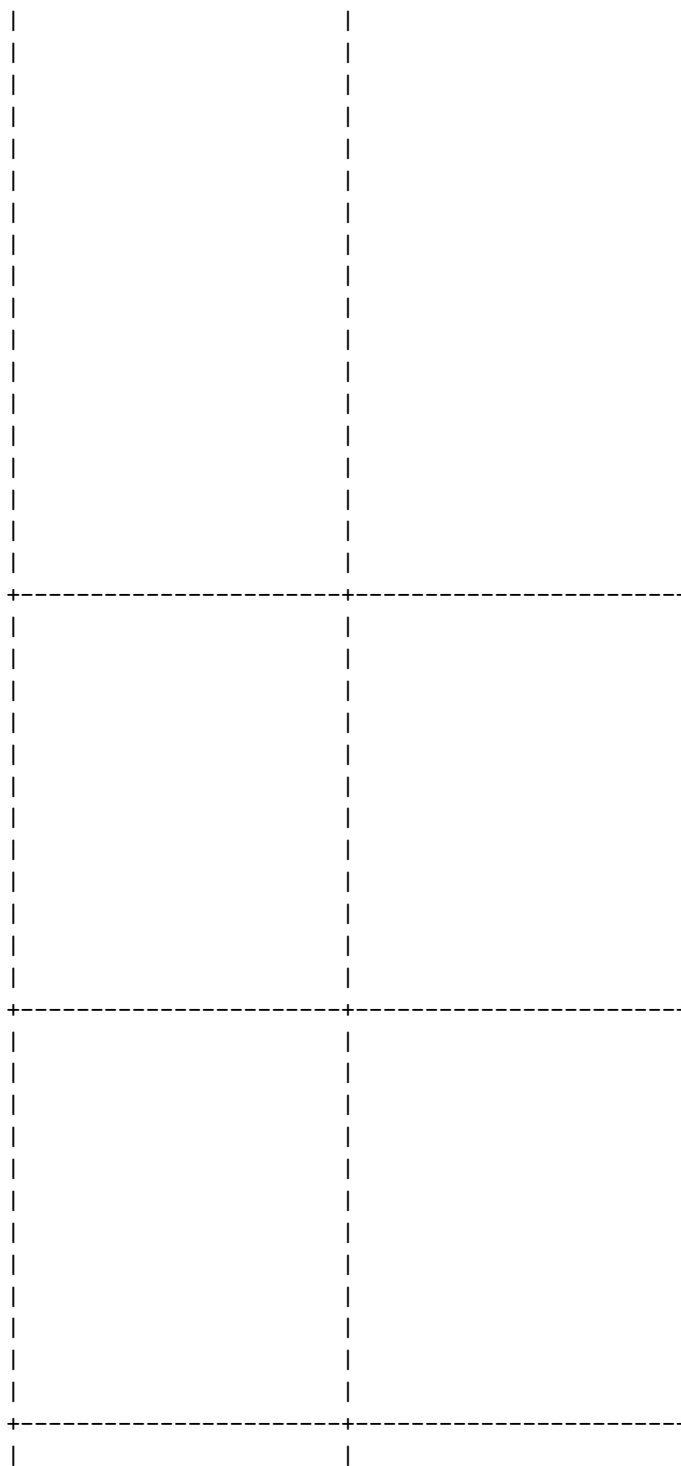
These steps are not necessarily linear or sequential, but rather iterative and cyclical, meaning that they can be repeated and revisited as needed throughout the web development process. Web development strategies can vary depending on the type, size, and complexity of the web application or system, as well as the preferences and practices of the web developers and the organisation. However, the general principles and goals of web development strategies are to create web applications and systems that are user-friendly, functional, reliable, secure, and scalable.

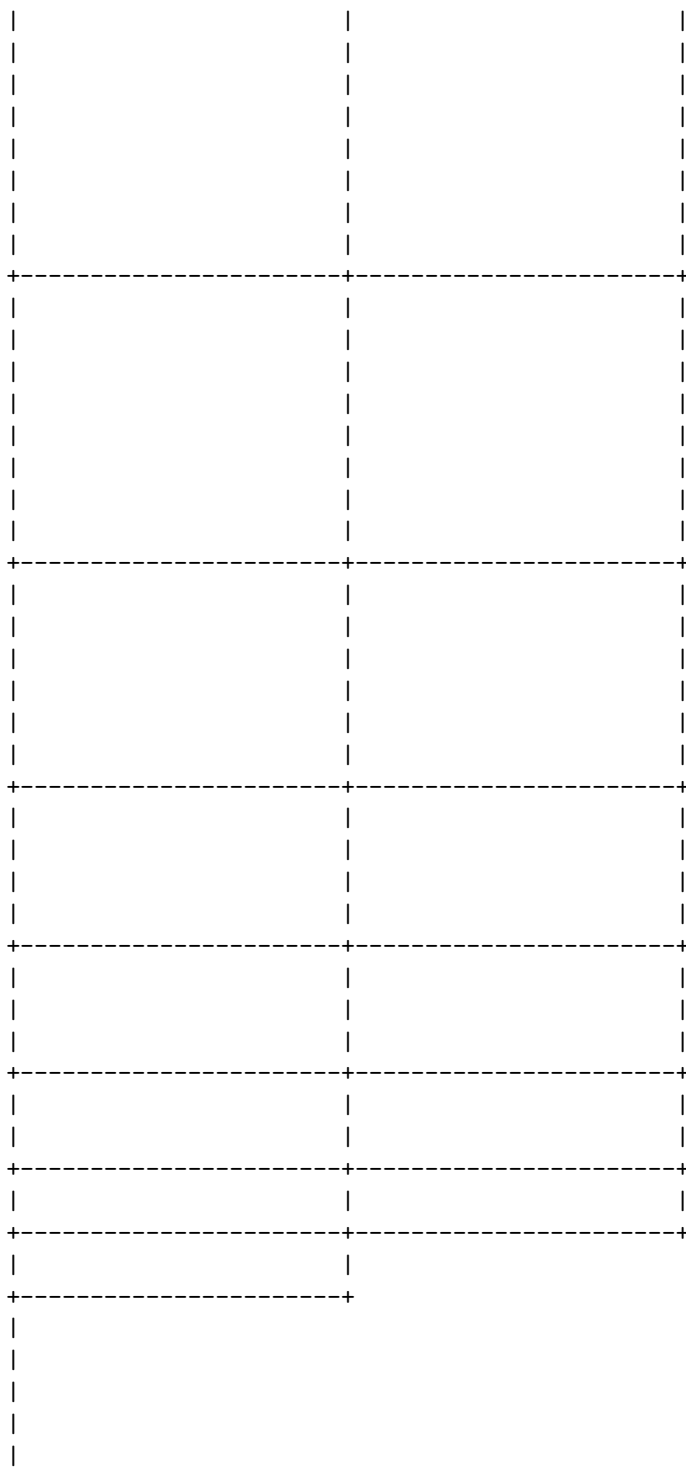
The history of the web is the story of how the World Wide Web (WWW) was invented and developed by Tim Berners-Lee and others at CERN, an international scientific organization based in Geneva, Switzerland, in 1989 . The web was originally conceived and developed to meet the demand for automated information-sharing between scientists in universities and institutes around the world. The web is based on several concepts and technologies, the most fundamental of which are the connections that exist between information, called hyperlinks, and the protocol that enables the communication between servers and clients, called HTTP .

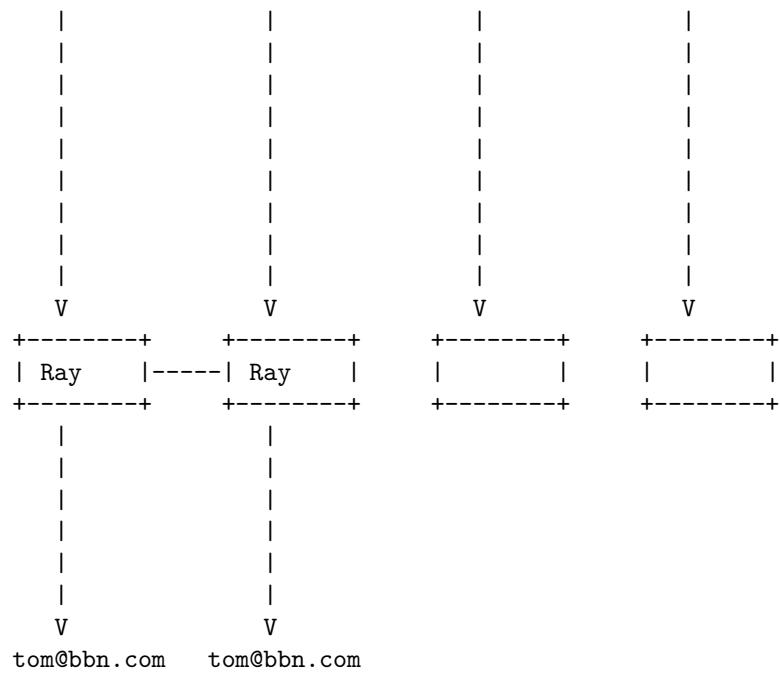
The following diagram illustrates the basic architecture of the web using ASCII characters:

History of Web

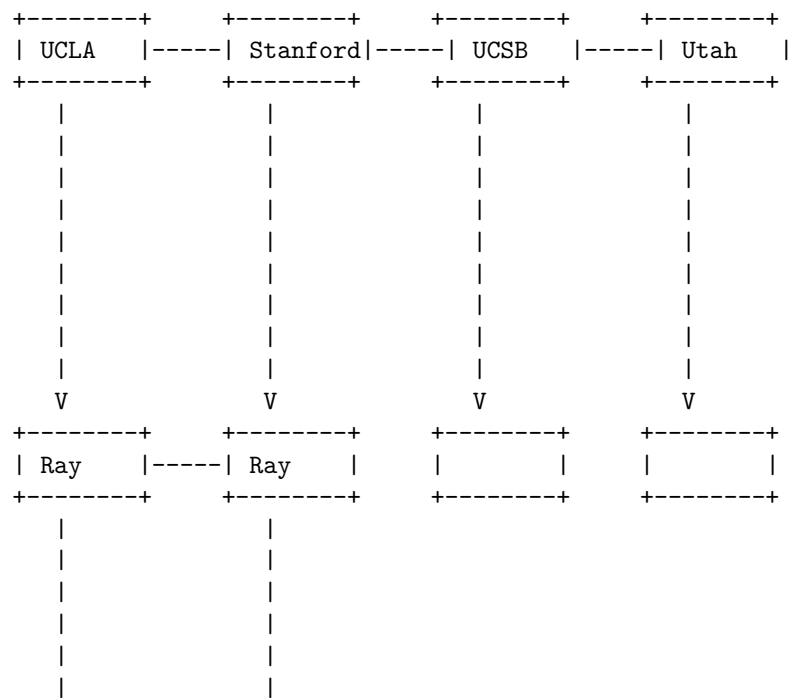


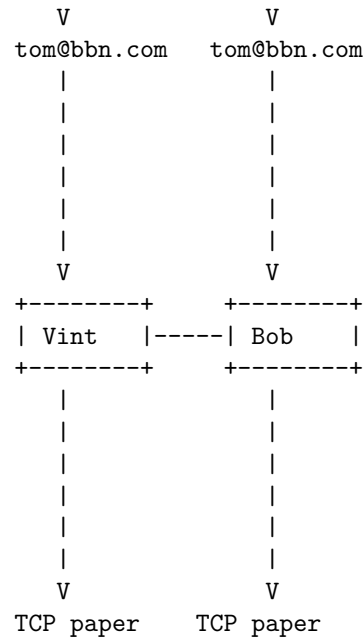






1974: Vinton Cerf and Robert Kahn publish a paper on Transmission Control Protocol (TCP), with





1983: ARPANET adopts TCP/IP as the standard protocol, marking the birth of the internet as we know it.

Protocols are a set of rules that govern the communication between different applications or devices on the web. The web uses various protocols to enable the exchange of data, such as text, images, audio, video, etc. Some of the common protocols that govern the web are:

- **TCP/IP (Transmission Control Protocol/Internet Protocol):** This is the fundamental protocol that defines how data is transmitted and routed over the internet. It consists of two layers: the IP layer, which handles the addressing and delivery of packets, and the TCP layer, which ensures the reliability and order of data transmission.
- **DNS (Domain Name System):** This is the protocol that translates domain names, such as `www.google.com`, into IP addresses, such as `142.250.64.100`, that can be understood by computers. It uses a hierarchical system of servers that store and update the mappings between domain names and IP addresses.
- **HTTP (HyperText Transfer Protocol):** This is the protocol that defines how web browsers and web servers communicate and exchange web resources, such as webpages, images, etc. It uses a request-response model, where the browser sends a request for a resource to the server, and the server responds with the resource or an error message.
- **HTTPS (HyperText Transfer Protocol Secure):** This is the protocol that adds encryption and authentication to HTTP, to ensure the security and privacy of web communication. It uses SSL (Secure Sockets Layer) or TLS (Transport Layer Security) protocols to encrypt the data and verify the

identity of the server and the client.

- FTP (File Transfer Protocol): This is the protocol that enables the transfer of files between computers on the web. It uses a client-server model, where the client initiates a connection to the server and requests to upload or download files. The server then grants or denies the access and transfers the files accordingly.

Protocols Governing Web

The following diagram illustrates the basic architecture of the web and the protocols involved in each layer:

Web Browser	Web Server	DNS Server
HTTPS	HTTPS	DNS
SSL	SSL	
TCP	TCP	UDP
IP	IP	IP
Ethernet	Ethernet	Ethernet

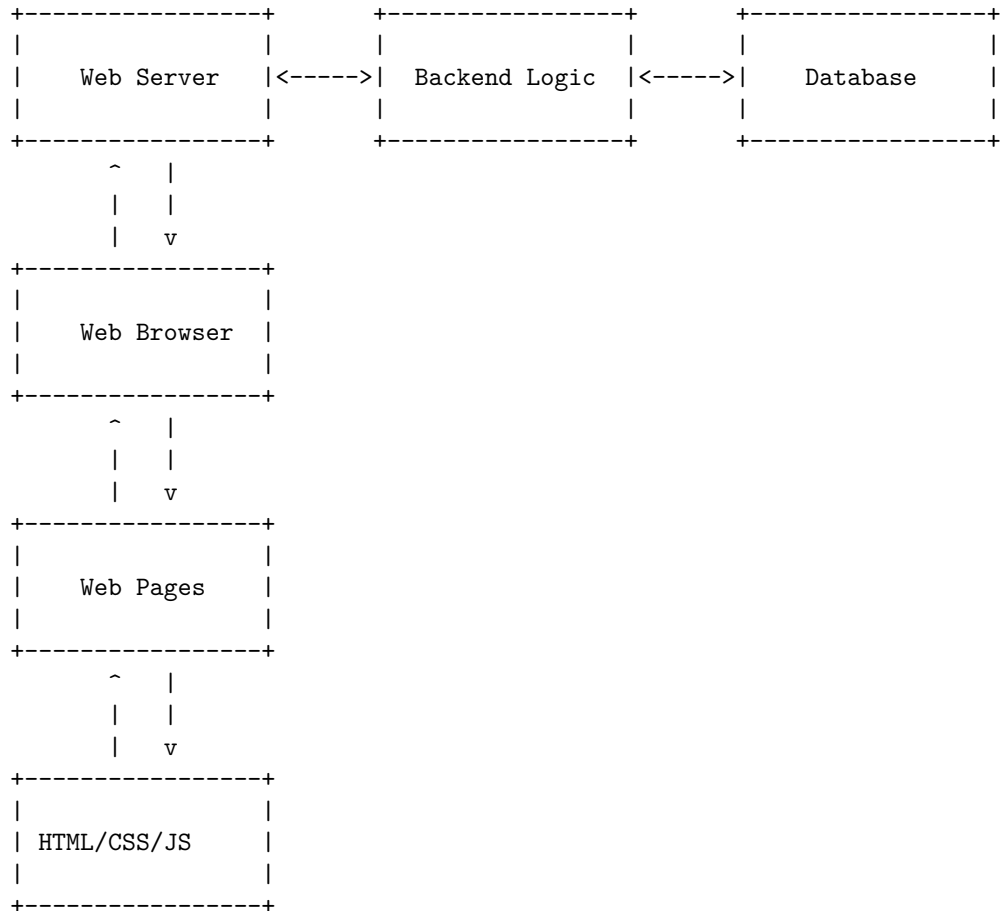
A web project is a collection of files and resources that are used to create a website or a web application. A web project typically consists of the following components:

- HTML files that define the structure and content of the web pages
- CSS files that define the style and layout of the web pages
- JavaScript files that define the behavior and interactivity of the web pages
- Images, fonts, icons, and other media files that are used in the web pages
- A web server that hosts the web project and responds to requests from

browsers

- A database that stores the data and information for the web project
- A backend framework or language that handles the logic and communication between the web server and the database

The following diagram illustrates the basic architecture of a web project using ASCII art:

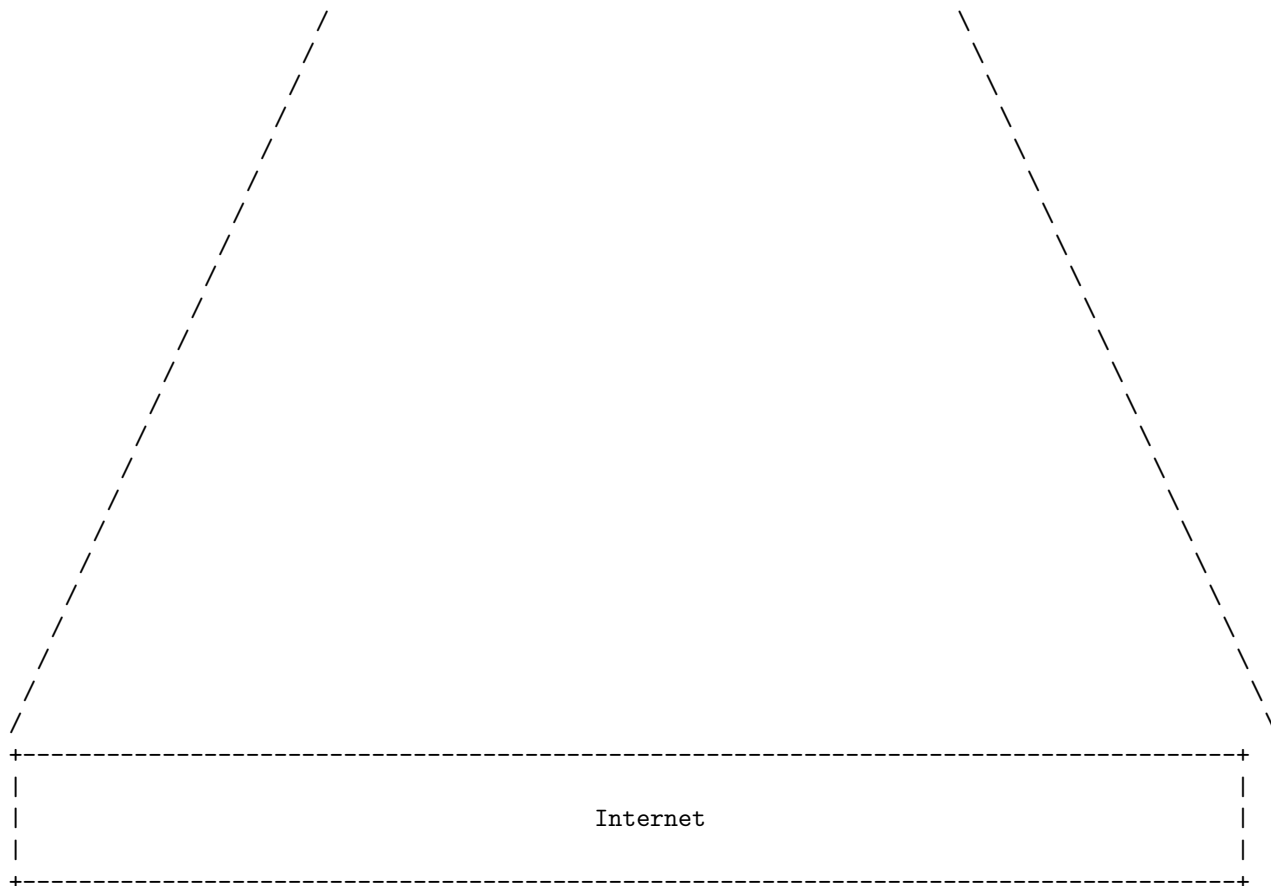


To write a web project, you need to have some knowledge of the following:

- HTML, CSS, and JavaScript for creating the web pages
- A web server software such as Apache, Nginx, or IIS for hosting the web project
- A backend framework or language such as PHP, Python, Ruby, or Node.js for handling the logic and communication
- A database system such as MySQL, MongoDB, or PostgreSQL for storing the data and information

- To draw a detailed ASCII diagram for connecting to the internet, you can use a web-based tool like ASCIIFlow or Textik, or a text editor like Vim with a plugin like DrawIt. You can use different characters to represent different components and connections, such as +, -, |, /, , _ , =, *, #, etc. Here is an example of a possible diagram:

The diagram illustrates a network topology. At the top, three rectangular boxes are arranged horizontally, each containing the text 'Computer', 'Router', and 'Modem' respectively. These boxes are connected by dashed lines, forming a linear sequence. Below this sequence, a large dashed triangle is drawn, with its base at the bottom and its apex pointing upwards towards the 'Modem' box. The triangle is formed by two dashed lines extending from the bottom corners of the diagram towards the 'Modem' box, and a third dashed line connecting the two extension points at the bottom.



Introduction to Internet services

Internet services are the applications or functions that allow us to access and exchange information over the internet. The internet is a global network of interconnected computers that use a common set of protocols to communicate and share data. There are four main categories of internet services, as shown in the following diagram:

Communication Services	Information Services	World Wide Web Services	Multimedia Services
- Email	- FTP	- HTTP	- Streaming
- Chat	- Telnet	- HTML	- Podcasting

- VoIP		- Gopher		- XML		- Webcasting	
- Video		- Archie		- CSS		- Online gaming	
conferencing		- Veronica		- JavaScript			
- Social		- WAIS		- PHP			
networking				- ASP			
				- CGI			
+-----+		+-----+		+-----+		+-----+	

Communication services allow us to exchange information with individuals or groups using text, voice, or video. Examples of communication services are email, chat, VoIP, video conferencing, and social networking.

Information services allow us to access or upload files and data over the internet. Examples of information services are FTP, Telnet, Gopher, Archie, Veronica, and WAIS.

World Wide Web services allow us to access and create web pages and applications using various languages and formats. Examples of World Wide Web services are HTTP, HTML, XML, CSS, JavaScript, PHP, ASP, and CGI.

Multimedia services allow us to access and create audio and video content over the internet. Examples of multimedia services are streaming, podcasting, webcasting, and online gaming.

Introduction to Internet tools

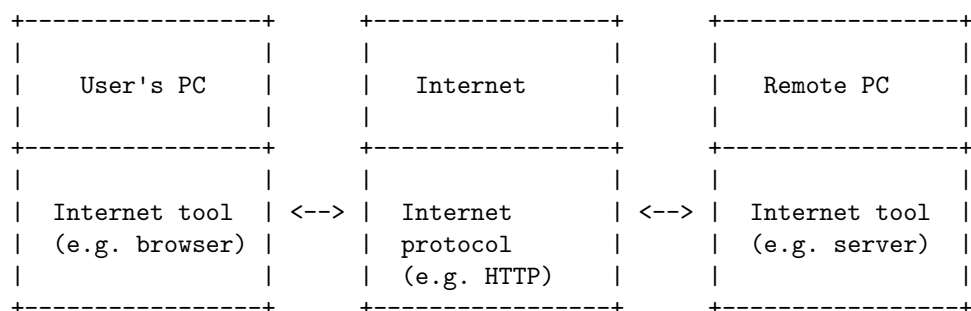
Internet tools are programs or applications that allow users to access, use, and communicate over the Internet. The Internet is a global network of interconnected computers that exchange information using standardized protocols. Some of the most common Internet tools are:

- Web browsers: These are software programs that allow users to view web pages and navigate the World Wide Web. Web browsers can also support multimedia content, such as images, audio, and video. Examples of web browsers are Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.
- Search engines: These are web-based tools that allow users to find information on the Internet by entering keywords or phrases. Search engines use algorithms to rank and display the most relevant web pages for a given query. Examples of search engines are Google, Bing, Yahoo, and DuckDuckGo.
- Email: This is a service that allows users to send and receive electronic messages over the Internet. Email messages can also contain attachments, such as documents, images, or files. Examples of email providers are Gmail, Outlook, Yahoo Mail, and ProtonMail.
- FTP: This stands for File Transfer Protocol, which is a standard method of transferring files between computers over the Internet. FTP can be used to upload or download files from a remote server, such as a website or a cloud storage service. Examples of FTP clients are FileZilla, WinSCP,

and Cyberduck.

- E-commerce: This is the activity of buying and selling goods or services online. E-commerce websites allow users to browse, select, and pay for products or services using a web browser. Examples of e-commerce platforms are Amazon, eBay, Shopify, and Etsy.
- Telnet: This is a protocol that allows users to remotely access another computer over the Internet. Telnet can be used to perform various tasks, such as managing servers, configuring routers, or testing network connections. Examples of telnet clients are PuTTY, Tera Term, and SecureCRT.

The following diagram illustrates the basic architecture of a typical Internet tool:

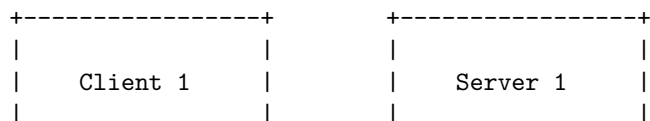


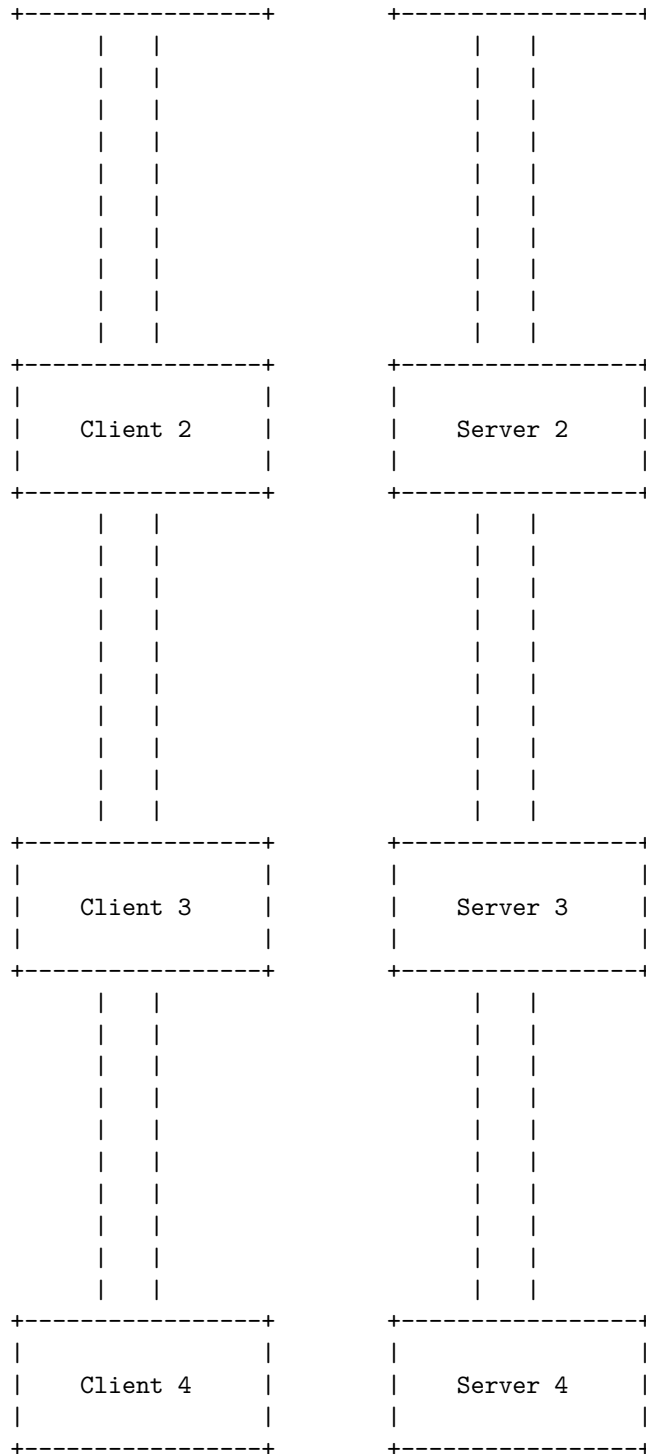
The user's PC runs an Internet tool that allows the user to interact with the Internet. The Internet tool communicates with the Internet using a specific protocol, such as HTTP for web browsing or FTP for file transfer. The Internet protocol defines the rules and formats for exchanging data between computers. The Internet is a network of networks that connects the user's PC to the remote PC. The remote PC also runs an Internet tool that responds to the user's requests or commands. The remote PC can be a web server, an email server, an FTP server, or any other type of computer that provides a service over the Internet.

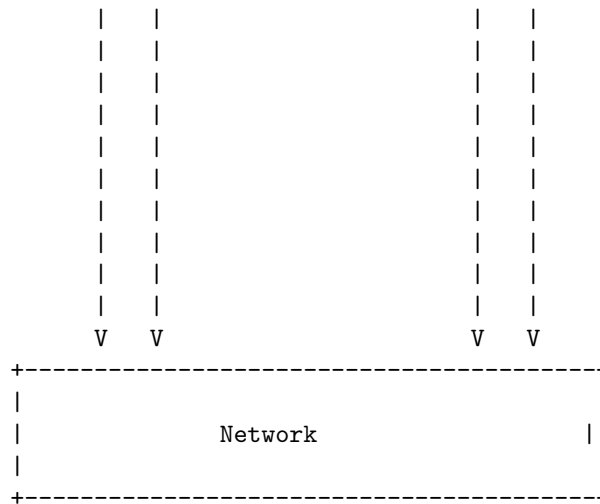
Introduction to client-server computing

Client-server computing is a form of distributed computing that involves a client process requesting services from a server process. The client and the server can be located on different machines connected by a network, or they can be on the same machine. The client is responsible for the user interface and the application logic, while the server is responsible for providing the data and the services.

The following diagram illustrates the basic architecture of a client-server system:







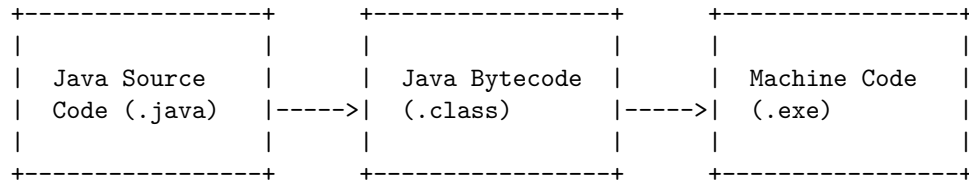
Some of the advantages of client-server computing are:

- Data security and integrity are improved as the data is stored and managed by the server.
- Data access and performance are improved as the server can handle multiple requests from different clients efficiently.
- Scalability and flexibility are enhanced as new clients and servers can be added or removed without affecting the existing system.
- Maintenance and administration are simplified as the server can be updated or repaired centrally without affecting the clients.

Core Java is the basic and core part of the Java programming language that is used for creating or developing a general-purpose application. Core Java architecture consists of three main components: JVM, JRE, and JDK.

Core Java

JDK	JRE	JVM
Java Compiler	Java Libraries	Class Loader
Java Debugger	Java Runtime Environment	Runtime Memory
Java Tools		Execution Engine



JDK stands for Java Development Kit. It is a software development environment that contains the Java compiler, debugger, tools, and libraries. It is used to create Java applications and applets.

JRE stands for Java Runtime Environment. It is a software package that contains the Java libraries, the Java runtime, and other components that are required to run Java applications. It is a subset of JDK.

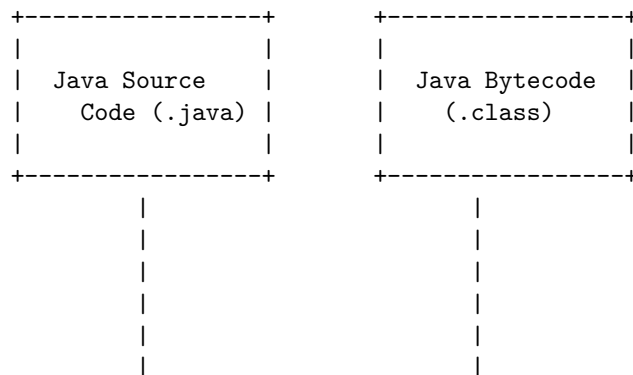
JVM stands for Java Virtual Machine. It is a software component that converts the Java bytecode into machine code and executes it on the underlying hardware. It provides platform independence and memory management for Java applications. It consists of a class loader, a runtime memory area, and an execution engine.

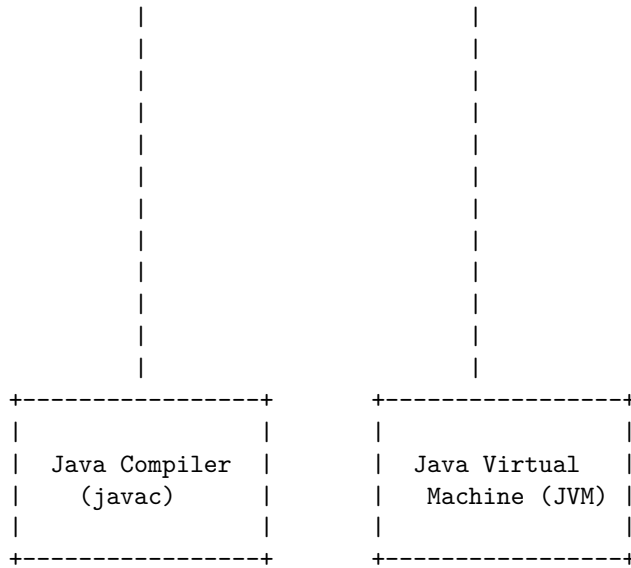
Introduction to Java

Java is a class-based, object-oriented, general-purpose, high-level programming language that was developed by James Gosling and his team at Sun Microsystems in 1991 . It is designed to have as few implementation dependencies as possible, which means that compiled Java code can run on any platform that supports Java without recompilation. This feature is known as “write once, run anywhere” (WORA).

Java is widely used for various applications, such as mobile applications (especially Android apps), desktop applications, web applications, web servers, and application servers . It is also one of the most popular programming languages in the world, with millions of developers and billions of devices running Java.

The following diagram illustrates the basic architecture of a Java application:





The Java compiler (javac) converts the Java source code (.java) into Java bytecode (.class), which is an intermediate representation of the program that can be executed by the Java Virtual Machine (JVM). The JVM is a software layer that interprets and runs the Java bytecode on different platforms, such as Windows, Linux, Mac OS, etc.. The JVM also provides various features, such as memory management, garbage collection, security, and exception handling.

Java is also a platform, which consists of a set of libraries and tools that support the development and execution of Java applications. The Java platform is divided into three main editions: Java Standard Edition (Java SE), Java Enterprise Edition (Java EE), and Java Micro Edition (Java ME). Each edition provides a different set of APIs and functionalities for different types of applications.

Java is a simple, robust, portable, secure, and dynamic programming language that has many advantages over other languages. Some of the benefits of Java are:

- It is easy to learn and use, as it has a clear and concise syntax and follows the object-oriented paradigm.
- It is platform-independent, as it can run on any system that has a JVM installed.
- It is fast and efficient, as it uses bytecode that is optimized for the JVM.
- It is secure, as it has a built-in mechanism to prevent unauthorized access and malicious code.
- It is dynamic, as it supports features such as reflection, dynamic loading, and runtime binding.
- It is scalable, as it can handle large and complex applications with ease.

- It is versatile, as it can be used for various types of applications, such as web, mobile, desktop, etc.

Java is a powerful and popular programming language that has a lot to offer to developers and programmers. If you want to learn more about Java, you can check out some of the online courses and tutorials available on the web. Happy coding!

An operator in Core Java is a symbol that is used to perform operations on data. There are many types of operators in Core Java, such as arithmetic, unary, assignment, relational, logical, bitwise, ternary, and shift operators. Each operator has a different precedence and associativity, which determines the order of evaluation of expressions.

The following diagram illustrates the basic architecture of an operator in Core Java using ASCII characters:

Operator Type	Symbol/Keyword	Example
Arithmetic	+ - * / % ++ --	a + b, c / d
Unary	+ - ! ~ ++ --	-a, !b, c++
Assignment	= += -= *= /= %=	a = b, c += d
Relational	== != > < >= <=	a == b, c < d
Logical	&& !	a && b, !c
Bitwise	& ^ ~ << >> >>>	a & b, c ^ d
Ternary	? :	a ? b : c
Shift	<< >> >>>	a << b, c >>> d

A data type in Core Java specifies the type and size of a value that a variable can store or an expression can return. There are two types of data types in Core Java: primitive and non-primitive.

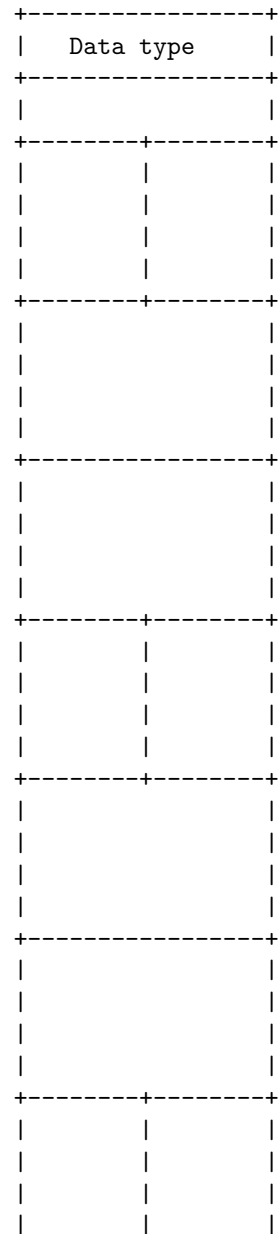
Primitive data types are the basic types that are predefined and supported by the Java language. They include boolean, char, byte, short, int, long, float and double. Each primitive data type has a fixed range of values and a default value.

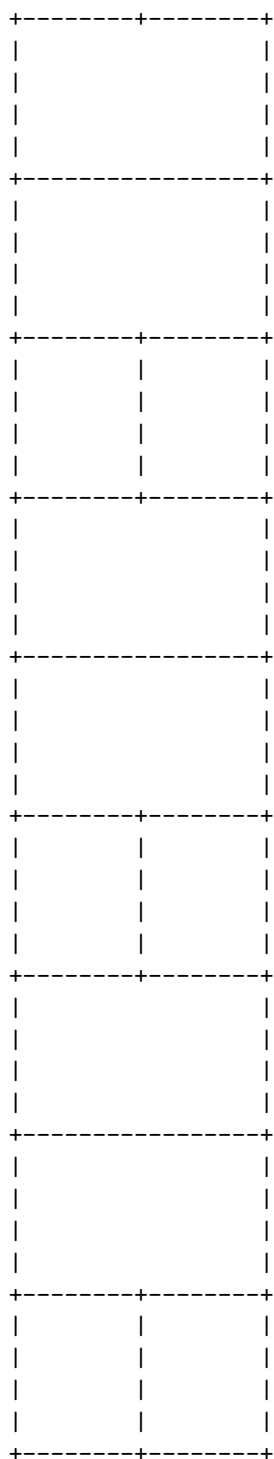
Non-primitive data types are the types that are defined by the programmer or the Java API. They include classes, interfaces, arrays, strings, enums, etc. Non-primitive data types are reference types, which means they store the address of the object they refer to, not the actual value. Non-primitive data types can

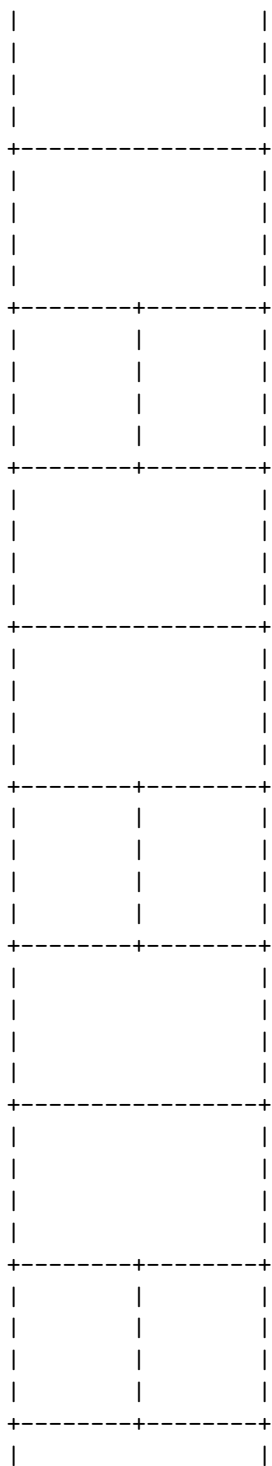
have null as their default value.

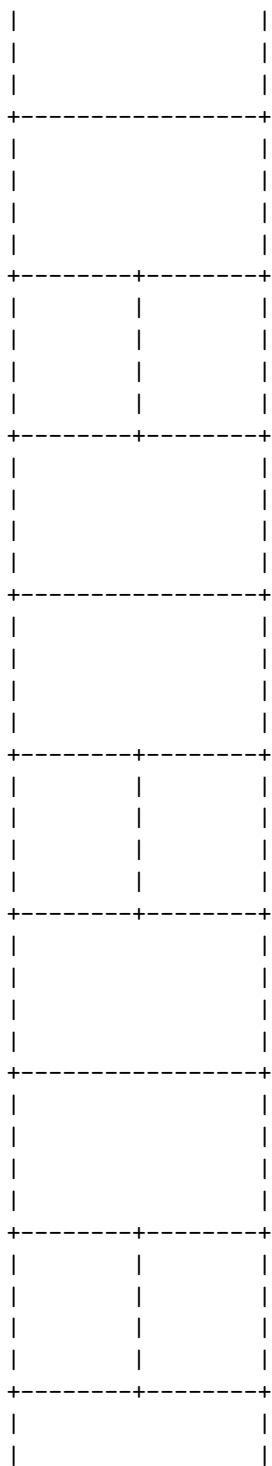
Data type in Core Java

The following diagram illustrates the data type hierarchy in Core Java using ASCII art:









Class	
Interface	
Array	
Enum	
+-----+	

A variable in Core Java is a data container that saves the data values during Java program execution. Every variable is assigned a data type that designates the type and quantity of value it can hold. A variable is a name given to a memory location. It is the basic unit of storage in a program.

Variables in Java can be defined anywhere in the code (inside a class, inside a method, or as a method argument) and can have different modifiers. Depending on these conditions variables in Java can be divided into four categories:

- **Instance Variable:** A variable that is declared inside a class but outside a method is known as an instance variable. It is not declared as static. It is called instance variable because its value is instance specific and is not shared among instances.
- **Static Variable:** A variable that is declared as static is known as static variable. It cannot be local. You can create a single copy of static variable and share among all the instances of the class. Memory allocation for static variable happens only once when the class is loaded in the memory.
- **Local Variable:** A variable that is declared inside the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists. A local variable cannot be defined with static keyword.
- **Parameter Variable:** A variable that is declared inside the parenthesis of the method is called parameter variable. It is used to pass the value to the method.

The following diagram illustrates the basic architecture of a variable in Core Java:

Variable in Core Java

+-----+			
Class Name	Instance Variable	Static Variable	Local Variable
+-----+			
class Example	int x;	static int y;	void method()
			{
			int z;
			}

Memory Allocation	Heap Memory	Static Memory	Stack Memory
Value Sharing	Instance Specific	Shared Among All Instances	Method Specific

An array in Java is an object that can store a fixed number of values of the same type. The values are called elements and they are accessed by their index, which starts from 0. An array can be declared as a static field, a local variable, or a method parameter. An array can store primitive values or objects, and can be single-dimensional or multi-dimensional.

Arrays in Core Java

The following diagram illustrates the basic structure of a single-dimensional array in Java:

0	1	2	3	4	5	6	7	8	9
5	8	3	9	6	2	4	7	1	0

The array has 10 elements, each of type `int`. The index of the first element is 0 and the index of the last element is 9. The value of the element at index `i` is shown below the index. For example, the value of the element at index 3 is 9.

To declare an array in Java, we use the following syntax:

```
type[] arrayName; // declare an array of type
arrayName = new type[size]; // create an array of size
```

For example, to declare and create the array shown in the diagram, we can write:

```
int[] numbers; // declare an array of int
numbers = new int[10]; // create an array of 10 int
```

To assign values to the elements of the array, we can use the following syntax:

```
arrayName[index] = value; // assign value to the element at index
```

For example, to assign 5 to the first element of the array, we can write:

```
numbers[0] = 5; // assign 5 to the element at index 0
```

To access the values of the elements of the array, we can use the same syntax:

```
value = arrayName[index]; // get the value of the element at index
```

For example, to get the value of the fourth element of the array, we can write:

```
int x = numbers[3]; // get the value of the element at index 3 and store it in x
```

A multi-dimensional array in Java is an array of arrays. The elements of a multi-dimensional array are arrays themselves. Each array can have a different length, which makes the multi-dimensional array a jagged array. A multi-dimensional array can be declared and created as follows:

```
type[] [] arrayName; // declare a two-dimensional array of type  
arrayName = new type[size1] []; // create a two-dimensional array of size1  
for (int i = 0; i < size1; i++) {  
    arrayName[i] = new type[size2]; // create an array of size2 for each element of the two-d  
}
```

For example, to declare and create a two-dimensional array of int with 3 rows and 4 columns, we can write:

```
int[] [] matrix; // declare a two-dimensional array of int  
matrix = new int[3] []; // create a two-dimensional array of 3 rows  
for (int i = 0; i < 3; i++) {  
    matrix[i] = new int[4]; // create an array of 4 columns for each row  
}
```

The following diagram illustrates the structure of the two-dimensional array:

```
+---+---+---+---+  
| 0 | 1 | 2 | 3 |  
+---+---+---+---+  
| 1 | 2 | 3 | 4 |  
+---+---+---+---+  
| 5 | 6 | 7 | 8 |  
+---+---+---+---+  
| 9 | 10| 11| 12|  
+---+---+---+---+
```

The array has 3 rows and 4 columns, each element of type int. The index of the first row is 0 and the index of the last row is 2. The index of the first column is

A class in Java is a blueprint that defines the attributes and behaviors of an object. A class can contain fields, which are variables that store data, and methods, which are functions that perform actions on the object. An object is an instance of a class that can be created and manipulated at runtime. A class can also have constructors, which are special methods that initialize the object when it is created.

Methods & Classes in Core Java

The following diagram shows a simple example of a class and an object in Java:

```
+-----+
|      Person      | <-- This is a class
+-----+
| - name : String   | <-- This is a field
| - age  : int      |
+-----+
| + Person(name, age) | <-- This is a constructor
| + getName() : String| <-- This is a method
| + getAge() : int   |
| + setName(name)    |
| + setAge(age)       |
+-----+

+-----+
|      p1          | <-- This is an object
+-----+
| - name : "Alice"  | <-- This is a value
| - age  : 25       |
+-----+
| + Person(name, age) |
| + getName() : String|
| + getAge() : int   |
| + setName(name)    |
| + setAge(age)       |
+-----+
```

The class `Person` has two fields, `name` and `age`, and five methods, a constructor and four getters and setters. The object `p1` is created by using the `new` keyword and calling the constructor with the values “Alice” and 25. The object `p1` can access the fields and methods of the class `Person` by using the dot operator (`.`). For example, `p1.getName()` will return “Alice” and `p1.setAge(26)` will change the value of the `age` field to 26.

Inheritance in Java is one of the core concepts of Object-Oriented Programming. It enables a class to inherit the properties and methods of another class. The class that inherits is called the subclass or child class, and the class that is inherited from is called the superclass or parent class. The subclass can access the members of the superclass, and also add its own members. The subclass can also override the methods of the superclass, to provide a different implementation. Inheritance in Java is implemented using the `extends` keyword.

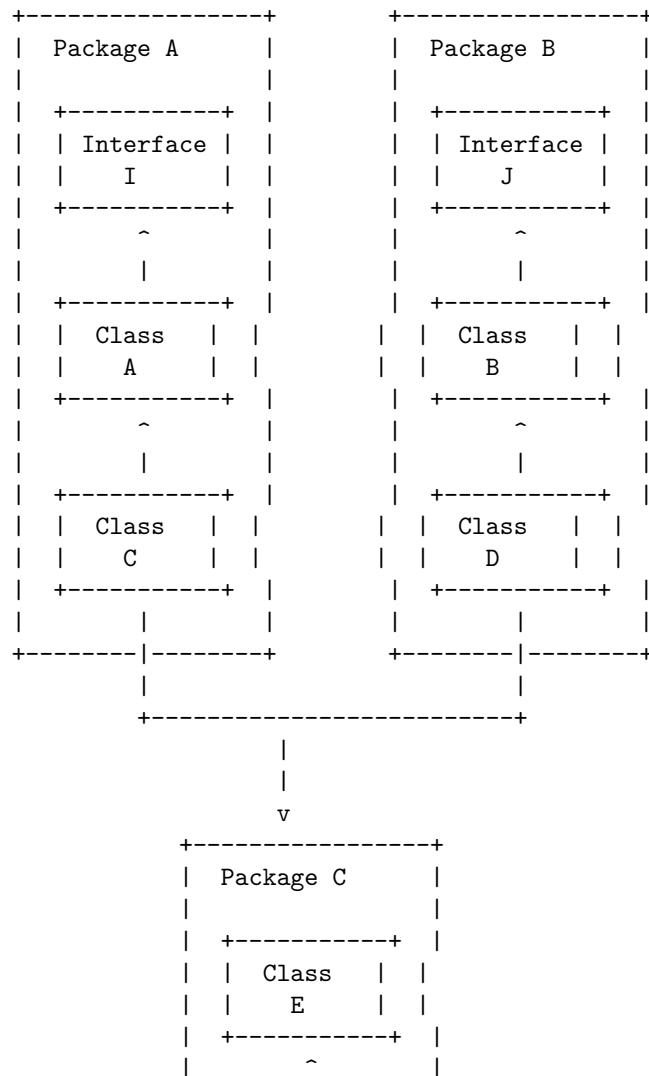
The following diagram illustrates the basic concept of inheritance in Java using ASCII art:

Inheritance in Java

Package and Interface in Core Java

A package is a group of classes and interfaces that are related in some way. A package helps to organize the code and avoid naming conflicts. An interface is a group of abstract methods that define a contract or a behavior that a class can implement. An interface helps to achieve abstraction and polymorphism in Java.

The following diagram illustrates the basic architecture of a package and an interface in Core Java using ASCII art:



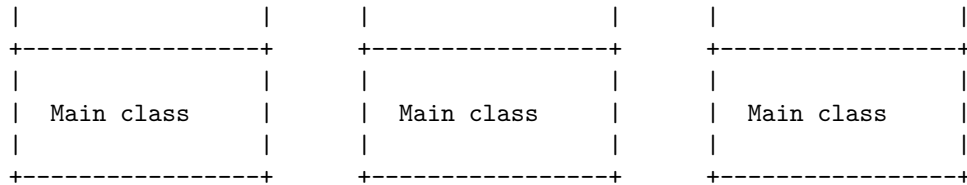
throws	throw	
Keyword to	Keyword to	
declare the	create and	
exceptions	throw an	
that a method	exception	
may throw	object	

Multithreading in Java is a process of executing multiple threads simultaneously to maximize the utilization of CPU. A thread is a lightweight sub-process, the smallest unit of processing. Multithreading can be achieved by two ways: extending the Thread class or implementing the Runnable interface.

Multithread programming in Core Java

The following diagram illustrates the basic architecture of a multithreaded program in Core Java:

Main Thread	Thread 1	Thread 2
run() method	run() method	run() method
start() method	start() method	start() method
Thread class	Thread class	Thread class
Runnable interface	Runnable interface	Runnable interface



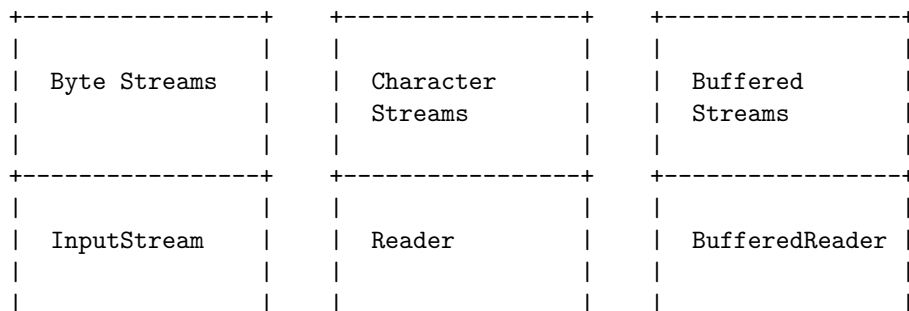
The main thread is created by the Java Virtual Machine (JVM) when the program starts. It executes the `main()` method of the main class. The main thread can create other threads by instantiating the `Thread` class or a subclass of it, and passing a `Runnable` object to the constructor or the `setRunnable()` method. The `Runnable` object defines the `run()` method that contains the code to be executed by the thread. The `start()` method of the `Thread` class invokes the `run()` method in a separate execution path.

The threads can communicate with each other by using shared variables, synchronized blocks, or inter-thread communication methods such as `wait()`, `notify()`, and `notifyAll()`. The threads can also be controlled by using methods such as `sleep()`, `join()`, `yield()`, `interrupt()`, and `stop()`.

The multithreading programming in Core Java can improve the performance and responsiveness of the program by utilizing the available CPU cores and allowing the program to perform multiple tasks concurrently. However, it also introduces some challenges such as thread safety, deadlock, race condition, and memory consistency errors. Therefore, it requires careful design and testing to ensure the correctness and efficiency of the program.

I/O in Core Java refers to the input and output operations performed by Java programs using the classes and interfaces in the `java.io` package. Java uses the concept of a stream to make I/O operations fast and efficient. A stream is a sequence of data that can be read from a source or written to a destination. Streams can be byte-oriented or character-oriented, depending on the type of data they handle. Byte-oriented streams are used for binary data, such as images, audio, or video. Character-oriented streams are used for text data, such as documents, web pages, or source code.

The following diagram illustrates the basic architecture of I/O in Core Java using ASCII characters:



OutputStream	Writer	BufferedWriter
FileInputStream	FileReader	PrintWriter
FileOutputStream	FileWriter	PrintStream
DataInputStream	InputStreamReader	LineNumberReader
DataOutputStream	OutputStreamWriter	PushbackReader

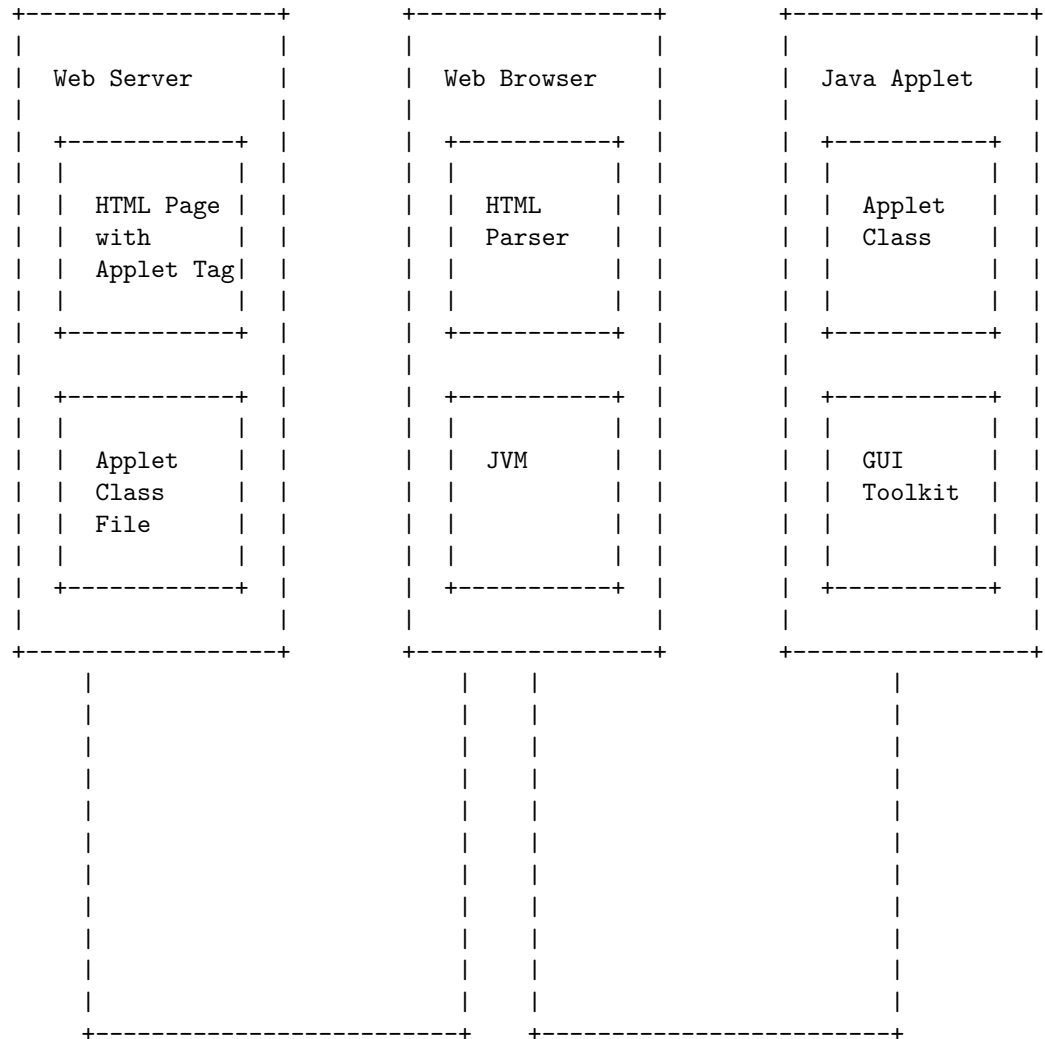
The diagram shows the hierarchy of the classes and interfaces in the java.io package. The byte streams are the base classes for all the other streams. The character streams are wrappers around the byte streams that provide methods for reading and writing text data. The buffered streams are subclasses of the character streams that improve the performance by using internal buffers. The file streams are subclasses of the byte streams that provide methods for reading and writing files. The data streams are subclasses of the byte streams that provide methods for reading and writing primitive data types and strings. The print streams are subclasses of the character streams that provide methods for printing formatted output. The input stream reader and output stream writer are bridges between the byte streams and the character streams that allow the conversion of bytes to characters and vice versa. The line number reader and pushback reader are subclasses of the buffered reader that provide additional functionality, such as counting the lines and pushing back characters to the stream.

A Java applet is a small Java application that can be accessed on an Internet server, transported over the Internet, and can be automatically installed and run as part of a web document. An applet is a Java class that extends the java.applet.Applet class. An applet does not have a main() method and is

viewed using a Java Virtual Machine (JVM) in a browser .

The following diagram illustrates the basic architecture of a Java applet in core Java:

Java Applet in Core Java



The diagram shows the following steps:

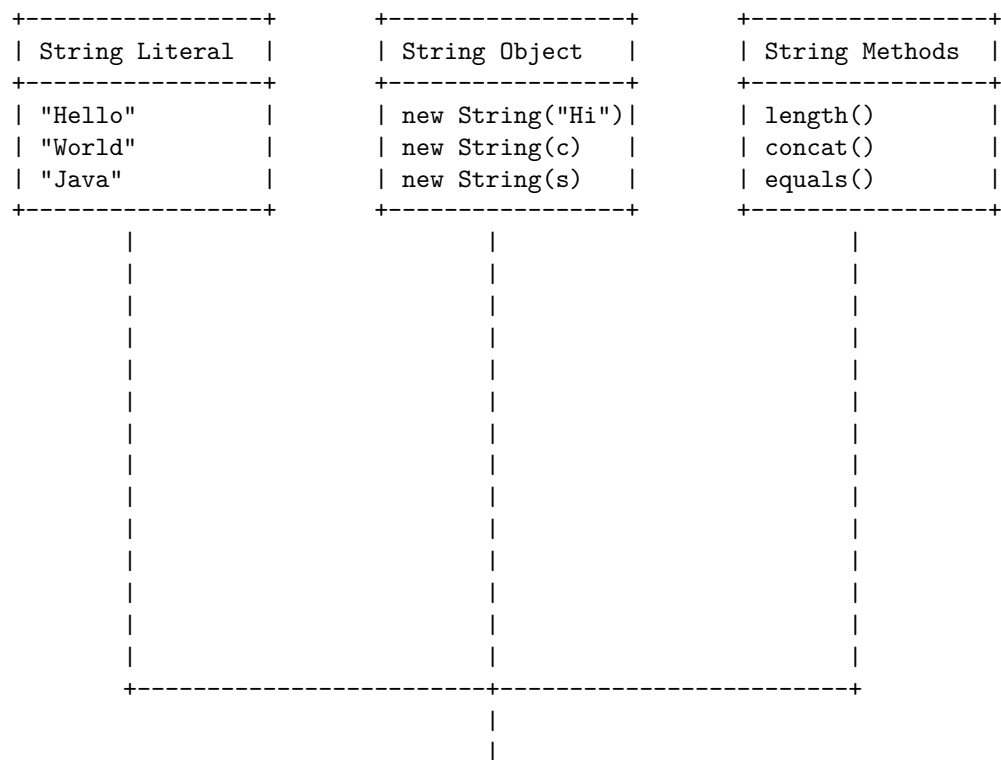
- The web server hosts an HTML page with an applet tag that specifies the applet class file.
- The web browser requests the HTML page from the web server.
- The web browser parses the HTML page and finds the applet tag.

- The web browser requests the applet class file from the web server.
- The web server sends the applet class file to the web browser.
- The web browser loads the applet class file into the JVM .
- The JVM creates an instance of the applet class and invokes its `init()` method to initialize the applet .
- The applet class creates its GUI using the GUI toolkit.
- The JVM invokes the applet's `start()` method to start the applet.
- The applet interacts with the user and performs its tasks.
- The JVM invokes the applet's `stop()` method to stop the applet when the user leaves the web page.
- The JVM invokes the applet's `destroy()` method to release the applet's resources when the browser is closed.

String handling in Core Java

String handling is a way of handling and manipulating strings in Java with the help of various concepts like concatenation, comparison, etc. A string is a sequence of characters that can be created by using the `String` class or by using string literals.

The following diagram illustrates the basic architecture of a string handling in Core Java:



|

|

|

|

|

|

|

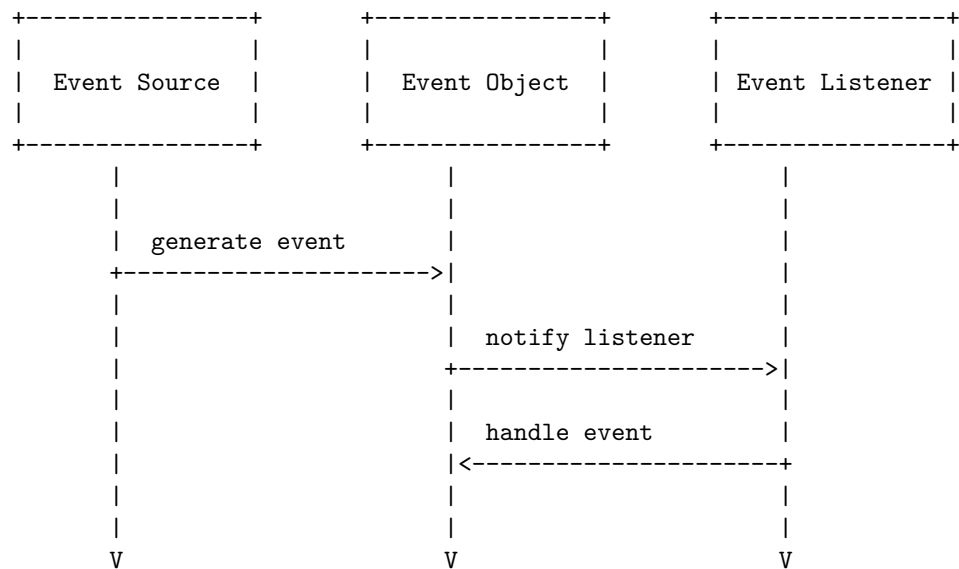
|
|
|
|
|
|

Event handling in Core Java

Event handling in Core Java is the process of controlling an event and performing appropriate action if it occurs. An event is a change in the state of an object or a user action, such as clicking a button, moving the mouse, typing a key, etc. An event handler is a code or a set of instructions that is executed when an event occurs. It consists of two major components: event sources and event listeners.

Event sources are the objects that generate events, such as buttons, text fields, menus, etc. Event listeners are the objects that receive events and handle them, such as action listeners, mouse listeners, key listeners, etc. Event sources and event listeners are connected by a mechanism called event delegation, which allows the event source to delegate the responsibility of handling the event to the event listener.

The following diagram illustrates the basic architecture of event handling in Core Java:



The steps to perform event handling in Core Java are:

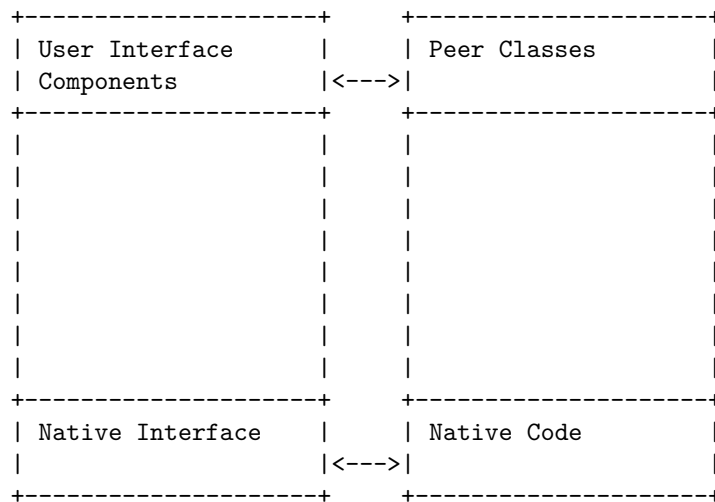
1. Create an event source object and add it to the GUI component.

2. Create an event listener object that implements the appropriate listener interface for the event type.
3. Register the event listener object with the event source object using the `addXXXListener()` method, where XXX is the event type.
4. Define the event handler method in the event listener object that overrides the abstract method of the listener interface. The event handler method takes an event object as a parameter and performs the desired action.

AWT stands for Abstract Window Toolkit, which is an API to develop graphical user interface or window-based applications in Java. AWT components are platform-dependent, meaning that they are displayed according to the view of the operating system. AWT is also heavy-weight, meaning that its components use the resources of the underlying operating system.

The basic architecture of AWT consists of four layers: the user interface components, the peer classes, the native interface, and the native code. The user interface components are the classes that provide the functionality and appearance of the GUI elements, such as buttons, labels, text fields, etc. The peer classes are the classes that communicate with the native interface and the native code to create and manage the actual GUI components on the screen. The native interface is the layer that defines the methods and constants that the peer classes use to interact with the native code. The native code is the layer that contains the platform-specific code that implements the GUI functionality.

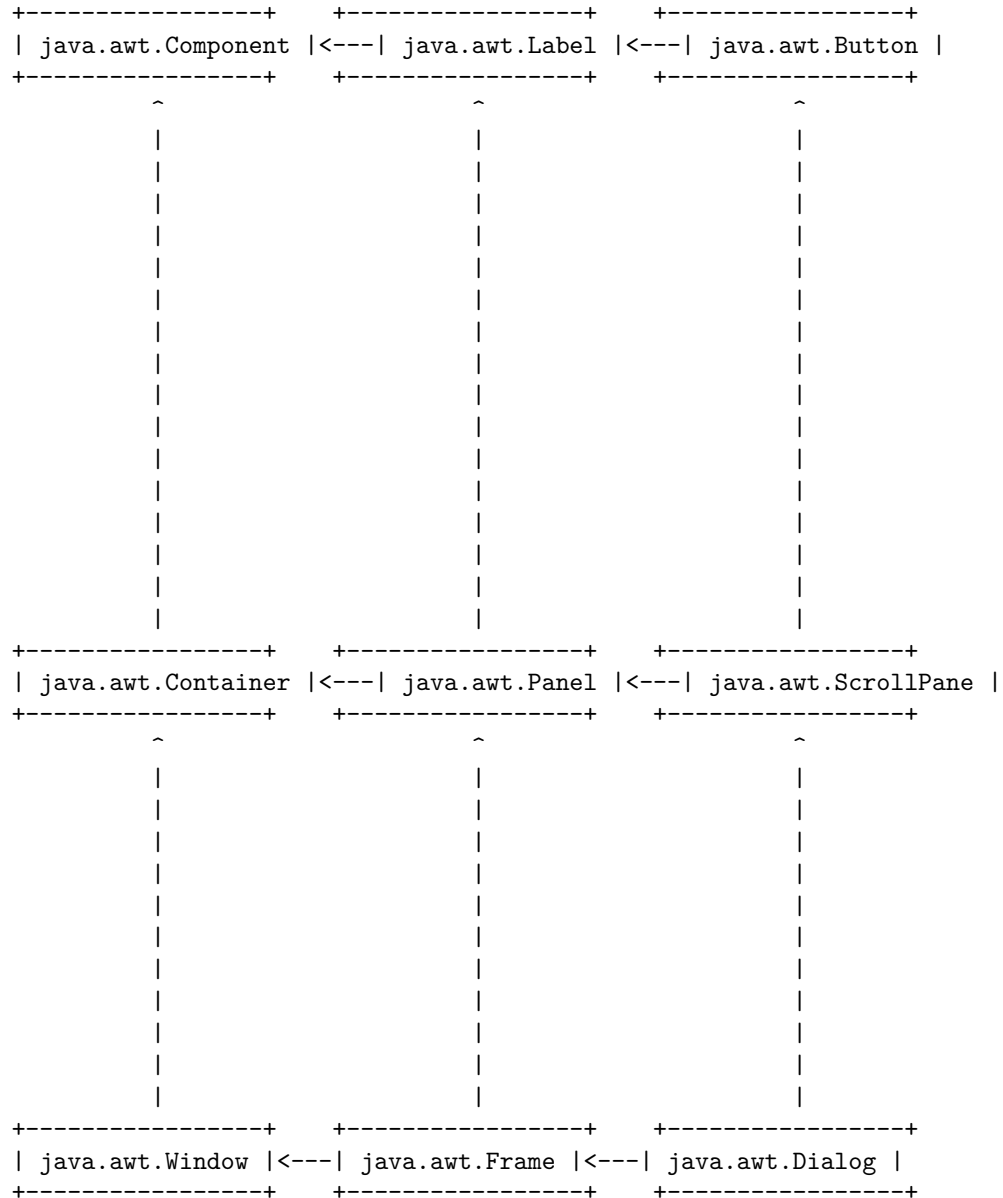
The following diagram illustrates the basic architecture of AWT in Core Java using ASCII art:



AWT controls are components that allow a user to interact with your application in various ways. The AWT supports the following types of controls: Labels, Push buttons, Check boxes, Choice lists, Lists, Scroll bars, and Text Editing .

AWT controls

The following diagram illustrates the basic architecture of AWT controls using ASCII art:

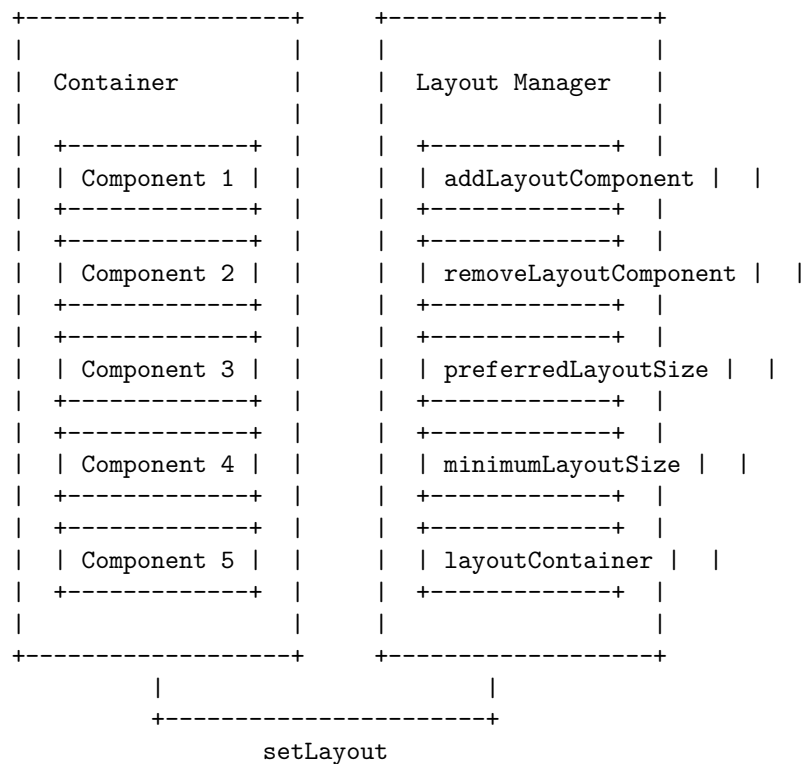


Each control is a subclass of the `java.awt.Component` class, which provides the basic functionality for all components. Some controls, such as `Panel`, `ScrollPane`, `Window`, `Frame`, and `Dialog`, are subclasses of the `java.awt.Container`

class, which allows them to contain other components .

Layout managers in AWT are classes that implement the `java.awt.LayoutManager` interface and determine how the components are arranged in a container. AWT provides five predefined layout managers: `FlowLayout`, `BorderLayout`, `GridLayout`, `CardLayout`, and `GridBagLayout`. Each layout manager has its own advantages and disadvantages, depending on the type of user interface you want to create.

The following diagram illustrates the basic architecture of a layout manager in AWT:



The container is the parent component that holds one or more child components. The container can be a frame, a panel, a dialog, or any other component that can contain other components. The container has a `setLayout` method that takes a layout manager object as an argument and assigns it to the container.

The layout manager is the object that implements the `LayoutManager` interface and defines how the components are arranged in the container. The layout manager has five methods that are invoked by the container when needed:

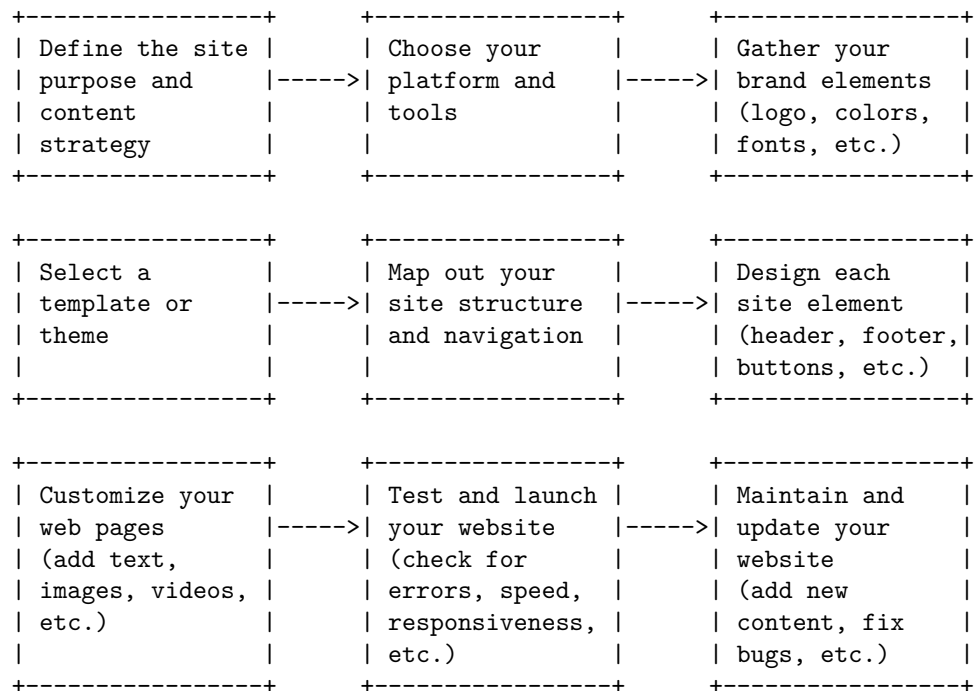
- `addLayoutComponent`: This method is called when a component is added to the container. The layout manager can store any information about the component that is needed for the layout.

- `removeLayoutComponent`: This method is called when a component is removed from the container. The layout manager can remove any information about the component that is no longer needed for the layout.
- `preferredLayoutSize`: This method is called when the container needs to know the preferred size of the layout. The layout manager should calculate and return the size that best fits the components in the container.
- `minimumLayoutSize`: This method is called when the container needs to know the minimum size of the layout. The layout manager should calculate and return the size that is required to display the components in the container.
- `layoutContainer`: This method is called when the container needs to position and resize the components in the layout. The layout manager should set the bounds of each component according to the layout algorithm.

Unit 2 - Web Page Designing

Web page designing is the process of creating websites and pages that reflect a company's brand and information and ensure a user-friendly experience. Web page designing involves organizing content and images across a series of pages and integrating applications and other interactive elements.

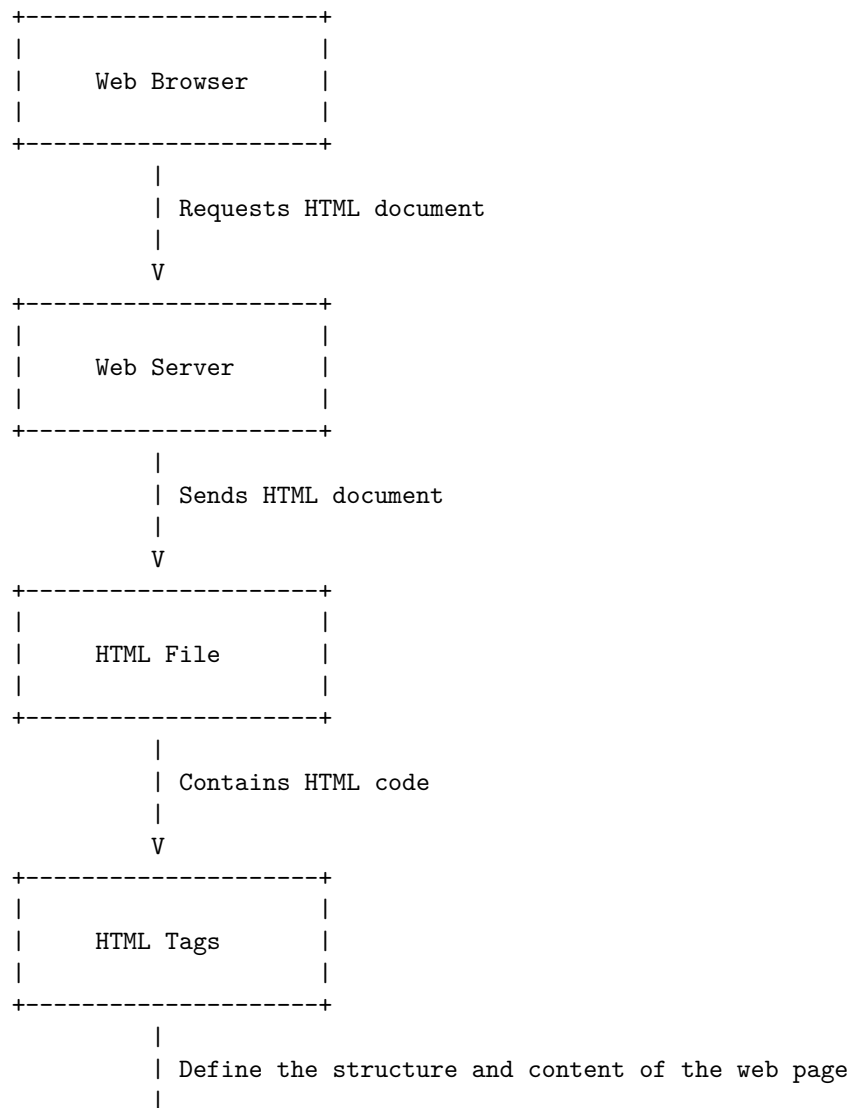
The following diagram illustrates the basic steps of web page designing in ASCII art:

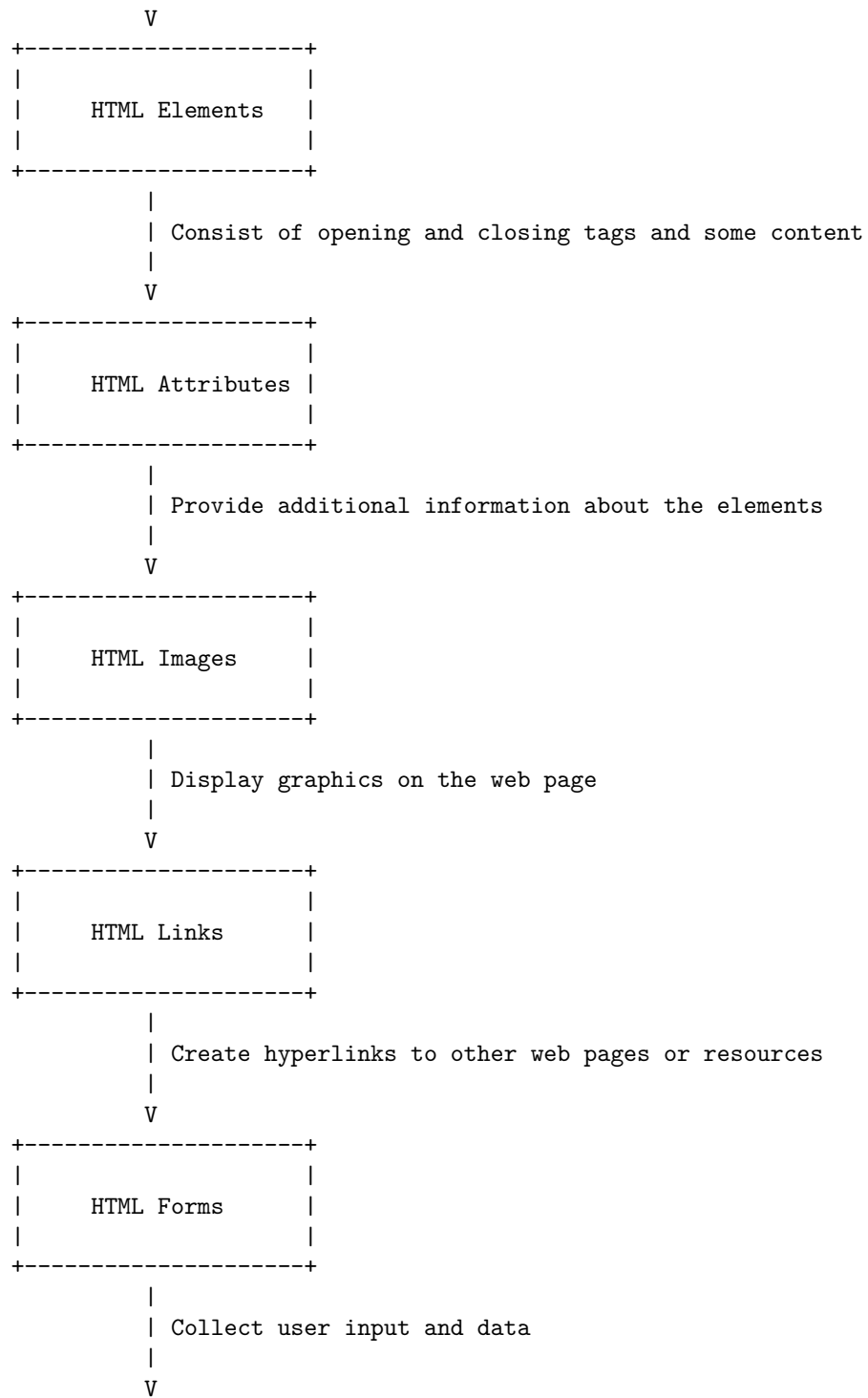


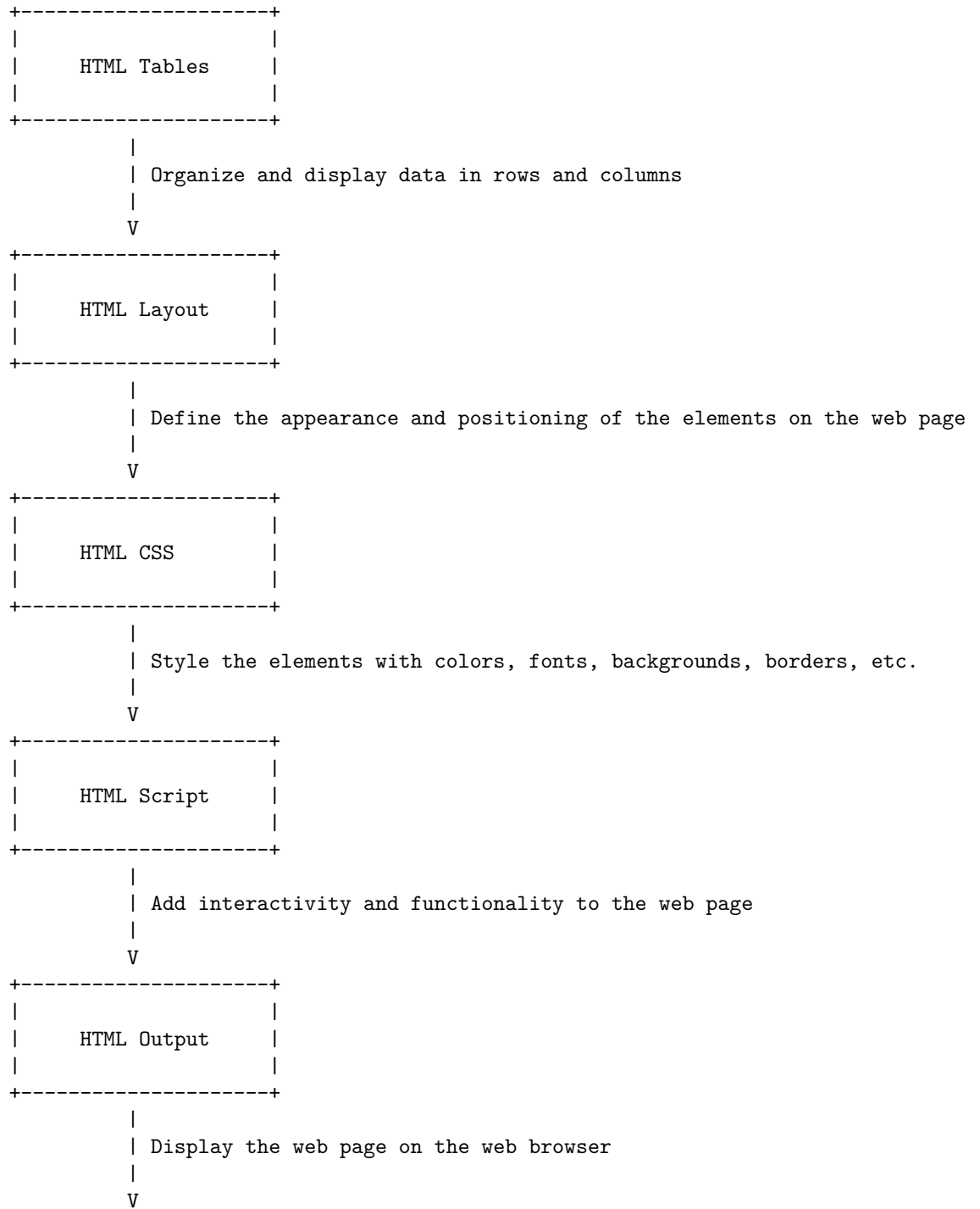
HTML stands for HyperText Markup Language and it is used to design web pages and their content. HTML uses different tags, elements, images and some latest components to make web pages more attractive and user-friendly. HTML also works with CSS (Cascading Style Sheets) to change the look and feel of the web page.

The basic architecture of a web page designed in HTML can be illustrated by the following ASCII diagram:

HTML in Web Page Designing







A list in web page designing is a way of displaying a collection of related items, using either numbered order (ordered list) or bullet points (unordered list). A list can also have sub-lists, which are indented under the main list items. A list can be used to provide users with an overview of and find entities to work with, such as customers, vendors, or sales orders.

The following diagram illustrates the basic architecture of a list in web page designing using markdown syntax:

List in Web Page Designing

- This is an unordered list
 - This is a sub-list item
 - Another sub-list item
 - This is another unordered list item
 - This is a sub-list with a different bullet point
 - This is a sub-sub-list item
 - Another sub-list item
-
1. This is an ordered list
 1. This is a sub-list item
 2. Another sub-list item
 2. This is another ordered list item
 - This is a sub-list with a bullet point
 - This is a sub-sub-list item
 - Another sub-list item

The output of the above markdown code would look like this:

- This is an unordered list
 - This is a sub-list item
 - Another sub-list item
 - This is another unordered list item
 - This is a sub-list with a different bullet point
 - * This is a sub-sub-list item
 - Another sub-list item
-
1. This is an ordered list
 1. This is a sub-list item
 2. Another sub-list item
 2. This is another ordered list item
 - This is a sub-list with a bullet point
 - This is a sub-sub-list item
 - Another sub-list item

A table in web page designing is a way of arranging data in rows and columns using HTML tags. Tables can be used for displaying tabular data or for creating simple layouts. However, tables are not recommended for responsive web design,

as they are not flexible and adaptable to different screen sizes. Instead, CSS grid or flexbox are preferred for creating modern layouts.

To create a table in HTML, you need to use the following tags:

- **<table>**: This tag defines the start and end of a table.
- **<tr>**: This tag defines a table row.
- **<th>**: This tag defines a table header cell, which is usually bold and centered.
- **<td>**: This tag defines a table data cell, which is usually regular and left-aligned.
- **<caption>**: This tag defines a table caption, which is displayed above or below the table.
- **<colgroup>**: This tag defines a group of columns in a table.
- **<col>**: This tag defines the attributes of a column in a table, such as width, span, or style.
- **<thead>**: This tag defines the header section of a table, which can contain one or more **<tr>** tags.
- **<tbody>**: This tag defines the body section of a table, which can contain one or more **<tr>** tags.
- **<tfoot>**: This tag defines the footer section of a table, which can contain one or more **<tr>** tags.

The following diagram illustrates the basic structure of a table in HTML using ASCII characters:

Table in Web Page Designing

```
+-----+
| <table>                                     |
| +-----+                                 |
| | <caption>Table caption</caption>         | |
| +-----+                                 | |
| | <colgroup>                               | |
| |   <col span="2">                         | |
| |   <col style="background-color:yellow">  | |
| | </colgroup>                             | |
| +-----+                                 | |
| | <thead>                                  | |
| |   <tr>                                   | |
| |     <th>Header 1</th>                   | |
| |     <th>Header 2</th>                   | |
| |     <th>Header 3</th>                   | |
| |   </tr>                                  | |
| | </thead>                                | |
| +-----+                                 | |
| | <tbody>                                  | |
```

```

| | <tr> | |
| | <td>Data 1</td> | |
| | <td>Data 2</td> | |
| | <td>Data 3</td> | |
| | </tr> | |
| | <tr> | |
| | <td>Data 4</td> | |
| | <td>Data 5</td> | |
| | <td>Data 6</td> | |
| | </tr> | |
| | </tbody> | |
| +-----+ |
| | <tfoot> | |
| | <tr> | |
| | <td>Footer 1</td> | |
| | <td>Footer 2</td> | |
| | <td>Footer 3</td> | |
| | </tr> | |
| | </tfoot> | |
| +-----+ |
| </table> |
+-----+

```

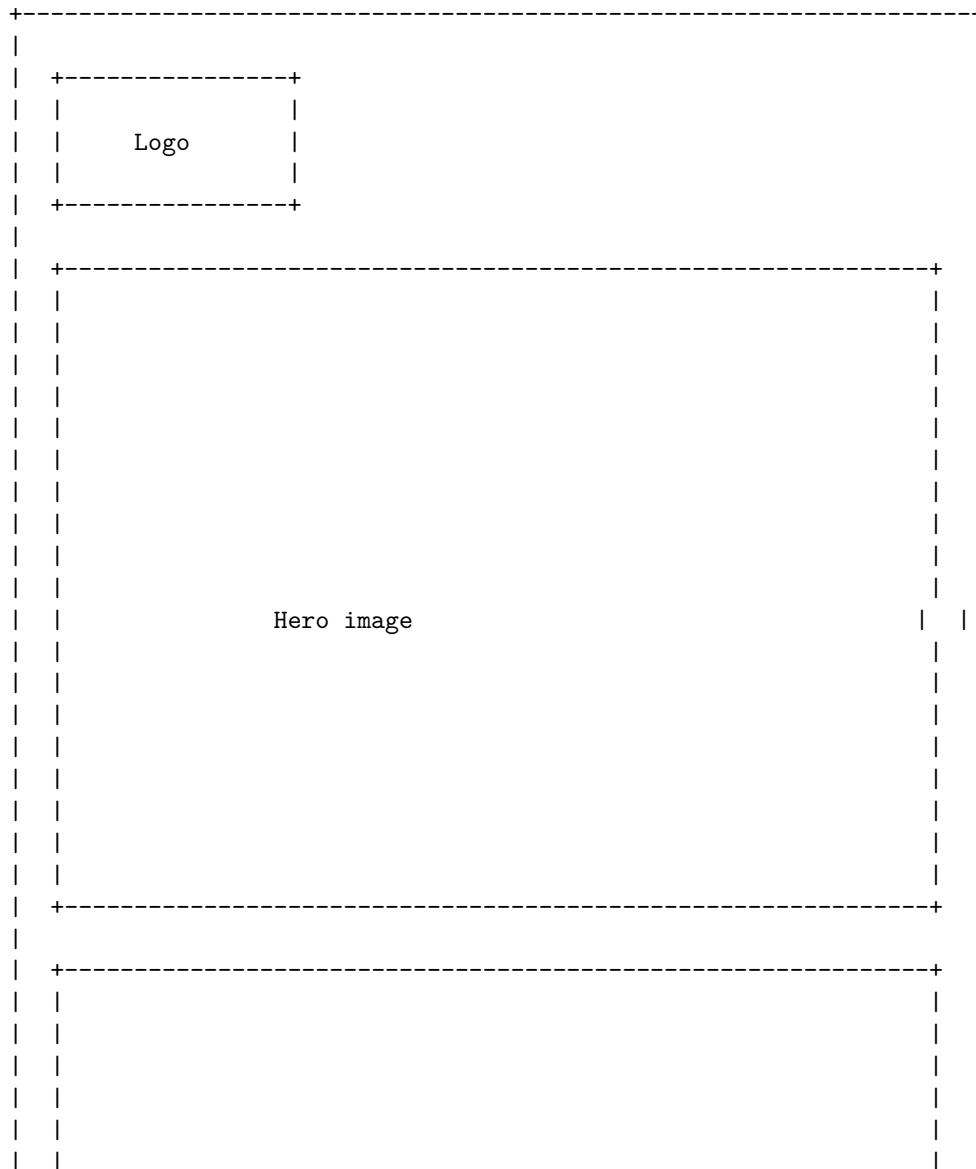
Images are an important element of web page designing, as they can enhance the appearance, usability, and SEO of a website. Images can be used for different purposes, such as:

- **Logo:** A logo is a graphical representation of a brand or company, usually placed at the top left or center of a web page. A logo should be simple, memorable, and consistent across the website. A logo can also be linked to the home page for easy navigation.
- **Hero image:** A hero image is a large, prominent image that occupies most of the space above the fold of a web page. A hero image is used to capture the attention of the visitors, convey the main message or value proposition of the website, and encourage them to take action. A hero image should be relevant, high-quality, and optimized for different screen sizes and devices.
- **Background image:** A background image is an image that fills the entire background of a web page or a section of it. A background image can create a visual contrast, add texture, or set the mood for the website. A background image should be subtle, low-contrast, and not interfere with the readability of the text or other elements on the page.
- **Content image:** A content image is an image that accompanies the text or other content on a web page. A content image can illustrate, explain, or support the information on the page, or provide visual interest or variety. A content image should be relevant, clear, and aligned with the content and the layout of the page.
- **Icon:** An icon is a small, simple image that represents an object, action,

or concept on a web page. An icon can be used to enhance the navigation, functionality, or aesthetics of the website. An icon should be recognizable, consistent, and scalable.

Images in Web Page Designing

The following diagram illustrates the basic architecture of a web page with different types of images:

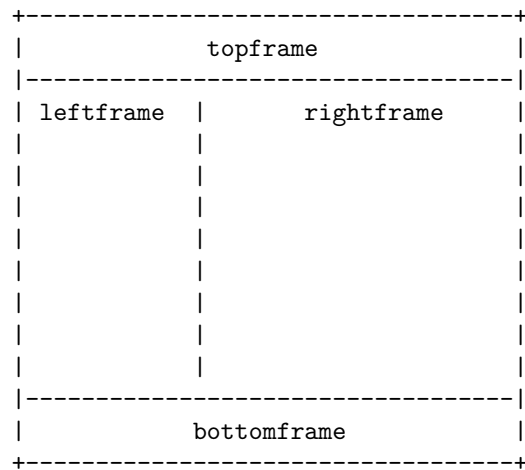



```

<frameset rows="10%,*,10%">
  <frame src="top.html" name="topframe">
  <frameset cols="20%,*">
    <frame src="left.html" name="leftframe">
    <frame src="right.html" name="rightframe">
  </frameset>
  <frame src="bottom.html" name="bottomframe">
</frameset>

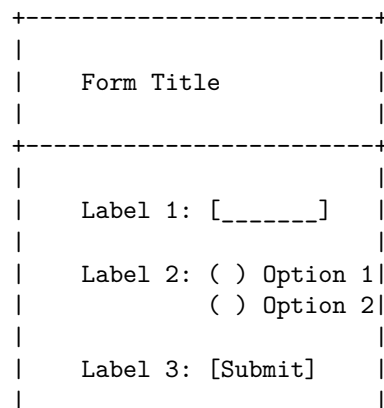
```

The following is an ASCII diagram of the frameset:



A form in web page designing is a document that stores information of a user on a web server using interactive controls. A form can contain different types of input elements, such as text fields, checkboxes, radio buttons, submit buttons, etc. A form can also have a structure, a layout, and a style that affect its appearance and functionality.

A basic ASCII diagram for forms in web page designing is shown below:



+-----+

The diagram illustrates the following components of a form:

- A form title that describes the purpose of the form.
- A label that identifies the input element and provides instructions or guidance for the user.
- A text field that allows the user to enter text data.
- A radio button that allows the user to select one option from a predefined set of options.
- A submit button that sends the user's input data to the web server for processing.

CSS stands for Cascading Style Sheets. It is used to format the layout of a webpage, such as the color, font, size, spacing, position, and display of HTML elements. CSS can be applied to HTML elements in three ways: inline, internal, and external.

Inline CSS is when the style attribute is used inside an HTML element. For example:

This is a paragraph with inline CSS.

Internal CSS is when the style element is used inside the head section of an HTML document. For example:

This is a paragraph with internal CSS.

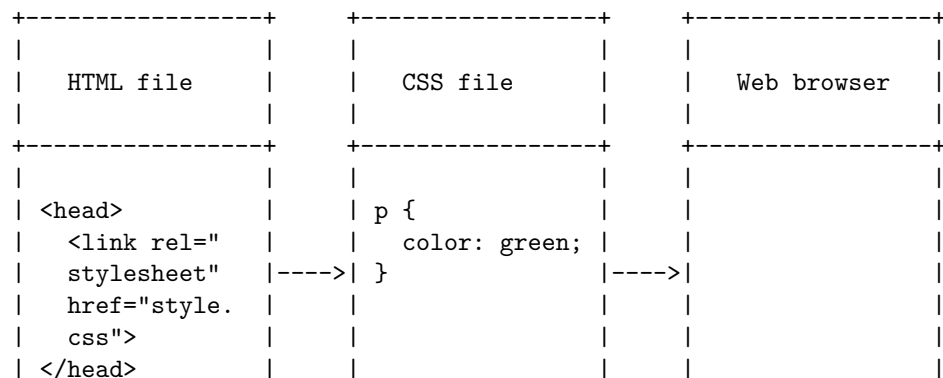
External CSS is when the link element is used to link an external CSS file to an HTML document. For example:

This is a paragraph with external CSS.

The external CSS file (style.css) would contain the following code:

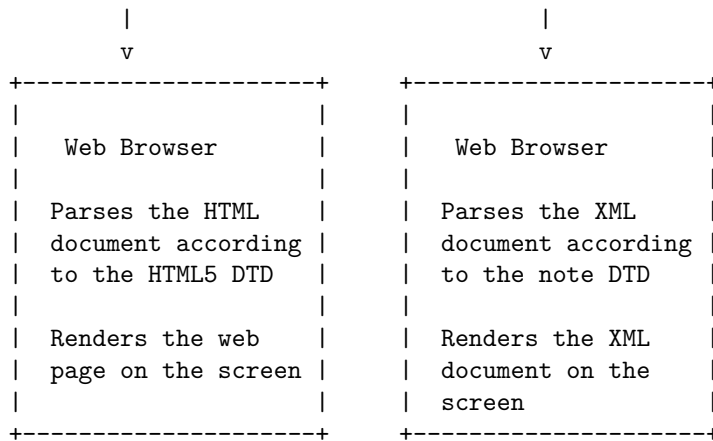
```
p { color: green; }
```

The following diagram illustrates the basic architecture of CSS in web page designing:



A document type definition (DTD) is an instruction that tells the web browser about the version of markup language in which a web page is written . It defines the structure and the legal elements and attributes of an XML document . A DTD can be declared inside an XML document as inline or as an external reference.

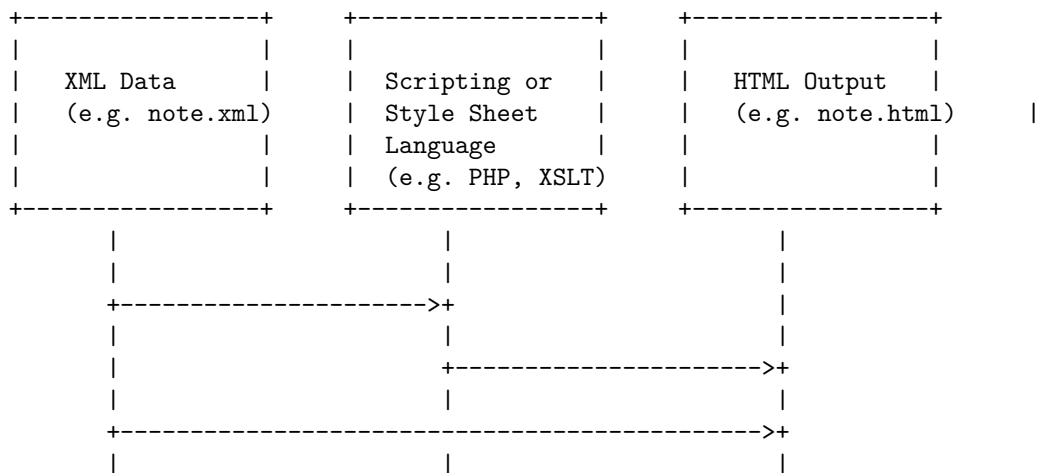
Web Page (.html)	XML Document
<pre><!DOCTYPE html> <html> ... </html></pre>	<pre><!DOCTYPE note SYSTEM "note.dtd"> <note> ... </note></pre>



XML stands for eXtensible Markup Language. It is a markup language that can be used to store and transport data, as well as to design web pages. XML is similar to HTML, but it allows for more flexibility and customization in the design. XML tags are not predefined, but can be defined by the user or by a schema.

To design a web page with XML, one needs to use a scripting language such as PHP, ASP, or Perl to process the XML data and generate the HTML output. Alternatively, one can use a style sheet language such as XSLT or CSS to transform the XML data into HTML. The following diagram illustrates the basic architecture of a web page designed with XML:

XML in Web Page Designing



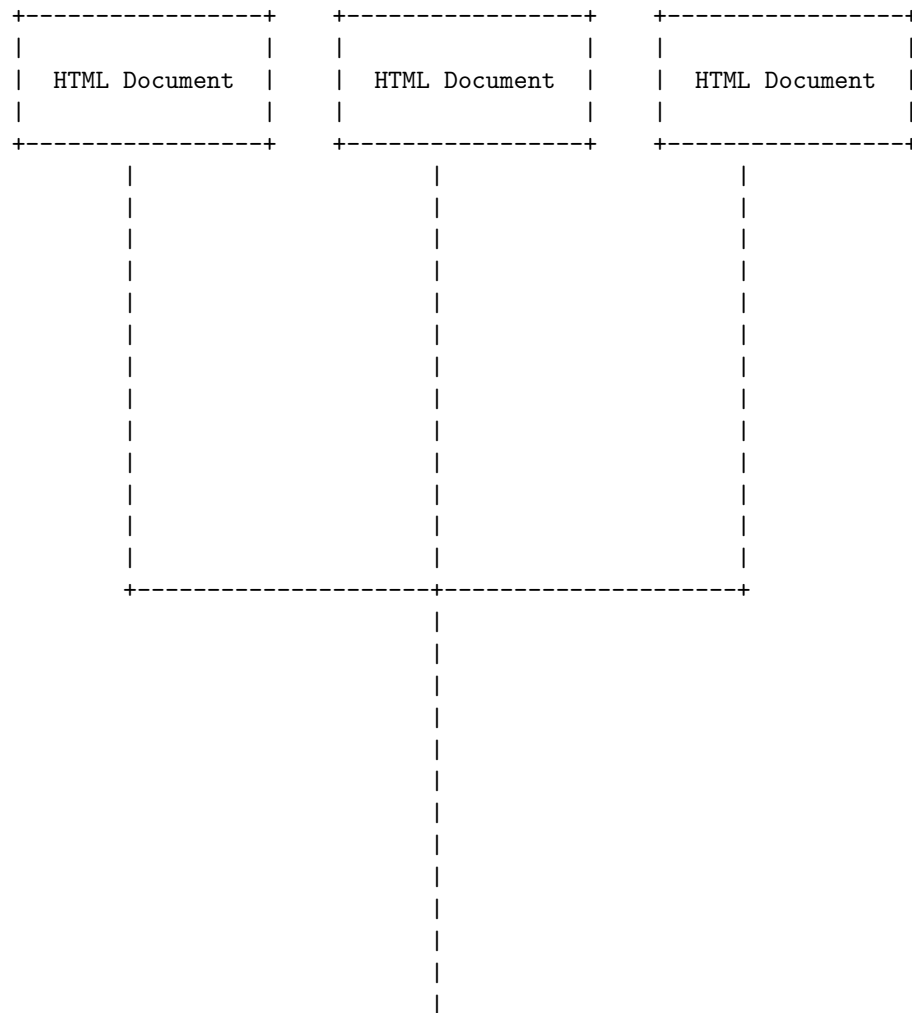
A DTD (Document Type Definition) is a set of rules that defines the structure and the legal elements and attributes of an XML document. A DTD can be

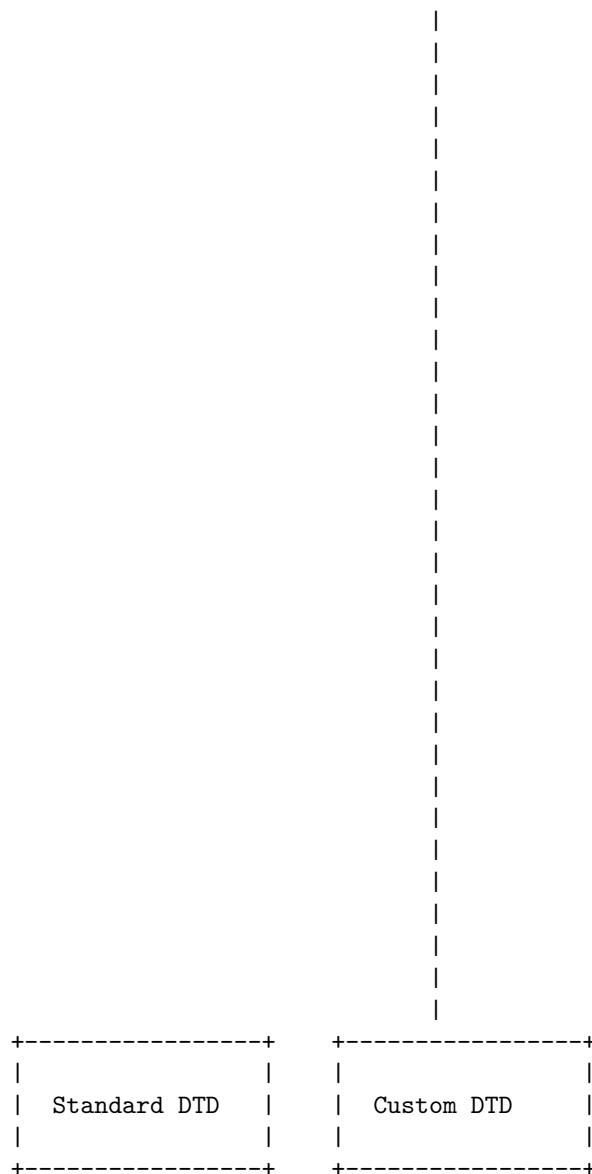
declared internally or externally in an XML document. A DTD can also be shared among multiple XML documents. A DTD helps to ensure that the XML document is well-formed and valid.

DTD in Web Page Designing

In web page designing, a DTD is used to declare the version of HTML that the web page is written in. This helps the web browser to parse the web page correctly and consistently. A DTD is specified by using the DOCTYPE declaration at the beginning of the HTML document. The DOCTYPE declaration can refer to a standard DTD or a custom DTD.

The following diagram illustrates the basic architecture of a DTD in web page designing:





The HTML documents can refer to either a standard DTD or a custom DTD. A standard DTD is a predefined DTD that follows the specifications of the World Wide Web Consortium (W3C). A custom DTD is a user-defined DTD that can have custom elements and attributes. A standard DTD can be referenced by using a public identifier or a system identifier. A custom DTD can be referenced by using a system identifier or an inline declaration. A system identifier is a URL that points to the location of the DTD file. An inline declaration is a DTD that is written inside the HTML document.

XML schemas are used to describe and validate the structure and the content of XML data. They define the elements, attributes and data types that can appear in an XML document. XML schemas can also support namespaces, which allow different XML vocabularies to be combined in a single document.

One way to design XML schemas is to use design patterns, which are common solutions to recurring problems. Some of the most common design patterns in XML schemas are:

- Russian Doll: This pattern defines all the elements locally within a single complex type. It is called Russian Doll because the elements are nested inside each other like the Russian matryoshka dolls. This pattern is simple and easy to understand, but it does not allow reuse of types or elements.
- Salami Slice: This pattern defines all the elements globally and then references them in a sequence within a single complex type. It is called Salami Slice because the elements are sliced into separate pieces and then put together in a sequence. This pattern allows reuse of elements, but it does not allow reuse of types or groups of elements.
- Venetian Blind: This pattern defines all the types globally and then references them in a sequence within a single complex type. It is called Venetian Blind because the types are hidden behind the complex type like the slats of a Venetian blind. This pattern allows reuse of types and groups of elements, but it does not allow reuse of elements.
- Garden of Eden: This pattern defines both the elements and the types globally and then references them in a sequence within a single complex type. It is called Garden of Eden because it allows maximum flexibility and reuse of both elements and types.

The following diagram illustrates the basic architecture of a XML schema using the Garden of Eden pattern:

XML Schema	XML Document
<code><xs:schema></code>	<code><note></code>
<code><xs:element</code>	<code><to></code>
<code> name="note"</code>	<code> Tove</code>
<code> type="Note"</code>	<code></to></code>
<code></></code>	<code><from></code>
<code><xs:element</code>	<code> Jani</code>
<code> name="to"</code>	<code></from></code>
<code> type="xs:string"</code>	
<code></></code>	<code><heading></code>
<code><xs:element</code>	<code> Reminder</code>
	<code></heading></code>

```

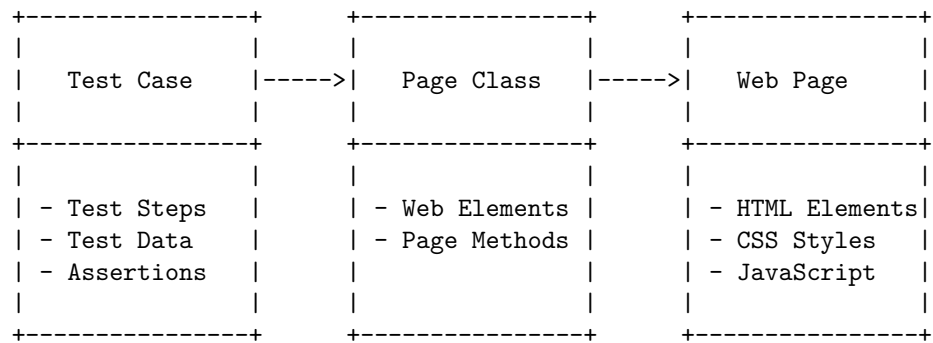
|   name="from" | | | |
|   type="xs:string" | | | <body> |
| /> | | | Don't forget |
| | | | me this |
| <xs:element | | | weekend! |
|   name="heading" | | | </body> |
|   type="xs:string" | | | |
| /> | | | </note> |
| | | |
| <xs:element | +-----+
|   name="body" |
|   type="xs:string" |
| /> |
| |
| <xs:complexType |
|   name="Note" |
| > |
| |
|   <xs:sequence |
|   > |
| |
|     <xs:element |
|       ref="to" |
|     /> |
| |
|     <xs:element |
|       ref="from" |
|     /> |
| |
|     <xs:element |
|       ref="heading" |
|     /> |
| |
|     <xs:element |
|       ref="body" |
|     /> |
| |
|   </xs:sequence |
|   > |
| |
| </xs:complexType |
| > |
| |
| </xs:schema> |
| |
+-----+

```

Object Models in Web Page Designing are a way of representing the web elements and interactions of a web page as classes, variables and methods. They are used to create an object repository and to improve the reusability and maintainability of the code. One of the most common design patterns for Object Models in Web Page Designing is the Page Object Model (POM), which divides the application into modules or pages and abstracts the web elements and actions of each page as a separate class.

The following diagram illustrates the basic architecture of a Page Object Model in Web Page Designing using ASCII art:

Object Models in Web Page Designing

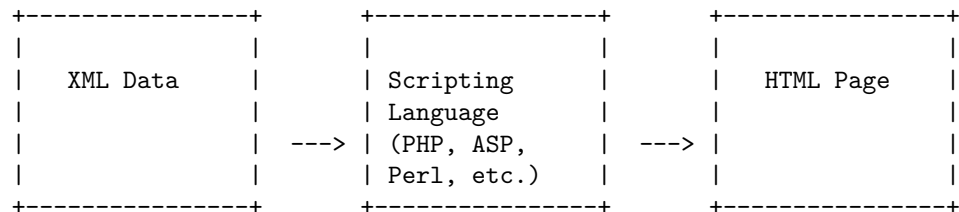


Presenting and using XML in Web Page Designing

XML stands for eXtensible Markup Language. It is a markup language that defines the structure and meaning of data. XML is often used to separate data from presentation, so that the same XML data can be used in different scenarios, such as web pages, mobile applications, or desktop applications.

To present and use XML in web page designing, we need to use a scripting language, such as PHP, ASP, or Perl, to dynamically generate HTML pages from XML data. The scripting language can also perform other tasks, such as validating, transforming, or querying the XML data.

The following diagram illustrates the basic architecture of presenting and using XML in web page designing:



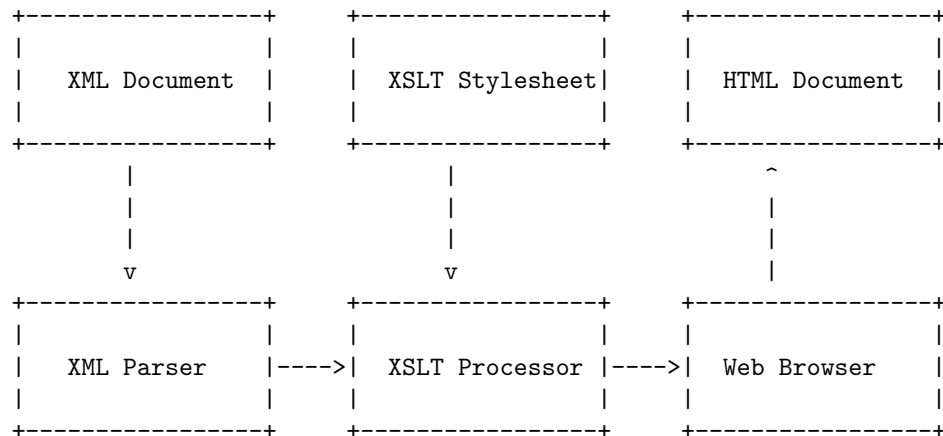
The XML data can be stored in a file, a database, or a web service. The scripting language can access the XML data using various methods, such as DOM, SAX, XPath, or XQuery. The scripting language can also use XSLT to transform the XML data into HTML or other formats. The HTML page can then be displayed in a web browser or other devices.

Using XML processors in web page designing is a way of creating dynamic and interactive web pages that can store, transport and display data in a structured and human-readable format. XML processors are software tools that can parse, validate, transform and manipulate XML documents. XML processors can be classified into two types: XML parsers and XSLT processors.

XML parsers are responsible for reading and validating XML documents, and creating a tree-like structure that represents the elements, attributes and text nodes of the document. XML parsers can also perform operations such as searching, modifying and deleting nodes in the tree. XML parsers can be either validating or non-validating, depending on whether they check the document against a schema or a DTD (Document Type Definition).

XSLT processors are responsible for transforming XML documents into other formats, such as HTML, using a set of rules and templates defined in an XSLT stylesheet. XSLT processors can also perform operations such as sorting, filtering and grouping data in the XML document. XSLT processors can be either embedded in the browser or run on the server side.

The following diagram illustrates the basic architecture of using XML processors in web page designing:

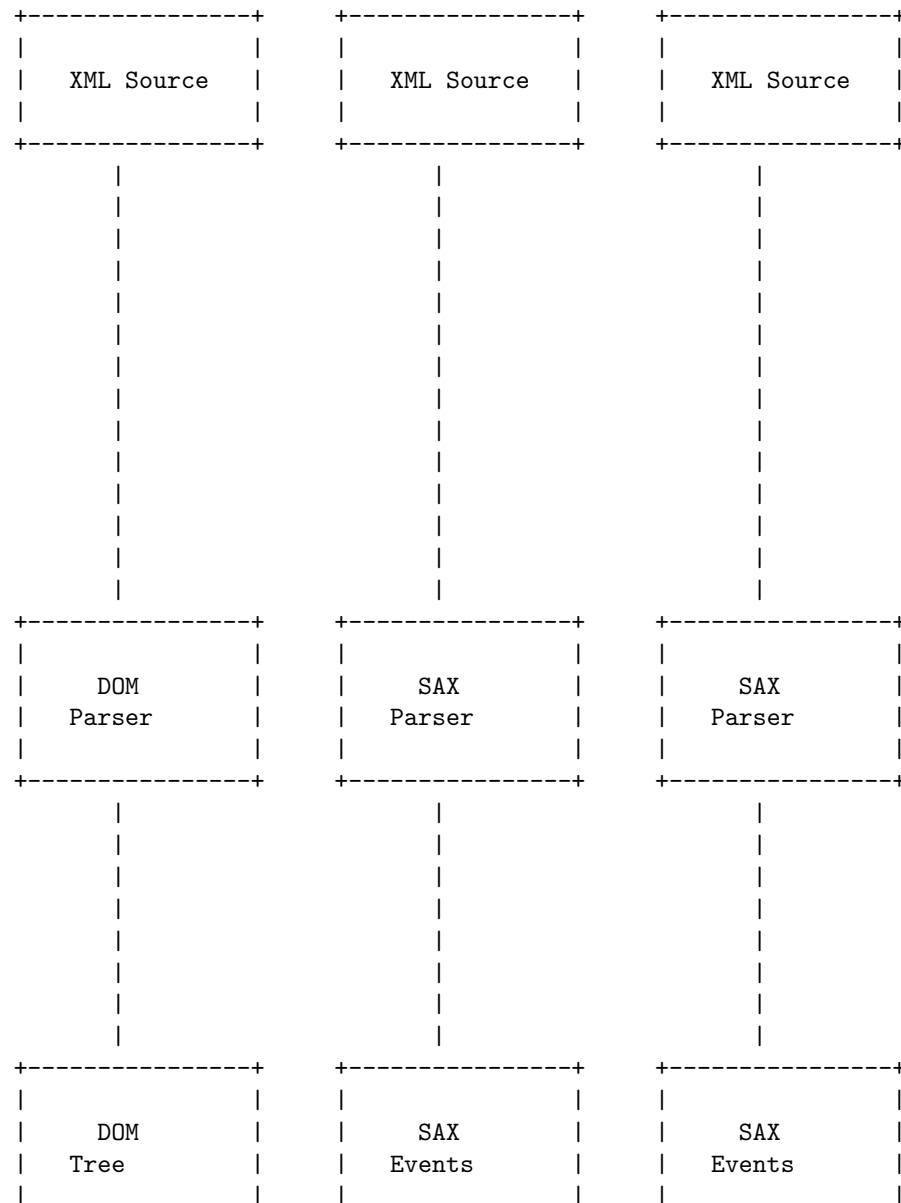


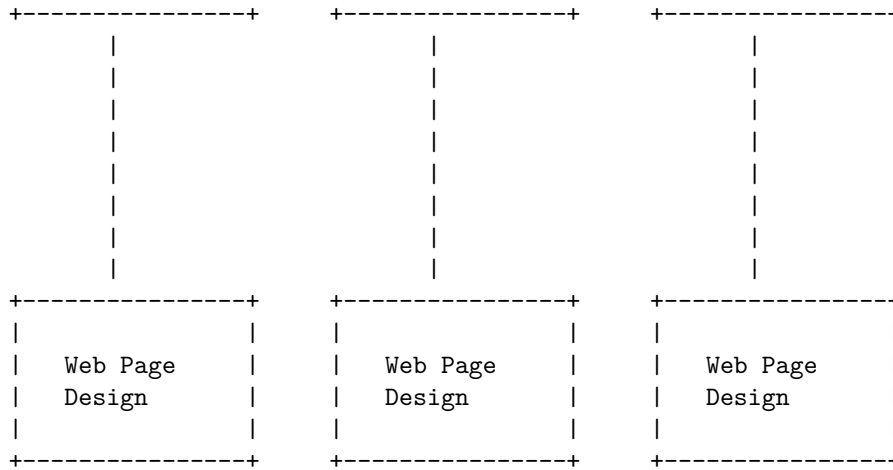
DOM and SAX are two different ways of parsing XML documents. DOM stands for Document Object Model, and SAX stands for Simple API for XML. DOM parses the whole XML document and creates a tree-like structure in memory, which can be manipulated and queried using various methods. SAX parses the XML document sequentially, and generates events for each element, attribute, text, etc. SAX is more efficient for large XML documents, as it does not load the

whole document in memory, but it does not allow random access or modification of the document.

DOM and SAX in Web Page Designing

The following diagram shows how DOM and SAX can be used in web page designing.





The diagram illustrates three possible scenarios:

- The first scenario uses a DOM parser to create a DOM tree from the XML source, and then uses the DOM tree to design the web page. This scenario allows the web page designer to access and modify any part of the XML document, but it also consumes more memory and processing time.
- The second scenario uses a SAX parser to generate events from the XML source, and then uses the events to design the web page. This scenario is more efficient for large XML documents, as it does not load the whole document in memory, but it also does not allow random access or modification of the XML document.
- The third scenario uses a SAX parser to generate events from the XML source, and then uses another SAX parser to generate events from the events, and then uses the events to design the web page. This scenario is useful when the XML source is not well-formed or valid, as it allows the web page designer to filter or transform the XML document before designing the web page.

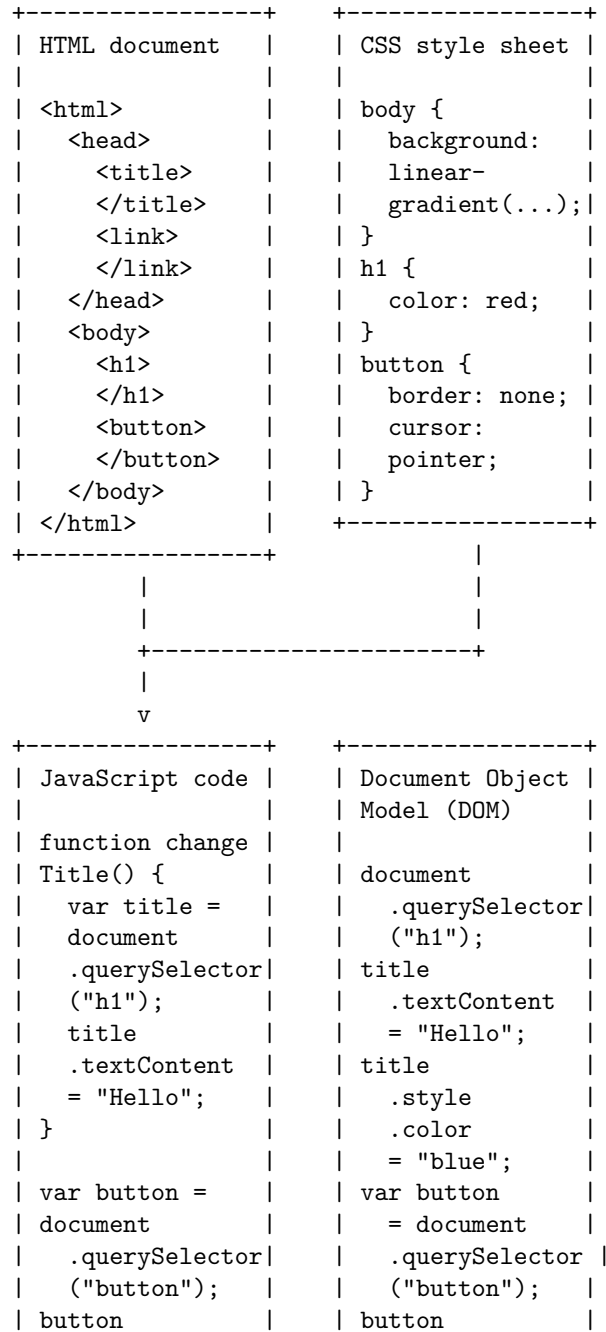
Dynamic HTML (DHTML) is a term that describes the use of various web technologies to create dynamic and interactive web pages. DHTML can work with HTML, JavaScript, XML, and CSS to manipulate the HTML elements and their styles, events, and behaviors .

The basic architecture of a DHTML web page consists of four components:

- The HTML document, which contains the structure and content of the web page.
- The CSS style sheet, which defines the presentation and layout of the HTML elements.
- The JavaScript code, which provides the functionality and interactivity of the web page.
- The Document Object Model (DOM), which is a representation of the

HTML document as a tree of objects that can be accessed and modified by JavaScript.

The following diagram illustrates the basic architecture of a DHTML web page:



```

| .onclick      | | .onclick      |
| = changeTitle;| | = changeTitle;|
+-----+      +-----+

```

: **DHTML Tutorial - Javatpoint** What is Dynamic HTML? - Code Institute Global
How Dynamic HTML (DHTML) Is Used to Create Interactive Pages - ThoughtCo

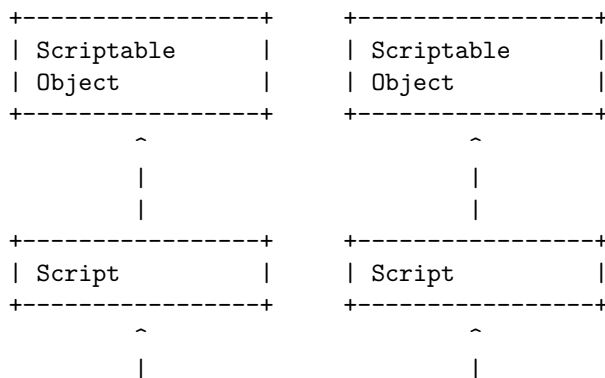
Unit 3 - Scripting

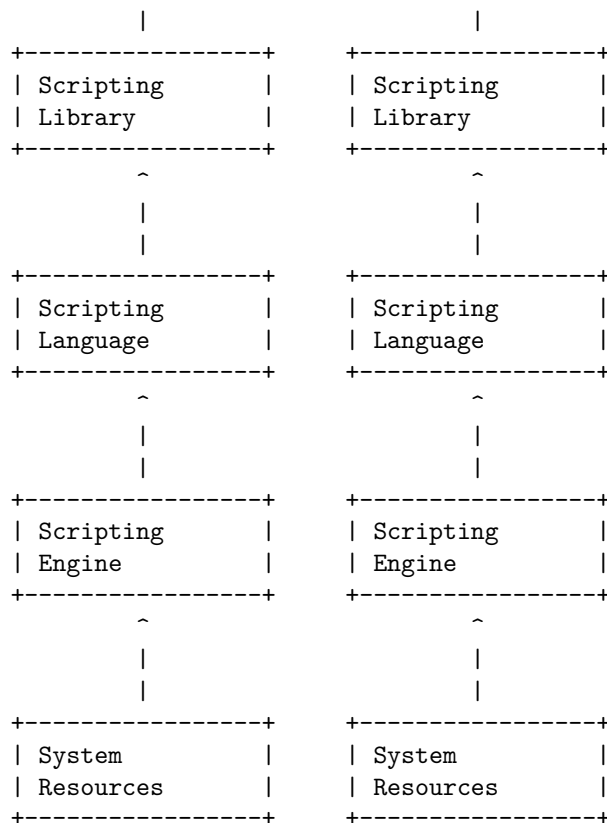
Scripting is a technique of writing code that can automate tasks, control other applications, or enhance the functionality of a system. Scripting languages are usually interpreted, high-level, and dynamically typed. Some examples of scripting languages are Python, Perl, JavaScript, and Ruby.

A scripting language can interact with other components of a system, such as applications, libraries, databases, or web servers. A scripting architecture is a way of organizing these components and defining how they communicate with each other. A scripting architecture can have different layers, such as:

- Scripting engine: The component that executes the scripts and provides an interface to the underlying system.
- Scripting language: The syntax and semantics of the code that the scripting engine can understand and execute.
- Scripting library: A collection of predefined functions or classes that the scripting language can use to perform common tasks or access system resources.
- Scriptable object: An object that can be manipulated by the scripting language, such as a file, a window, a button, or a database record.
- Script: A piece of code that uses the scripting language to control or communicate with the scriptable objects.

The following diagram illustrates the basic architecture of a scripting system using ASCII art:

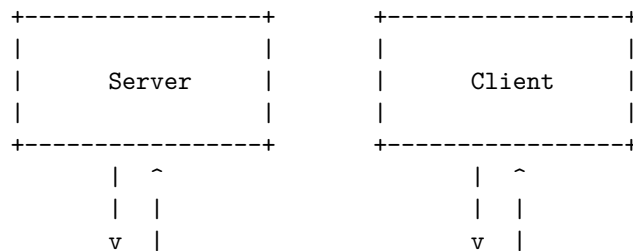


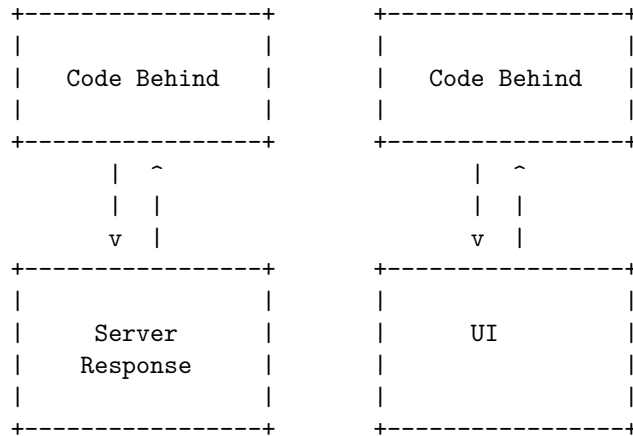


JavaScript is a scripting or programming language that allows you to implement complex features on web pages. It can also run on other environments, such as Node.js or Electron, using different JavaScript engines. A JavaScript engine is a program that executes and compiles JavaScript into native machine code.

A typical JavaScript application architecture uses the bottom-up approach, always placing the User Interface (UI) at the center of the development at all times. As shown in the diagram, both the UI and the Server directly link to the code behind.

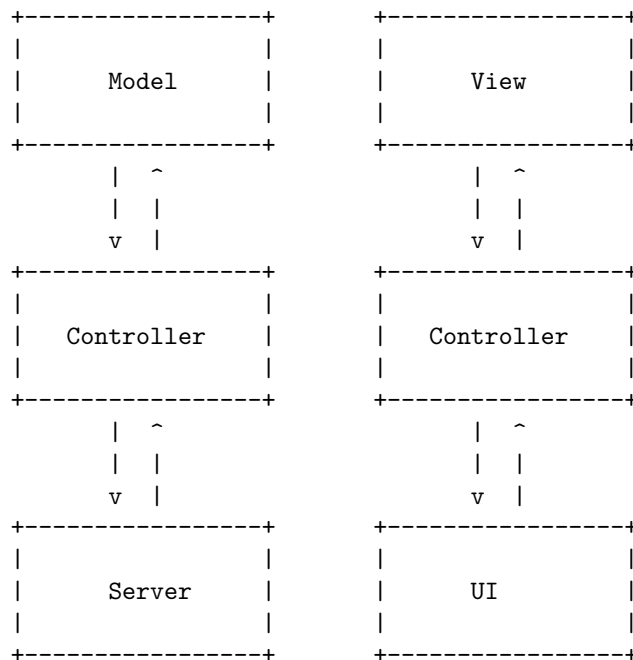
JavaScript in Scripting





Another common JavaScript architecture is the Model-View-Controller (MVC) pattern, which separates the data (model), the presentation (view), and the logic (controller) of the application. This allows for better modularity, reusability, and maintainability of the code. The diagram below shows how the MVC components interact with each other.

JavaScript in MVC



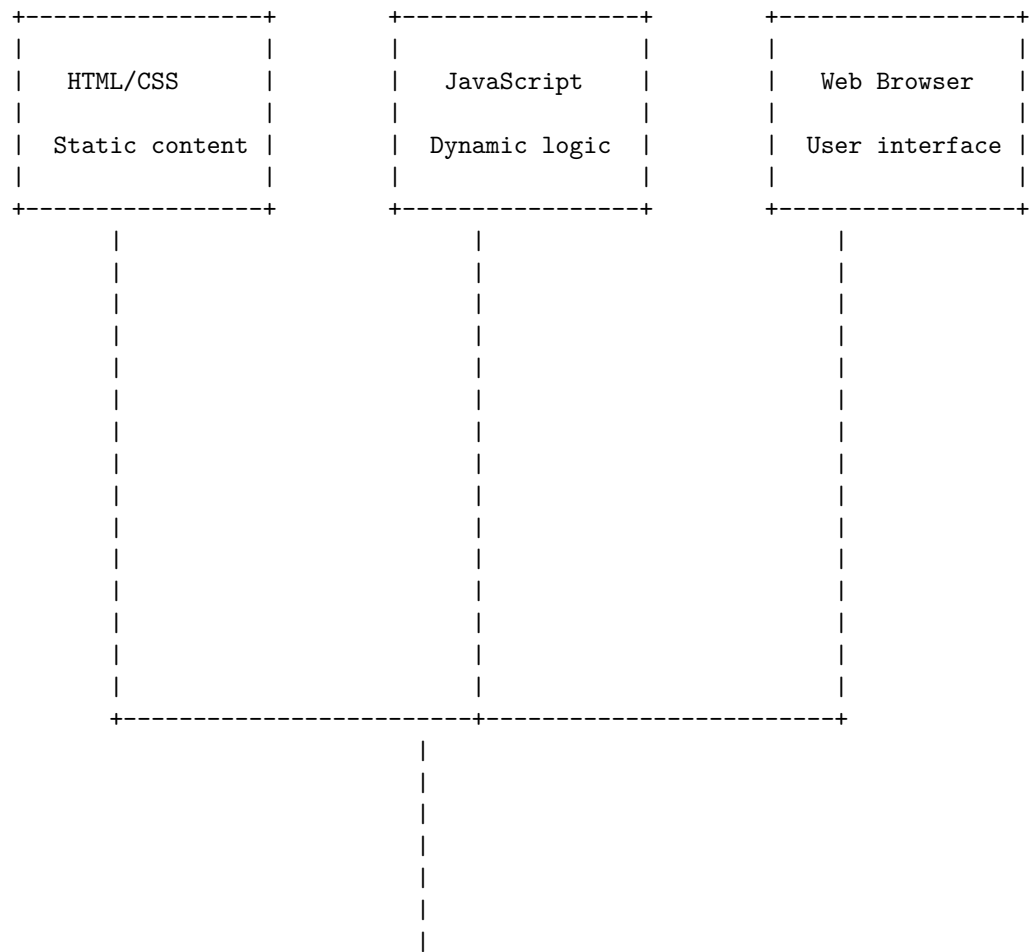
Introduction to JavaScript

JavaScript is a scripting language that is used to create and manage dynamic web pages, basically anything that moves on your screen without requiring you to refresh your browser. It can be anything from animated graphics to an automatically generated Facebook timeline.

JavaScript was initially created to “make web pages alive”. The programs in this language are called scripts. They can be written right in a web page’s HTML and run automatically as the page loads. Scripts are provided and executed as plain text. They don’t need special preparation or compilation to run.

JavaScript is a multi-paradigm, dynamic language with types and operators, standard built-in objects, and methods. Its syntax is based on the Java and C languages — many structures from those languages apply to JavaScript as well. JavaScript supports object-oriented programming with object prototypes and classes.

The following diagram illustrates the basic architecture of a JavaScript program:



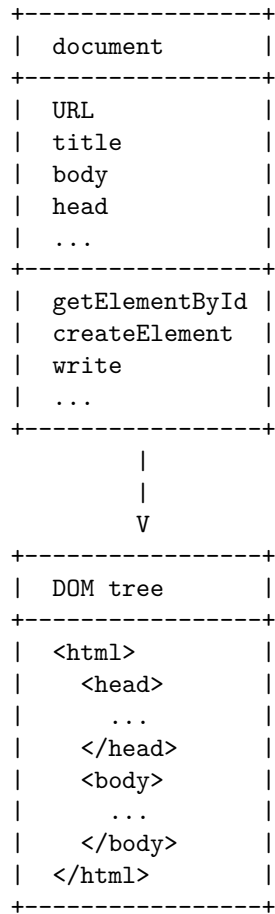
|





A document in JavaScript is an object that represents a web page and provides access to its content and structure, often referred to as the Document Object Model (DOM) tree. The document object has various properties and methods that can be used to manipulate the elements and attributes of the web page. For example, the `document.getElementById()` method can be used to find an element by its id attribute, and the `document.createElement()` method can be used to create a new element.

The following diagram illustrates the basic architecture of a document in JavaScript using ASCII characters:



A form in JavaScript is an HTML element that allows users to enter and submit data. A form typically consists of one or more input fields, a submit button, and an action attribute that specifies the URL that processes the form data. A form can also have a method attribute that specifies the HTTP method (GET or POST) to use when sending the data.

A basic form in HTML looks like this:

```

<form action="/signup" method="post" id="signup">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required>
  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>
  <button type="submit">Sign up</button>
</form>

```

To access and manipulate a form in JavaScript, you can use the `document.getElementById()` method to get a reference to the form element by its

id attribute. For example:

```
const form = document.getElementById("signup");
```

You can also use the document.forms property to get a collection of all the forms in the document. For example:

```
const forms = document.forms; // returns an HTMLFormControlsCollection
const form = forms[0]; // returns the first form in the document
```

To access the input fields within a form, you can use the form.elements property, which returns an HTMLFormControlsCollection of all the form controls. You can access each control by its name or index. For example:

```
const name = form.elements.name; // returns the input element with name="name"
const email = form.elements[1]; // returns the second input element in the form
```

To get or set the value of an input field, you can use the value property of the input element. For example:

```
name.value = "Alice"; // sets the value of the name input to "Alice"
console.log(email.value); // prints the value of the email input
```

To validate the input fields, you can use the required attribute in the HTML, which prevents the form from being submitted if the field is empty. You can also use the pattern attribute to specify a regular expression that the input value must match. For example:

```
<input type="text" id="name" name="name" required pattern="[A-Za-z\s]+">
```

This input field requires a non-empty value that consists of only letters and spaces.

To perform custom validation in JavaScript, you can use the checkValidity() method of the input element, which returns true if the input value passes the validation rules, or false otherwise. You can also use the validity property of the input element, which returns a ValidityState object that contains various properties indicating the validity state of the input. For example:

```
if (name.checkValidity()) {
  console.log("Name is valid");
} else {
  console.log("Name is invalid");
}

if (email.validity.typeMismatch) {
  console.log("Email is not a valid email address");
}
```

To submit a form in JavaScript, you can use the submit() method of the form element, which sends the form data to the specified action URL using the specified method. For example:

```
form.submit(); // submits the form data
```

You can also use the FormData object to create and manipulate form data programmatically. The FormData object can be used to append key-value pairs of form data, or to get the form data from an existing form element. For example:

```
const formData = new FormData(); // creates an empty FormData object
formData.append("name", "Alice"); // appends a name field with value "Alice"
formData.append("email", "alice@example.com"); // appends an email field with value "alice@example.com"
```

```
const formData = new FormData(form); // creates a FormData object from an existing form element
```

The FormData object can be used to send form data using the XMLHttpRequest or the Fetch API. For example:

```
const xhr = new XMLHttpRequest(); // creates a new XMLHttpRequest object
xhr.open("POST", "/signup"); // sets the request method and URL
xhr.send(formData); // sends the form data
```

```
fetch("/signup", {
  method: "POST", // sets the request method
  body: formData // sets the request body to the form data
})
.then(response => console.log(response))
.catch(error => console.error(error));
```

The following diagram illustrates the basic architecture of a form in JavaScript:

```
+-----+ +-----+
```

A statement in JavaScript is a programming instruction that specifies what scripts will do and how they will do it. A statement can be composed of values, operators, expressions, keywords, and comments. A statement can span multiple lines, or multiple statements can occur on a single line if separated by semicolons. Statements can be grouped together inside curly brackets to form code blocks. JavaScript ignores multiple white spaces and line breaks in statements.

The following diagram illustrates the basic structure of a statement in JavaScript using ASCII characters:

Expression	Operator	Expression
Value	Keyword	Value

	Comment		Semicolon
			Comment
+	-----+	+	-----+

An example of a statement in JavaScript is:

```
// This is a comment
var x = 5 + 10; // This statement assigns the value of 5 + 10 to the variable x
```

A function in JavaScript is a reusable block of code that performs a specific task, taking some form of input and returning an output. A function can be defined with the function keyword, followed by a name, followed by parentheses that may include parameter names separated by commas. A function can also be assigned to a variable or a property of an object, or passed to or returned from another function. A function can have properties and methods just like any other object.

The following diagram illustrates the basic structure of a function in JavaScript:

+	-----+	
	function name	<--- function declaration
+	-----+	
	(parameter1,	<--- function parameters
	parameter2,	
	...)	
+	-----+	
	{	<--- function body
	// statements	<--- function statements
	return value;	<--- function return value
	}	
+	-----+	

The following diagram illustrates the basic usage of a function in JavaScript:

+	-----+	+	-----+	
	function name		variable name	<--- function expression
+	-----+	+	-----+	
	(parameter1,		= function	
	parameter2,		(parameter1,	
	...)		parameter2,	
+	-----+		...)	
	{	+	-----+	
	// statements		{	
	return value;		// statements	
	}		return value;	
+	-----+		}	
		+	-----+	

+-----+	
object name	<--- function as a property
+-----+	
name: function	
(parameter1,	
parameter2,	
...)	
+-----+	
{	
// statements	
return value;	
}	
+-----+	
+-----+	
function name	<--- function as an argument
+-----+	
(function	

```
+-----+
| object name | <--- function as a property
```

```
+-----+
| name: function
|   (parameter1,
|     parameter2,
|     ...)
```

```
+-----+
| {
|   // statements
|   return value;
| }
```

```
+-----+
| function name | <--- function as an argument
```

```
+-----+
| (function
```

```

| (parameter1, |
|   parameter2, |
|   ...) |
+-----+
| { |
|   // statements |
|   return value; |
| } |
+-----+
| |
| |
| |
| |
| |
| |
| |
| |
| |
| |
+-----+
| variable name | <--- function as a return value
+-----+
| = function |
| (parameter1, |
|   parameter2, |
|   ...) |
+-----+
| { |
|   // statements |
|   return |
|     function |
|       (parameter1, |
|         parameter2, |
|         ...) |
| } |
+-----+

```

An object in JavaScript is a variable that contains multiple data values, which are called properties. Properties can be primitive values, such as strings, numbers, or booleans, or reference values, such as arrays, functions, or other objects. Properties are accessed using dot notation or bracket notation, such as `obj.name`

or `obj["name"]`. Objects can also have methods, which are functions that belong to the object and can perform actions on it. Methods are invoked using parentheses, such as `obj.sayHello()`.

The following diagram illustrates the basic structure of an object in JavaScript using ASCII characters:

```

+-----+
|      Object      |
+-----+
| name: "John"      |
| age: 25            |
| hobbies: ["reading", | +-----+
|           "gaming"] |>|   Array   |
| sayHello: function(| +-----+
| {                 | | 0: "reading" |
|   console.log("Hi, I | | 1: "gaming" |
|   am " + this.name);| +-----+
| }                 |
+-----+

```

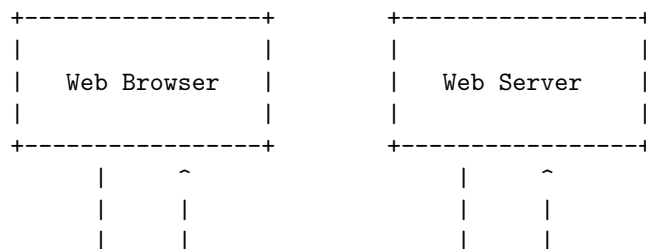
The object has four properties: **name**, **age**, **hobbies**, and **sayHello**. The **name** and **age** properties are primitive values, while the **hobbies** property is a reference to an array object, which has two elements. The **sayHello** property is a method, which is a function that prints a greeting message using the **name** property of the object. The **this** keyword refers to the current object.

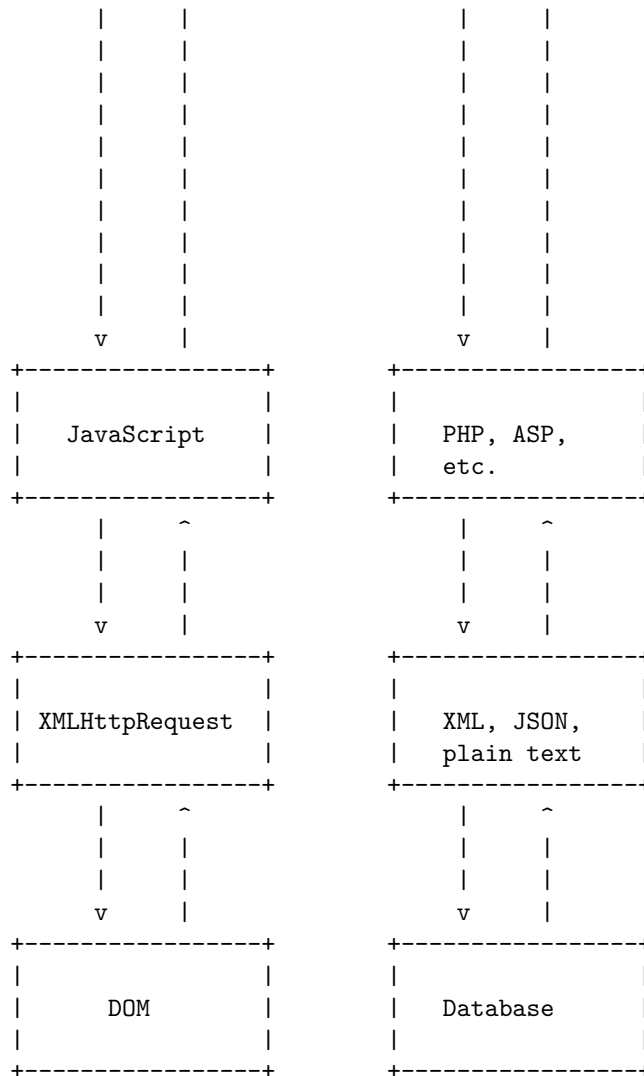
Introduction to AJAX in JavaScript

AJAX stands for Asynchronous JavaScript and XML. It is a technique that allows web pages to communicate with the server without reloading the whole page. It uses a combination of the following technologies:

- JavaScript: to create and send an XMLHttpRequest object to the server
- XMLHttpRequest: to request and receive data from the server
- DOM: to manipulate and display the data on the web page
- XML, JSON, or plain text: to format and transport the data between the server and the client

The following diagram illustrates the basic architecture of an AJAX application:





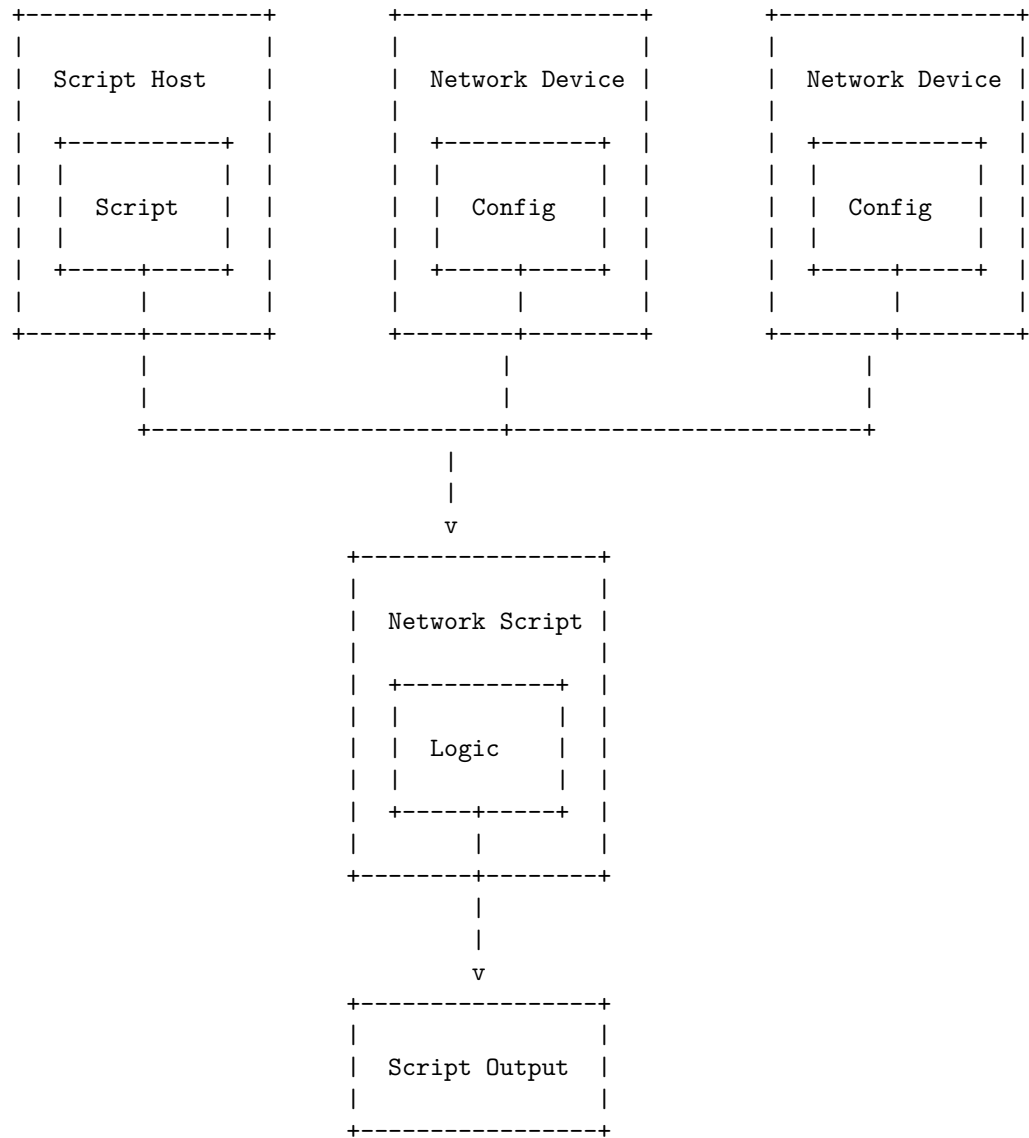
The steps involved in an AJAX communication are:

1. The user interacts with the web page and triggers an event, such as clicking a button or entering some text.
2. The JavaScript code creates an XMLHttpRequest object and sends it to the server with some parameters, such as the URL, the method (GET or POST), and the data (if any).
3. The server processes the request and sends back a response, which can be in XML, JSON, or plain text format.
4. The XMLHttpRequest object receives the response and passes it to the JavaScript code.
5. The JavaScript code uses the DOM to update the web page with the new

data, without reloading the page.

Networking in Scripting is the process of using scripts to automate various network administration tasks, such as mapping network drives, configuring network devices, monitoring network performance, and troubleshooting network issues. Scripts are written in different languages, such as shell, Python, PowerShell, Perl, and Ruby, and can be executed on different platforms, such as Windows, Linux, and macOS.

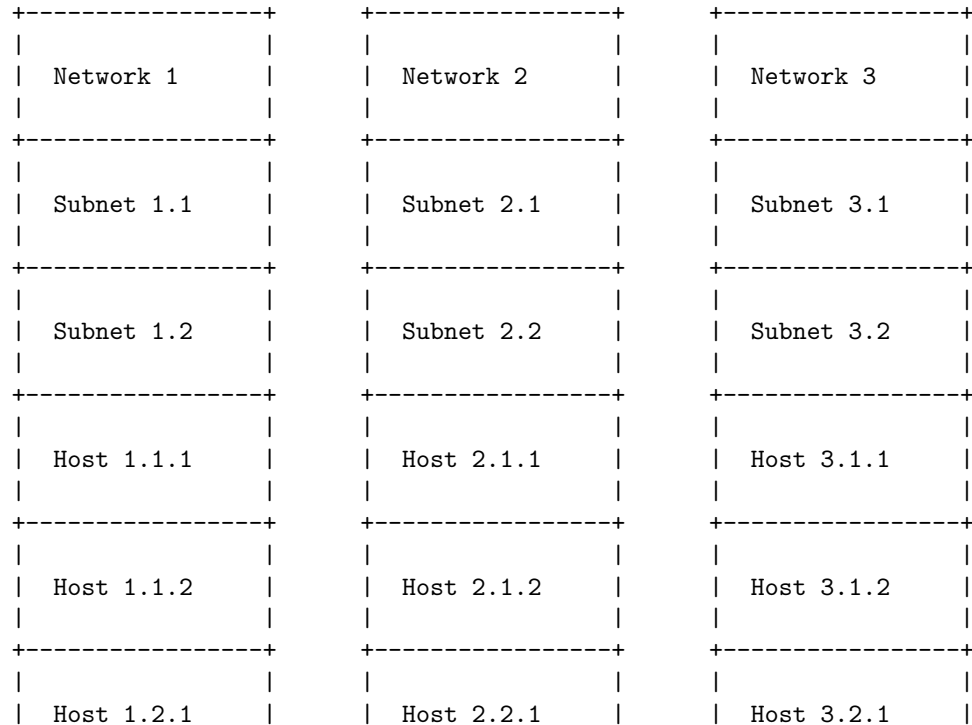
The following diagram illustrates the basic architecture of a network script:



The script host is the machine where the script is executed. It can be a local or remote computer, depending on the script language and the network protocol used. The script host communicates with the network devices through the network script, which contains the logic and commands to perform the network administration tasks. The network script can use different protocols, such as SSH, Telnet, SNMP, REST, or NETCONF, to interact with the network devices. The network devices are the routers, switches, firewalls, or other devices that are configured, monitored, or troubleshooted by the script. The network devices have their own configuration files, which store the settings and parameters of the device. The script output is the result of the script execution, which can be displayed on the screen, saved to a file, or sent to a database or a monitoring system. The script output can show the status, performance, or configuration of the network devices, or any errors or warnings that occurred during the script execution.

Internet addressing is the process of assigning unique identifiers to devices on a network. The most common protocol for internet addressing is TCP/IP, which stands for Transmission Control Protocol/Internet Protocol. TCP/IP uses a hierarchical structure of addresses, consisting of four levels: network, subnet, host, and socket.

The following diagram illustrates the basic architecture of internet addressing in TCP/IP:



Host 1.2.2	Host 2.2.2	Host 3.2.2
Socket 1.1.1.1	Socket 2.1.1.1	Socket 3.1.1.1
Socket 1.1.1.2	Socket 2.1.1.2	Socket 3.1.1.2
Socket 1.1.2.1	Socket 2.1.2.1	Socket 3.1.2.1
Socket 1.1.2.2	Socket 2.1.2.2	Socket 3.1.2.2
Socket 1.2.1.1	Socket 2.2.1.1	Socket 3.2.1.1

InetAddress in Networking

The `InetAddress` class is a representation of an IP address, which is a numerical label assigned to a machine in a network. An IP address can be either 32-bit (IPv4) or 128-bit (IPv6). An instance of `InetAddress` encapsulates both the numerical IP address and the domain name for that address, if available. The `InetAddress` class can handle both unicast and multicast addresses. Unicast addresses are used to identify a single host, while multicast addresses are used to identify a group of hosts that can receive the same message.

The following diagram illustrates the basic architecture of an `InetAddress`:

<code>InetAddress</code>	<code>InetAddress</code>
- address: int	- address: int
- family: int	- family: int
- hostName: String	- hostName: String

```
| + getAddress(): byte[] | | + getAddress(): byte[] |
| + getHostAddress(): String | | + getHostAddress(): String |
| + getHostName(): String | | + getHostName(): String |
| + isMulticastAddress(): boolean | | + isMulticastAddress(): boolean |
| + isAnyLocalAddress(): boolean | | + isAnyLocalAddress(): boolean |
| + isLoopbackAddress(): boolean | | + isLoopbackAddress(): boolean |
| + isLinkLocalAddress(): boolean | | + isLinkLocalAddress(): boolean |
| + isSiteLocalAddress(): boolean | | + isSiteLocalAddress(): boolean |
| + isMCGlobal(): boolean | | + isMCGlobal(): boolean |
| + isMCNodeLocal(): boolean | | + isMCNodeLocal(): boolean |
| + isMCLinkLocal(): boolean | | + isMCLinkLocal(): boolean |
| + isMCSiteLocal(): boolean | | + isMCSiteLocal(): boolean |
| + isMCOrgLocal(): boolean | | + isMCOrgLocal(): boolean |
```

|

|

|

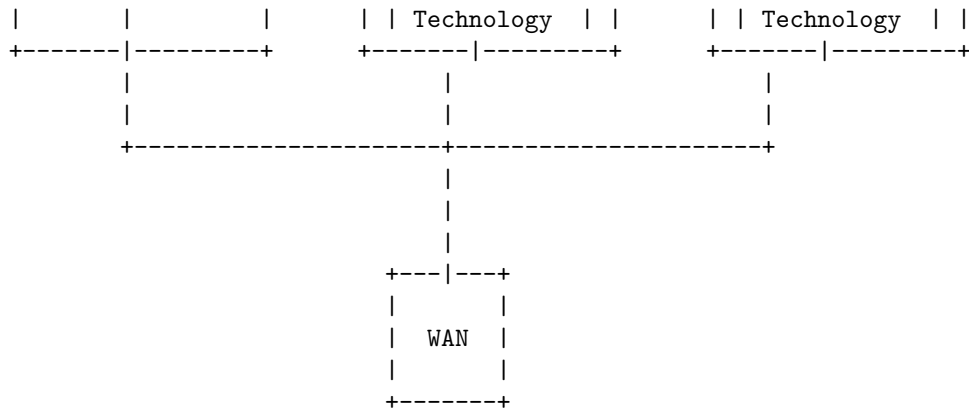
|



Factory Methods in Networking are a design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created. This pattern is useful when the type of objects to be created depends on the network environment, such as the protocol, the topology, or the device level.

Factory Methods in Networking

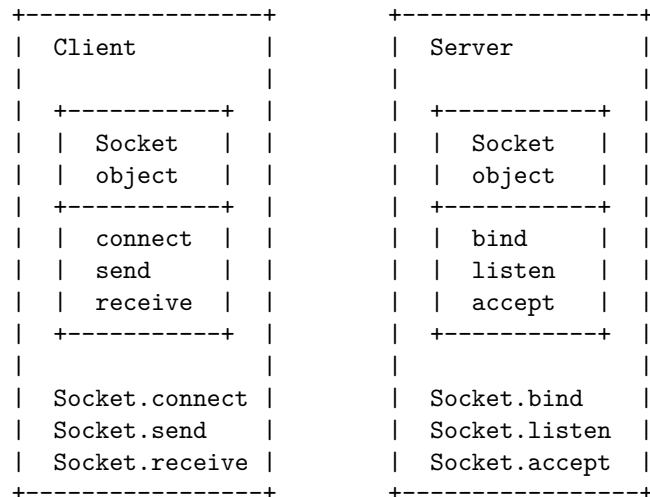
Enterprise Level	Control Level	Device Level
Application	Application	Application
HTTP, FTP, SNMP	TCP, UDP	IP
TCP, UDP	Ethernet	Ethernet
IP	Physical Transmission	Physical Transmission



Instance methods in networking are methods that belong to an object of a class, not to the class itself. They can be used to perform operations on the object's state or to communicate with other objects. For example, an instance method of a socket class could be used to send or receive data over a network connection.

A class method, on the other hand, is a method that belongs to the class itself, not to any specific object. It can be used to perform operations that are relevant to the class as a whole, such as creating new objects or accessing class variables. For example, a class method of a socket class could be used to create a new socket object or to get the default timeout value.

The following diagram illustrates the basic architecture of a network application that uses instance methods and class methods of a socket class:



The client creates a socket object and calls its instance method `connect` to establish a connection with the server. Then it calls its instance methods `send` and `receive` to exchange data with the server. The server also creates a socket

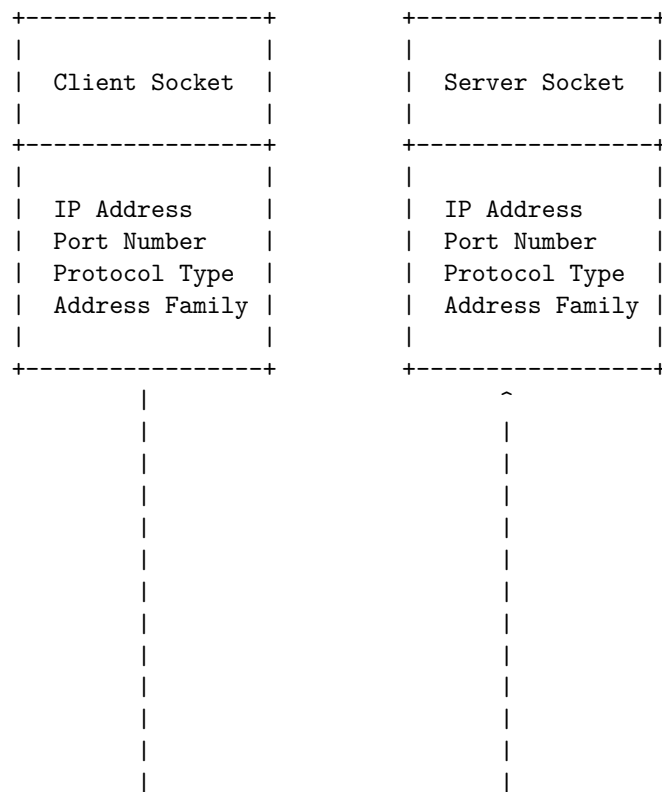
object and calls its instance methods `bind`, `listen`, and `accept` to set up a listening socket and accept incoming connections. Then it calls its instance methods `send` and `receive` to exchange data with the client. Both the client and the server use the class methods of the socket class to create new socket objects or to access class variables.

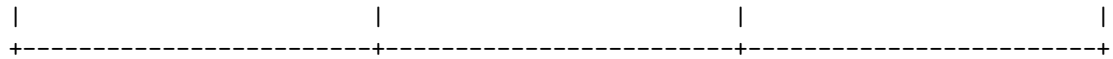
TCP/IP Client Sockets in Networking ##### TCP/IP Client Sockets in Net-
working

TCP/IP sockets are used to implement reliable, bidirectional, persistent, point-to-point, stream-based connections between hosts on the Internet. A socket can be used to connect Java's I/O system to other programs that may reside either on the local machine or on any other machine on the Internet.

A TCP/IP client socket is an endpoint of a communication link between a client program and a server program. A client socket initiates a connection request to a server socket, which listens for and accepts incoming connections. A client socket specifies the IP address and port number of the server socket, as well as the protocol type (TCP or UDP) and the address family (IPv4 or IPv6) .

The following diagram illustrates the basic architecture of a TCP/IP client socket in networking:

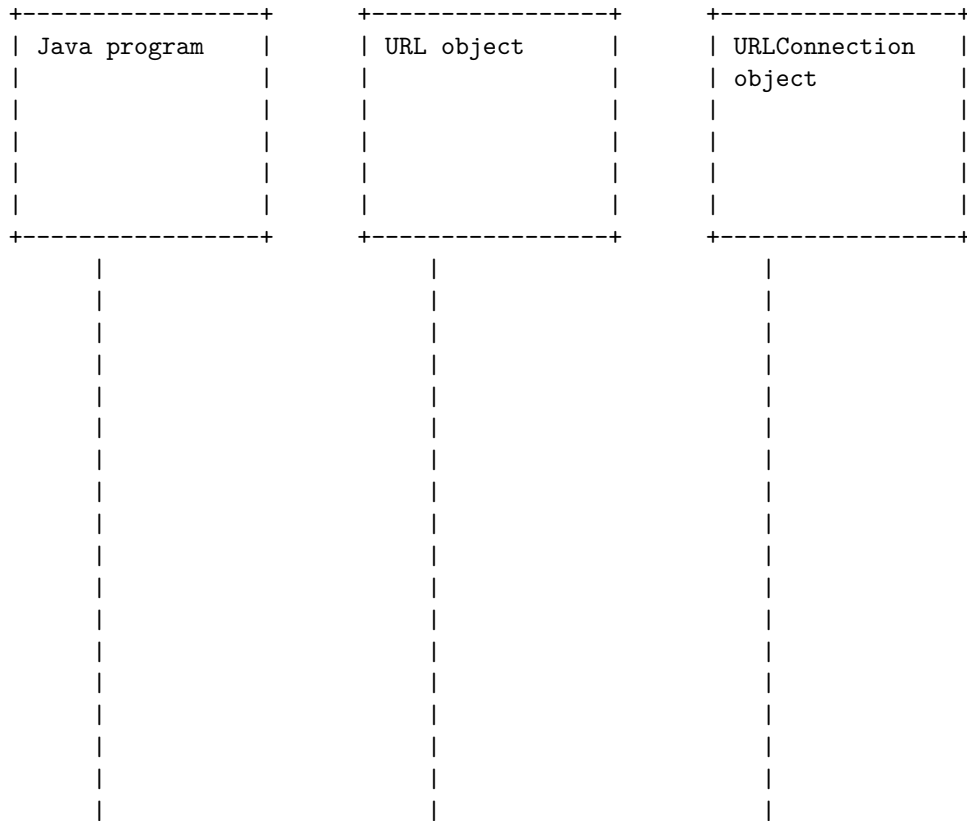


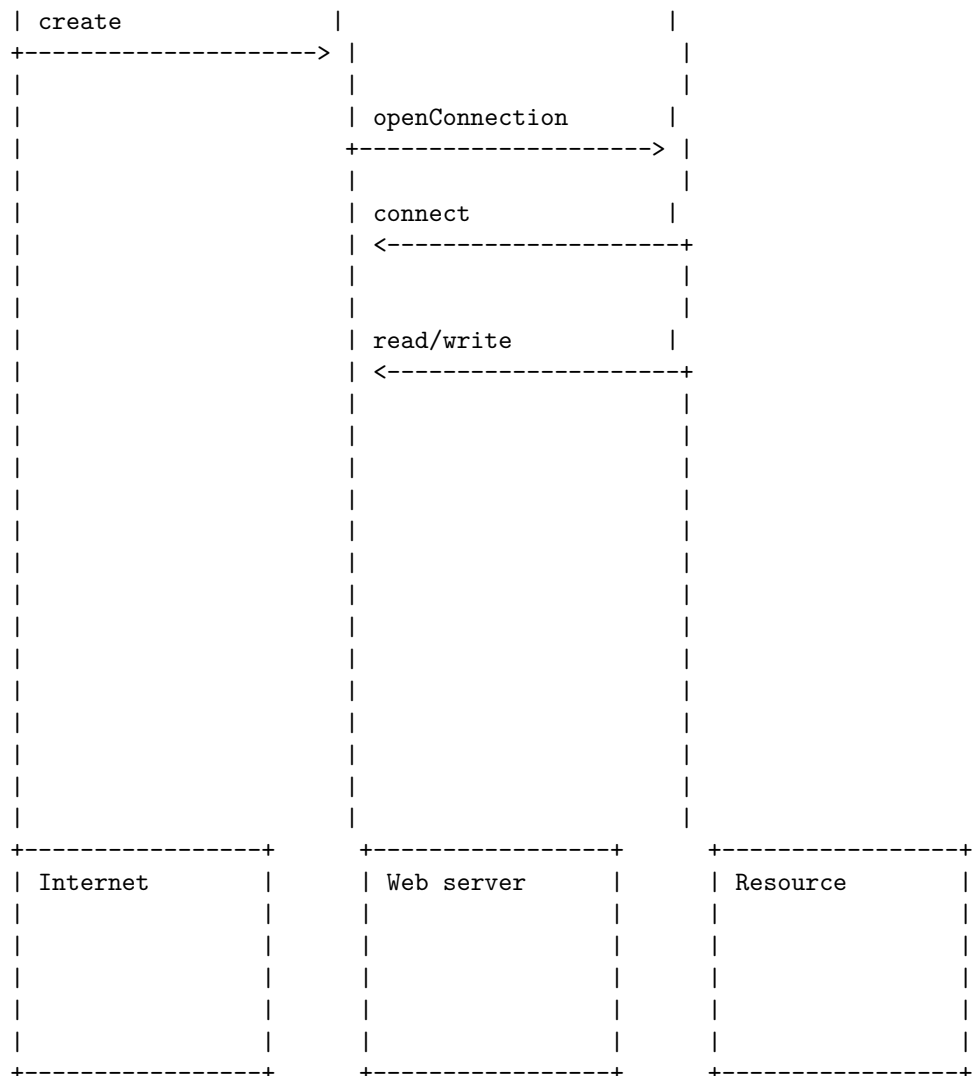


The protocol scheme indicates the network communication protocol used to access the resource, such as HTTP, HTTPS, FTP, etc. The authority consists of the hostname and the port number, separated by a colon. The hostname is the domain name of the server that hosts the resource, such as `www.example.com`. The port number is the numerical identifier of the network port used by the server to communicate with clients, such as 8080. The path specifies the location of the resource on the server, such as `/path/to/resource.html`. The path may also include query parameters and fragment identifiers, which provide additional information to the server or the client about the resource.

A URL connection in networking is a way of establishing a communication link between a Java program and a resource identified by a URL. A URL (Uniform Resource Locator) is a unique identifier that specifies the location and protocol of a resource on the Internet. A URL connection can be used to read from or write to the resource, depending on the protocol and the permissions.

The following diagram illustrates the basic architecture of a URL connection in networking using ASCII characters:

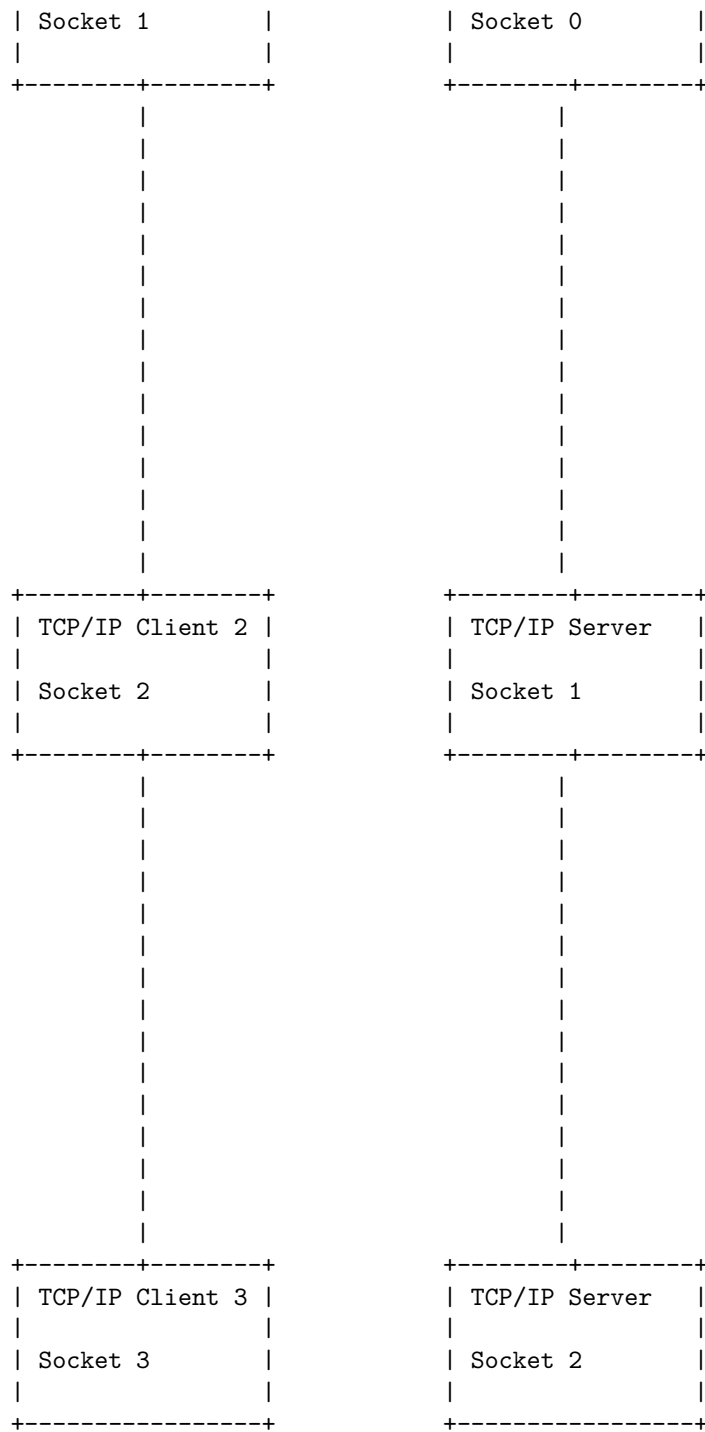




A TCP/IP server socket is a software structure that serves as an endpoint for sending and receiving data over a TCP/IP network. A TCP/IP server socket listens on a specific port number and accepts incoming connections from TCP/IP client sockets. A TCP/IP server socket can handle multiple concurrent connections from different clients by creating a new socket for each connection.

The following diagram illustrates the basic architecture of a TCP/IP server socket in networking:



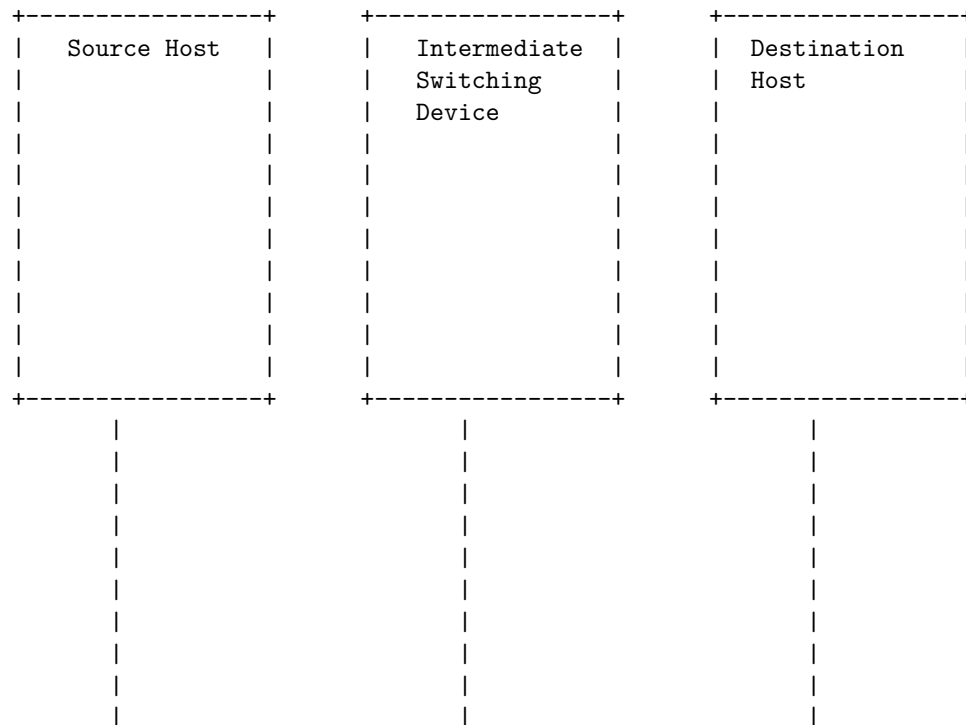


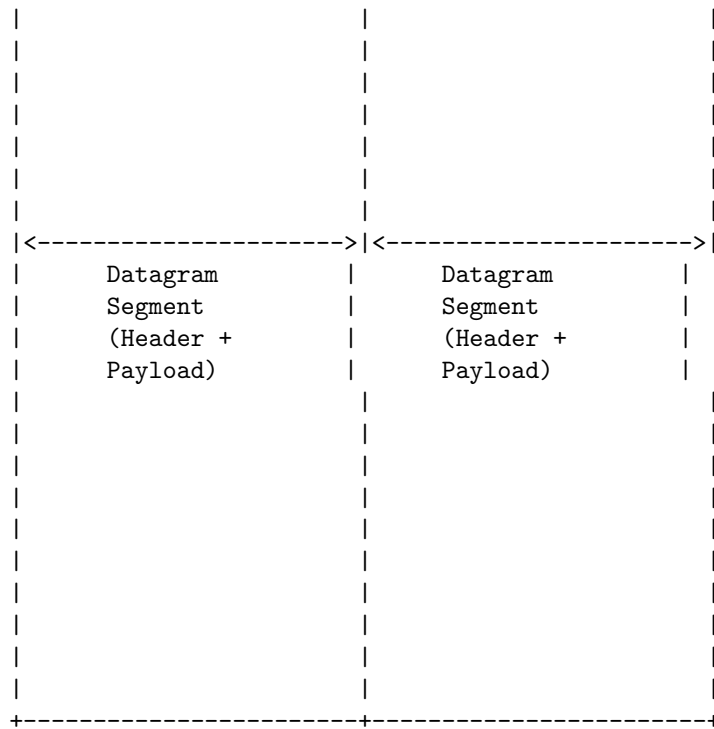
In this diagram, the TCP/IP server socket 0 is listening on port 80 and accepts incoming connections from three TCP/IP client sockets. For each connection, the TCP/IP server socket 0 creates a new socket (socket 1, socket 2, and socket 3) and assigns a different port number to communicate with the corresponding TCP/IP client socket. The TCP/IP server socket 0 can then handle multiple concurrent connections by using different sockets for different clients. The TCP/IP client sockets use the IP address and the port number of the TCP/IP server socket 0 to initiate the connection, and then use the IP address and the port number of the new socket assigned by the TCP/IP server socket 0 to send and receive data. The TCP/IP server socket 0 can close the connection by closing the corresponding socket for each client.

A datagram is a basic transfer unit associated with a packet-switched network. Datagrams are data packets which contain adequate header information so that they can be individually routed by all intermediate network switching devices to the destination. Datagrams provide a connectionless communication service across a packet-switched network. A datagram is an independent, self-contained message sent over the network whose arrival, arrival time, and content are not guaranteed.

Datagram in Networking

The following diagram illustrates the basic architecture of a datagram network:



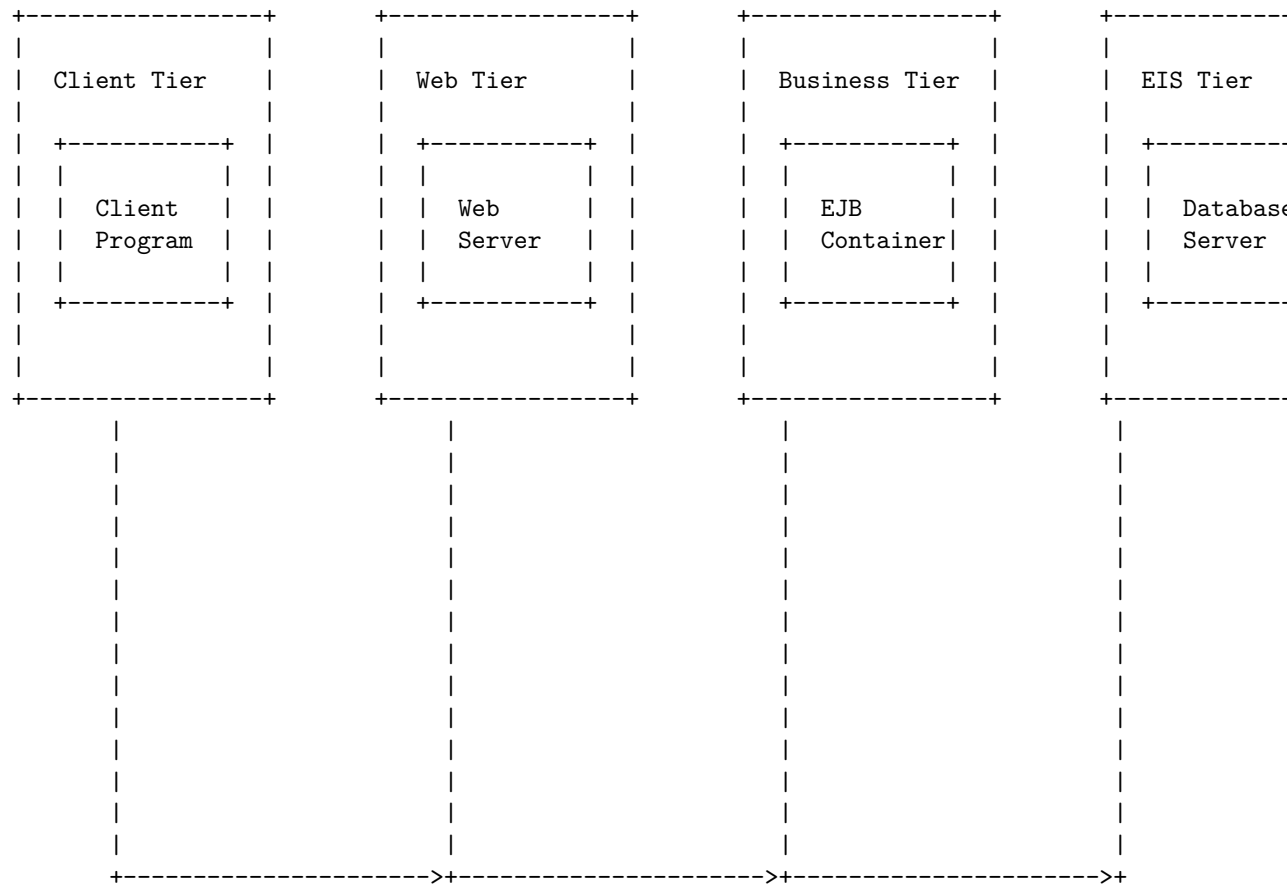


The source host divides the data into smaller parts called datagrams and adds a header to each datagram. The header contains information such as the source and destination addresses, the length of the datagram, the sequence number, and the checksum. The source host then sends the datagrams to the intermediate switching device, which may be a router, a switch, or a gateway. The intermediate switching device examines the header of each datagram and forwards it to the next hop along the route to the destination host. The intermediate switching device does not maintain any state information about the datagrams or the connection. The destination host receives the datagrams and reassembles them into the original data. The destination host may also perform error checking and retransmission requests if some datagrams are lost or corrupted. The destination host does not send any acknowledgment to the source host for the received datagrams. The datagram network does not guarantee the order, delivery, or integrity of the datagrams. The datagram network is suitable for applications that can tolerate some loss or delay of data, such as voice or video streaming. The datagram network is also scalable and robust, as it does not require any connection establishment or termination, and can handle network congestion and failures. The datagram network is also known as the connection-less network or the best-effort network.

Unit 4 - Enterprise Java Bean

Enterprise Java Bean (EJB) is a server-side component architecture for developing and deploying distributed, transactional, secure and portable applications based on Java technology. EJB is conceptually based on the Java RMI (Remote Method Invocation) specification. In EJB, the beans are run in a container having four-tier architecture .

The following diagram illustrates the basic architecture of a EJB application using ASCII art:



The client tier consists of the client program that accesses the EJB components. The client program can be a web browser, a standalone application, or another web server.

The web tier consists of the web server that hosts the web pages and servlets that communicate with the EJB components. The web server can use the Java EE web container to manage the web components.

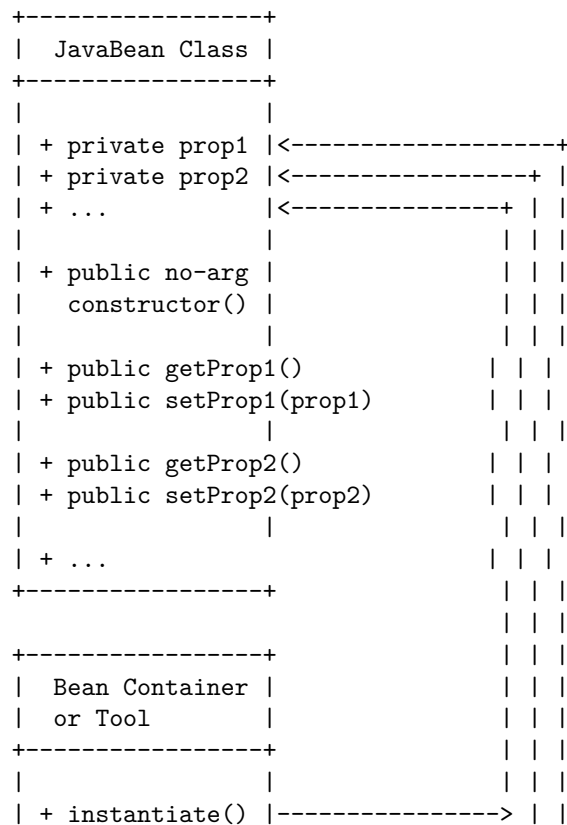
The business tier consists of the EJB server that hosts the EJB components. The EJB server can use the Java EE EJB container to manage the EJB components. The EJB components provide the business logic and services for the application.

The EIS tier consists of the database server that stores the data and resources for the application. The EJB components can access the database server using the Java EE Connector Architecture (JCA) or the Java Database Connectivity (JDBC) API.

A JavaBean is a Java class that follows some conventions to be used as a reusable component. To prepare a class to be a JavaBean, it must:

- Implement the `java.io.Serializable` interface, which allows the object to be saved and restored.
- Have a public no-argument constructor, which allows the object to be instantiated by a bean container or a tool.
- Have private properties with public getter and setter methods, which follow the naming convention of `getPropertyName` and `setPropertyName`. This allows the object to be manipulated by a bean container or a tool.

The following diagram illustrates the basic structure of a JavaBean class:



```

|               |               | |
| + manipulate() |-----+ |
|               |               |
| + save()       |-----+
| + restore()    |
|               |
+-----+

```

To create a Java bean, you need to follow some conventions:

- The class must implement the Serializable interface.
- The class must have a public no-argument constructor.
- The class must have private fields with public getter and setter methods.
- The class may have other methods or properties as needed.

Creating a JavaBean

The following diagram illustrates the basic architecture of a Java bean using ASCII art:

```

+-----+ +-----+ +-----+
| Java bean | | Service bean | | Data control |
| (Person.java) | | (PersonService.java) | (PersonServiceDC.java) |
+-----+ +-----+ +-----+
| -name: String | | | |
| -email: String | | | |
+-----+ +-----+ +-----+
| +getName(): String | | +getPersonList(): List<Person> | | |
| +setName(String): void | | | |
| +getEmail(): String | | | |
| +setEmail(String): void | | | |
+-----+ +-----+ +-----+

```

The Java bean class (Person.java) has two private fields (name and email) and four public methods (getName, setName, getEmail, setEmail) that follow the Java bean convention.

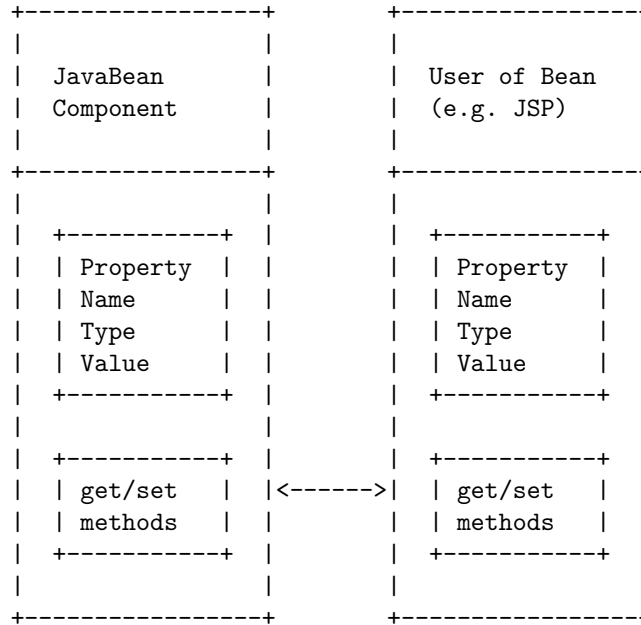
The service bean class (PersonService.java) has a public method (getPersonList) that returns a list of Person objects.

The data control class (PersonServiceDC.java) is generated from the service bean class and provides access to the data and methods of the service bean.

JavaBeans Properties are named attributes that can be accessed by the user of the object. The attribute can be of any Java data type, including the classes that you define. A JavaBean property may be read, write, read only, or write only .

JavaBeans Properties

The following diagram illustrates the basic architecture of a JavaBean property:



The JavaBean component defines the property name, type, and value, as well as the get and set methods to access and modify the property. The user of the bean can use the get and set methods to read and write the property value. The property type can be any Java data type, including primitive types, arrays, collections, and user-defined classes. The property name follows the Java naming conventions, starting with a lowercase letter and using camel case for multiple words. The get and set methods follow the JavaBeans naming conventions, starting with “get” or “set” followed by the capitalized property name. For example, if the property name is “color”, the get and set methods would be “getColor” and “setColor”. For boolean properties, the get method can also start with “is”, such as “isEditable”.

According to the web search results, there are three types of beans in Enterprise Java Bean: session beans, entity beans, and message-driven beans .

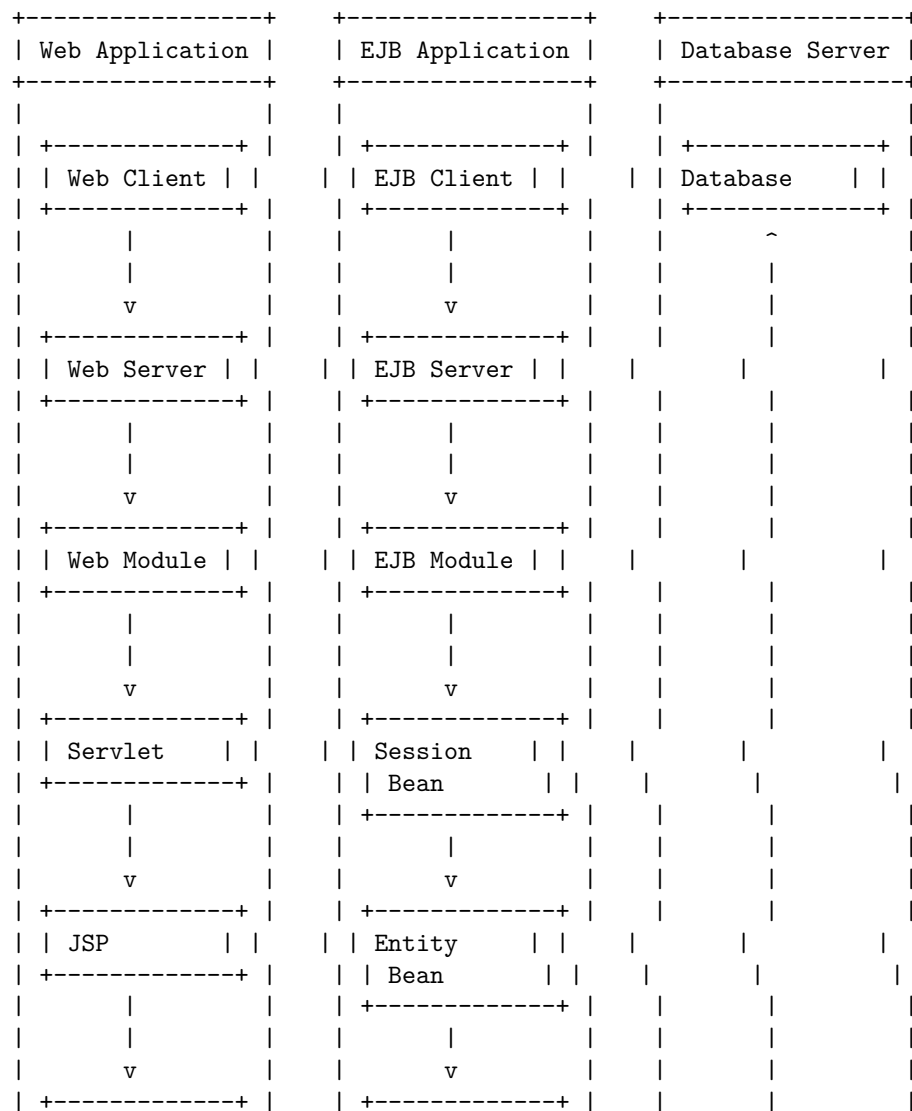
Session beans contain business logic that can be invoked by local, remote or web service clients. There are two types of session beans: stateful and stateless. Stateful session beans maintain a conversational state with the client, while stateless session beans do not. There is also a third type of session bean called singleton, which is instantiated only once and shared by all clients .

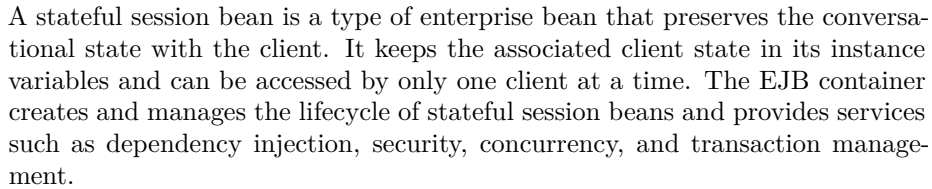
Entity beans represent persistent data stored in a database. They can be accessed by multiple clients and support transactions and concurrency. There are

two types of entity beans: container-managed and bean-managed. Container-managed entity beans delegate the persistence logic to the container, while bean-managed entity beans implement their own persistence logic .

Message-driven beans are used to process asynchronous messages from a message queue or a topic. They act as message consumers and can be triggered by the arrival of a message. They do not maintain any state and cannot be accessed directly by clients .

The following diagram illustrates the basic architecture of a Java EE application using different types of beans:



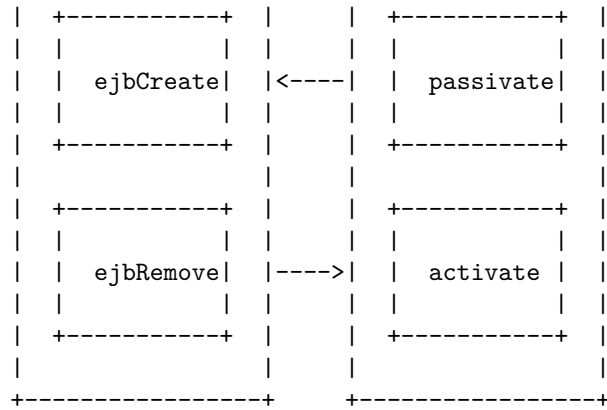


```

classDiagram
    class Client
    class EJB_Home["EJB Home"]
    class EJB_Object["EJB Object"]
    class EJB_Class["EJB Class"]
    class Bean_Pool["Bean Pool"]

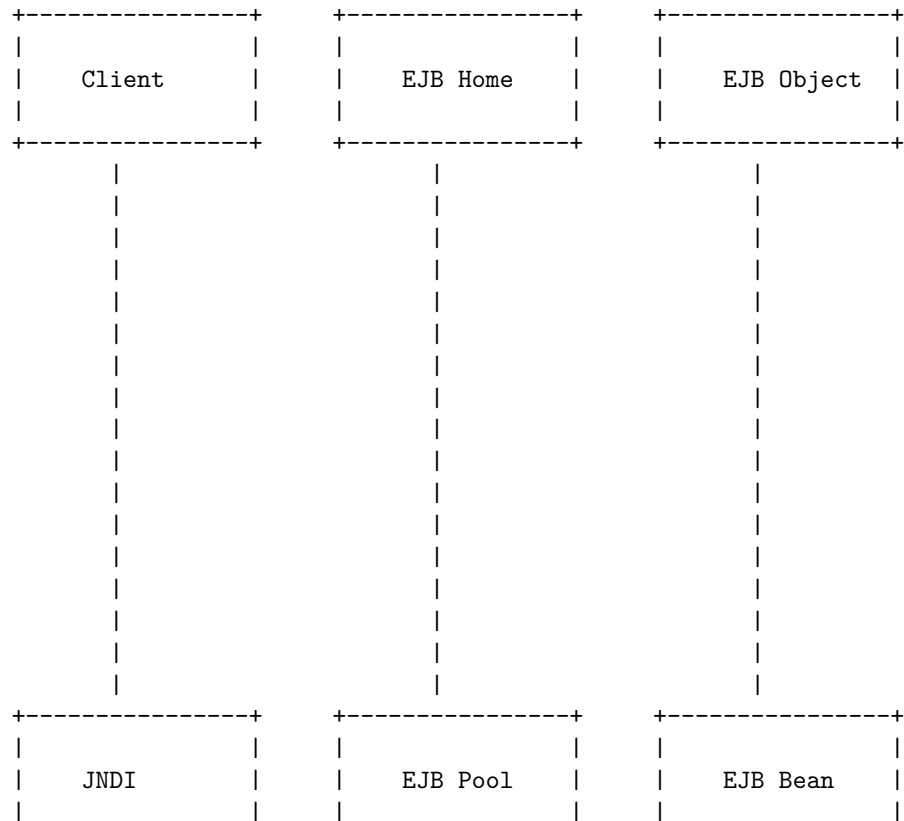
    Client --> EJB_Home : lookup()
    EJB_Home --> EJB_Object : create()
    EJB_Object --> EJB_Class : v
    EJB_Object --> Bean_Pool : v
  
```

The diagram illustrates the interaction between a Client and EJB components. The Client calls the `lookup()` method on the EJB Home. The EJB Home then calls the `create()` method on the EJB Object. The EJB Object contains business methods. The EJB Home and EJB Object are associated with the EJB Class and Bean Pool respectively, with vertical dashed lines indicating the association and 'v' at the bottom of each association line.



A stateless session bean is a type of enterprise bean, which is normally used to perform independent operations. A stateless session bean as per its name does not have any associated client state, but it may preserve its instance state .

The following diagram illustrates the basic architecture of a stateless session bean in enterprise java bean using ASCII art:



-----+ +-----+ +-----+

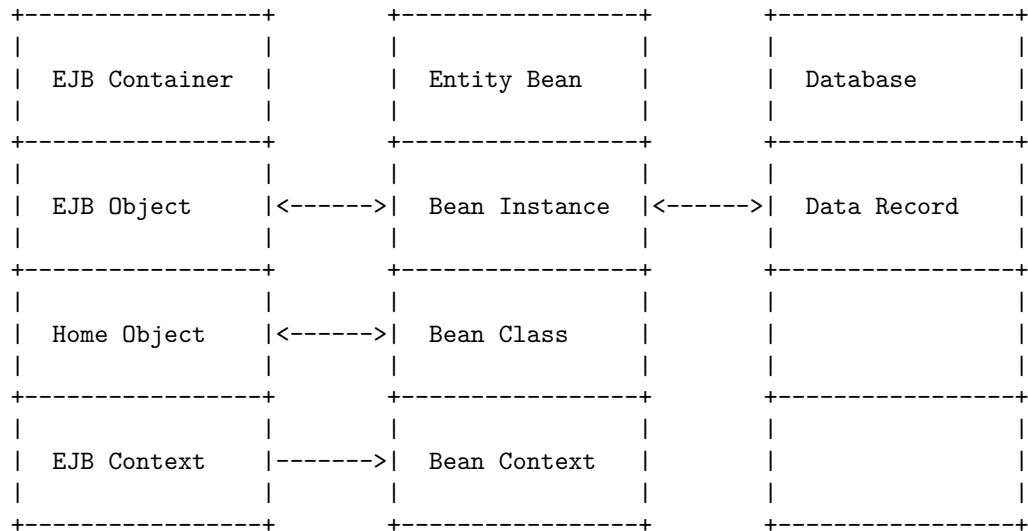
The steps involved in the diagram are:

- The client looks up the EJB home interface in the JNDI directory.
- The client invokes a method on the EJB home interface to create an EJB object.
- The EJB container selects an available EJB bean from the EJB pool and assigns it to the EJB object.
- The EJB object invokes the business method on the EJB bean.
- The EJB bean performs the business logic and returns the result to the EJB object.
- The EJB object returns the result to the client.
- The EJB bean is returned to the EJB pool for reuse by other EJB objects.

An entity bean is a type of Enterprise JavaBean (EJB), a server-side Java EE component, that represents persistent data maintained in a database. An entity bean can manage its own persistence (bean managed persistence) or can delegate this function to its EJB container (container managed persistence). An entity bean is identified by a primary key.

Entity bean in Enterprise Java Bean

The following diagram shows the basic architecture of an entity bean in Enterprise Java Bean:



The EJB container provides the runtime environment for the entity bean and manages its lifecycle, security, transactions, concurrency, and persistence. The EJB container also provides the EJB object, the home object, and the EJB context for the entity bean.

The EJB object is the proxy for the entity bean that implements the business interface and handles the client requests. The EJB object delegates the requests to the bean instance, which is the actual object that represents the entity bean's state and behavior.

The home object is the factory for the entity bean that implements the home interface and provides methods for creating, finding, and removing entity beans. The home object interacts with the bean class, which is the class that defines the entity bean's properties, methods, and annotations.

The EJB context is the interface that allows the entity bean to access the container services, such as security, transactions, and naming. The EJB context is passed to the bean context, which is the interface that allows the entity bean to access the bean instance, the primary key, and the bean managed persistence.

The database is the external data source that stores the persistent data for the entity bean. The database contains the data record, which is the row that corresponds to the entity bean's primary key and attributes. The database can be accessed by the entity bean either directly (bean managed persistence) or indirectly (container managed persistence).

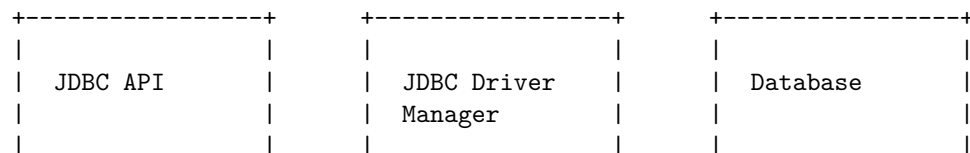
Java Database Connectivity (JDBC) is an API that allows Java programs to interact with various databases such as Oracle, MySQL, MS Access and SQL Server. JDBC supports both two-tier and three-tier processing models for database access.

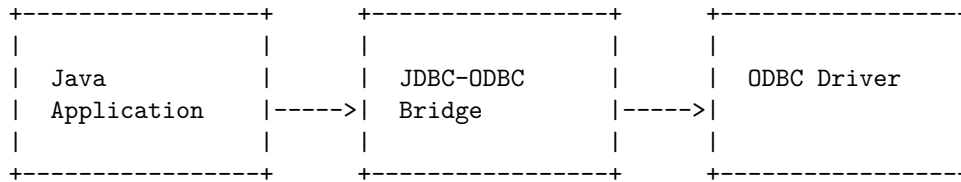
Java Database Connectivity (JDBC) Architecture

The JDBC architecture consists of four main components:

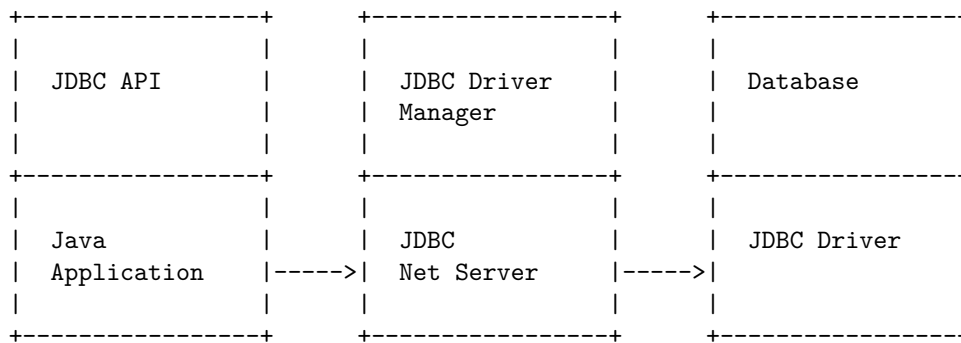
- The JDBC API: This defines the interfaces and classes that enable Java applications to execute SQL statements, process the results, and manage transactions.
- The JDBC Driver Manager: This is a class that manages the loading and registration of JDBC drivers, and provides a connection to a database through the appropriate driver.
- The JDBC Driver: This is a software component that implements the JDBC API for a specific database. It converts the JDBC calls into the database-specific protocol and communicates with the database server.
- The Database: This is the data source that stores the data and responds to the queries and updates from the JDBC driver.

The following diagram illustrates the basic architecture of a JDBC application:



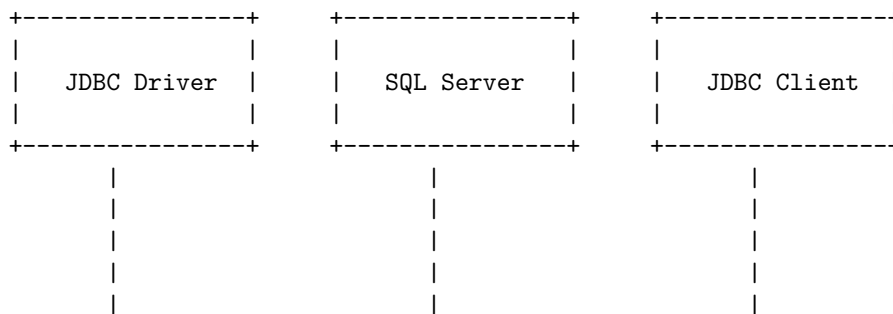


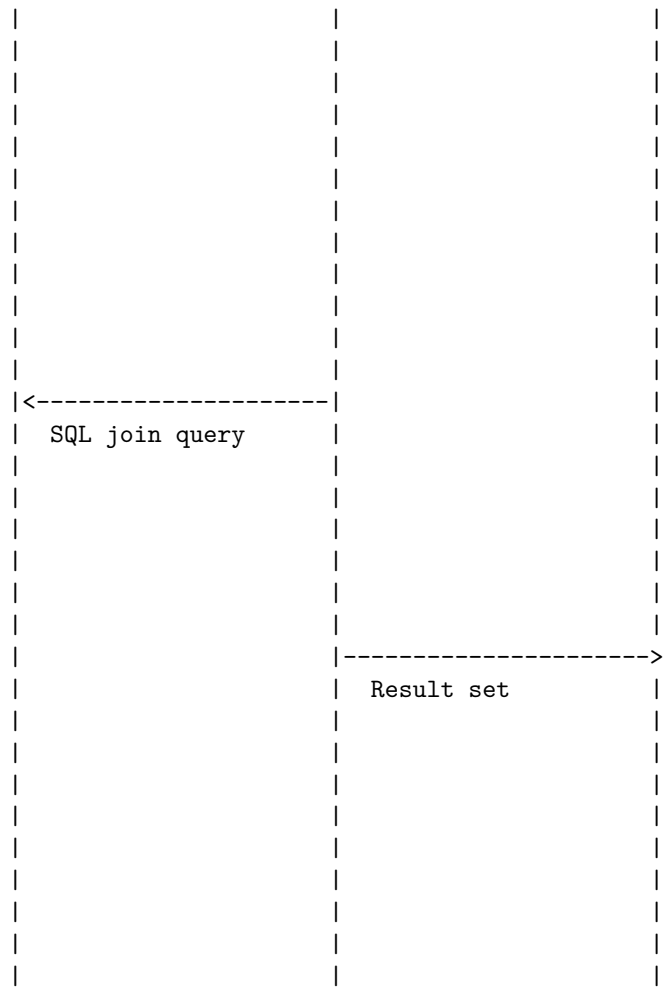
In the two-tier model, the JDBC application communicates directly with the database through the JDBC driver. In the three-tier model, the JDBC application communicates with a middle-tier server that handles the database access and business logic, and the server communicates with the database through the JDBC driver. The following diagram illustrates the three-tier architecture of a JDBC application:



Merging data from multiple tables in JDBC is a process of combining the data from different sources into a single result set. There are different ways to achieve this, such as using SQL joins, subqueries, or custom code. One of the common methods is to use SQL joins, which allow you to specify the conditions for matching the rows from different tables based on common fields or expressions. There are different types of SQL joins, such as inner join, outer join, cross join, natural join, etc. Each type of join has a different effect on the result set and the performance of the query.

The following diagram illustrates the basic architecture of a SQL join operation in JDBC:





The diagram shows the following steps:

- The JDBC client initiates a connection to the SQL server using the JDBC driver.
- The JDBC client sends a SQL join query to the SQL server using the JDBC driver.
- The SQL server executes the SQL join query and joins the data from multiple tables based on the join conditions.
- The SQL server returns the result set to the JDBC client using the JDBC driver.
- The JDBC client processes the result set and displays the merged data.

Joining in JDBC is the process of combining data from two or more tables based on a common column or condition. JDBC provides the `JoinRowSet` interface to perform join operations on `RowSet` objects. A `RowSet` object is a container for

The following diagram illustrates the basic architecture of a join operation in JDBC using the `JoinRowSet` interface:



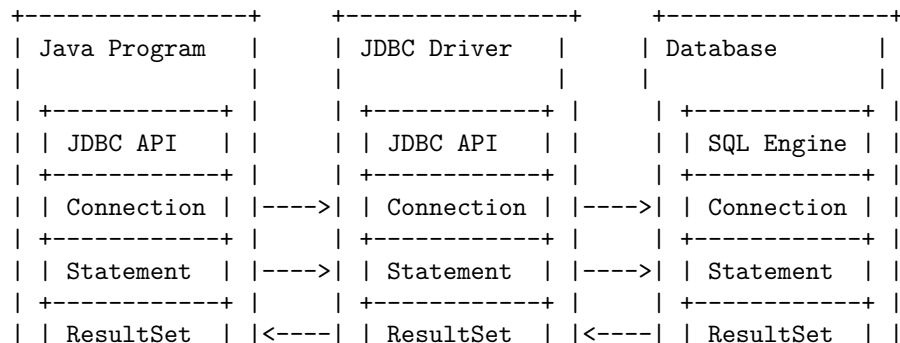
	V		V		V
+-----+-----+-----+					
	JoinRowSet				
+-----+-----+-----+					
	Column 1 (PK)		Column 2		Column 3
					Column 4
					...
+-----+-----+-----+					
	Row 1		Row 1		Row 1
	Row 2		Row 2		Row 2
	Row 3		Row 3		Row 3
	Row 4		Row 4		Row 4
	...		...		...
+-----+-----+-----+					

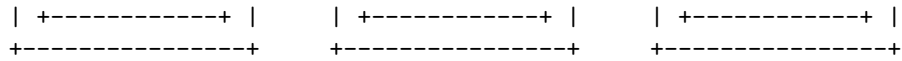
The JoinRowSet object contains the result of the join operation, which can be accessed and manipulated using the methods of the RowSet interface. The join operation can be performed using different types of joins, such as inner join, left outer join, right outer join, full outer join, or cross join. The type of join can be specified using the setJoinType method of the JoinRowSet interface. The join operation can also be performed on more than two RowSet objects, as long as they have a common match column. The match column is the column on which the join is based, and it must be a primary key or a unique column in each RowSet object. The match column can be specified using the setMatchColumn method of the RowSet interface.

Manipulating in JDBC means using the JDBC API to create, insert into, update, and query tables in a database. JDBC is a Java-based interface that allows Java applications to connect to and interact with various types of databases. JDBC drivers are the software components that implement the JDBC API for different database vendors.

Manipulating in JDBC

The following diagram illustrates the basic architecture of manipulating in JDBC using ASCII characters:





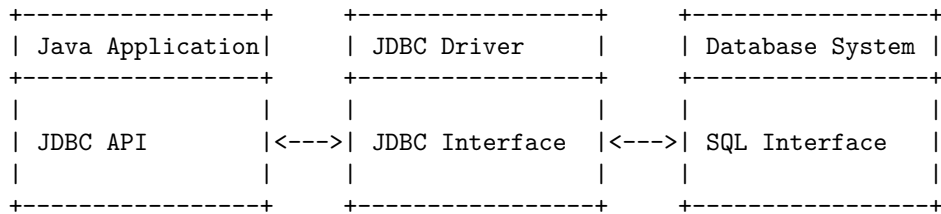
The diagram shows the following steps:

- The Java program uses the JDBC API to create a Connection object that represents a connection to the database.
- The Java program uses the Connection object to create a Statement object that represents a SQL statement to be executed on the database.
- The Java program uses the Statement object to execute the SQL statement and obtain a ResultSet object that represents the result of the query.
- The Java program uses the ResultSet object to access and manipulate the data returned by the query.

Databases with JDBC in JDBC

JDBC stands for Java Database Connectivity, which is an API that allows Java applications to interact with various types of databases using a common interface. JDBC consists of two main components: the JDBC driver and the JDBC API. The JDBC driver is a software module that implements the JDBC interface for a specific database system, such as MySQL, Oracle, or SQL Server. The JDBC API is a set of classes and interfaces that define the methods and properties for connecting to a database, executing SQL statements, and processing the results.

The following diagram illustrates the basic architecture of a JDBC application:



The JDBC application uses the JDBC API to communicate with the JDBC driver, which in turn communicates with the database system using the SQL interface. The JDBC driver acts as a bridge between the Java and SQL worlds, translating the JDBC calls into SQL commands and the SQL results into Java objects. The JDBC driver can be either embedded in the Java application or loaded dynamically at runtime.

To connect to a database using JDBC, the application needs to specify a JDBC URL, which is a string that identifies the database system, the host, the port, the database name, and any other properties. The JDBC URL format can vary depending on the database system and the JDBC driver, but it usually follows this general pattern:

`jdbc:<subprotocol>:<subname>`

where is the name of the database system, such as mysql, oracle, or sqlserver, and is a database-specific string that contains the host, port, database name, and other properties. For example, a JDBC URL for MySQL could look like this:

```
jdbc:mysql://mysql.db.server:3306/my_database?useSSL=false&serverTimezone=UTC
```

where mysql.db.server is the host name, 3306 is the port number, my_database is the database name, and useSSL and serverTimezone are some properties. A JDBC URL for Oracle could look like this:

```
jdbc:oracle:thin:@oracle.db.server:1521:my_database
```

where oracle.db.server is the host name, 1521 is the port number, and my_database is the database name. A JDBC URL for SQL Server could look like this:

```
jdbc:sqlserver://sqlserver.db.server:1433;databaseName=my_database;integratedSecurity=true
```

where sqlserver.db.server is the host name, 1433 is the port number, my_database is the database name, and integratedSecurity is a property.

To connect to a database using JDBC, the application also needs to provide a user name and a password, which are used to authenticate the connection. The user name and password can be either passed as parameters to the JDBC API methods, or included in the JDBC URL as properties. For example, a JDBC URL for MySQL with user name and password could look like this:

```
jdbc:mysql://mysql.db.server:3306/my_database?user=my_user&password=my_password
```

where my_user is the user name and my_password is the password. Alternatively, the user name and password can be passed as parameters to the DriverManager.getConnection method, which is one of the JDBC API methods for establishing a connection. For example, in Java, the code for connecting to a MySQL database could look like this:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class JDBCExample {

    public static void main(String[] args) {

        // JDBC URL for MySQL
        String url = "jdbc:mysql://mysql.db.server:3306/my_database";

        // User name and password
        String user = "my_user";
        String password = "my_password";
```

```

// Connection object
Connection conn = null;

try {
    // Load the JDBC driver
    Class.forName("com.mysql.jdbc.Driver");

    // Establish the connection
    conn = DriverManager.getConnection(url, user, password);

    // Do some database operations
    // ...

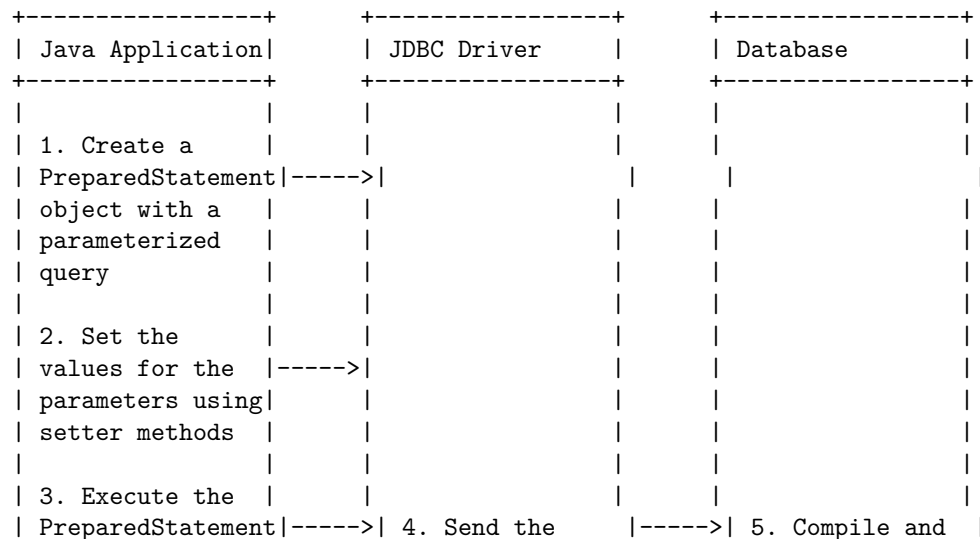
} catch (ClassNotFoundException e) {
    // Handle the exception for loading the driver
    e.printStackTrace();
} catch (SQLException

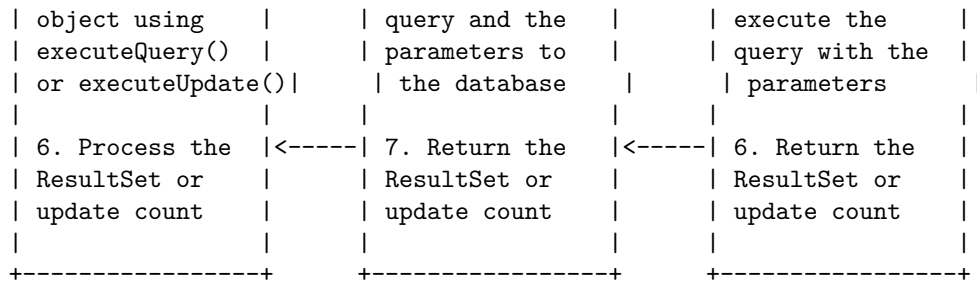
```

Prepared Statements in JDBC are a special type of statements that allow you to execute parameterized queries against the database. A parameter is represented by a question mark (?) symbol in JDBC. Prepared Statements are precompiled by the database and can be executed multiple times with different values for the parameters. Prepared Statements extend the Statement interface and provide methods to set the values for the parameters and execute the query.

Prepared Statements in JDBC

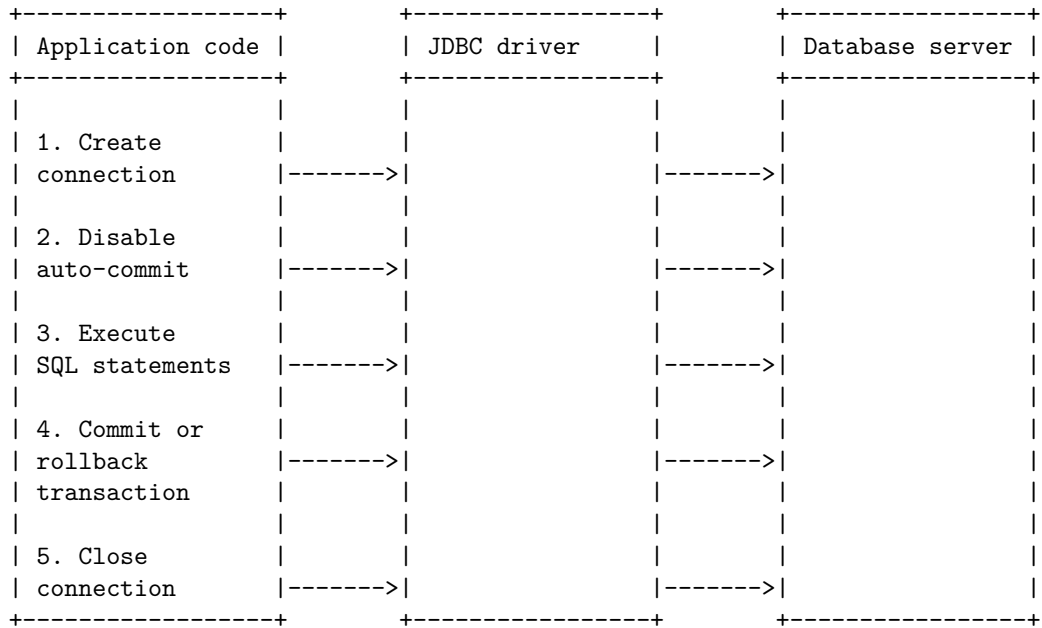
The following diagram illustrates the basic architecture of a Prepared Statement in JDBC:





Transaction processing in JDBC is a way of ensuring the consistency and integrity of the data in a database by executing a set of SQL statements as a unit. A transaction is either committed or rolled back, meaning that all the changes made by the statements are either saved or discarded. JDBC supports both local and distributed transactions, depending on the driver and the database.

The following diagram illustrates the basic architecture of a local transaction in JDBC:



The steps are as follows:

1. The application code creates a connection object by using the JDBC driver.
2. The application code disables the auto-commit mode of the connection, which means that the SQL statements will not be committed automatically after execution.
3. The application code executes one or more SQL statements using the

connection object. The statements are sent to the database server by the JDBC driver.

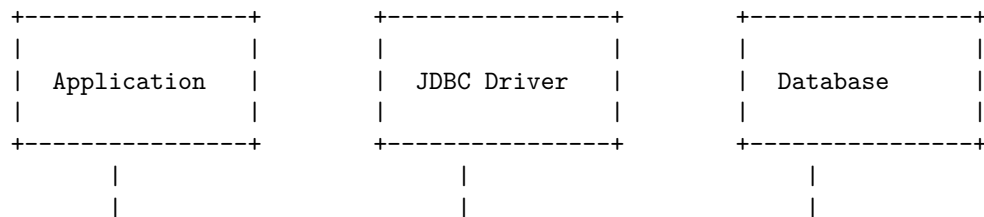
4. The application code decides whether to commit or rollback the transaction, based on the results of the SQL statements. The commit or rollback operation is performed by the JDBC driver on the database server.
5. The application code closes the connection object, which releases the resources used by the transaction.

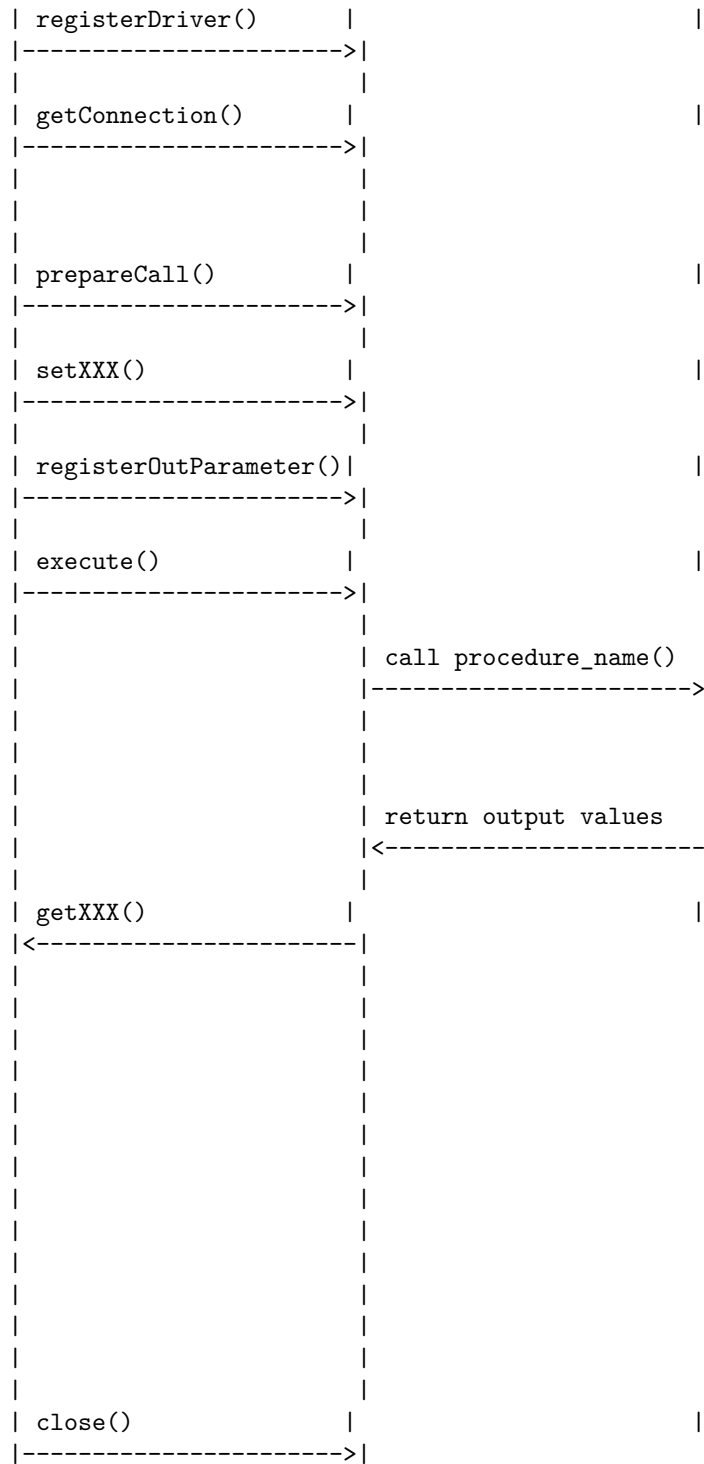
Stored procedures are subroutines, segments of SQL statements that are stored in the SQL catalog. They can be accessed by applications that can access relational databases, such as Java, Python, PHP, etc. Stored procedures can improve performance, security, and modularity of database applications.

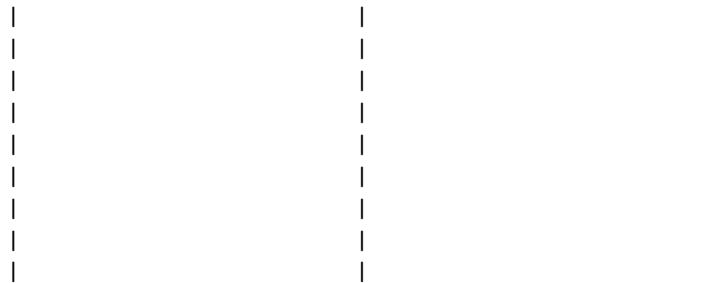
To call a stored procedure using JDBC, you need to:

- Register the driver class using the `registerDriver()` method of the `DriverManager` class.
- Establish a connection to the database using the `getConnection()` method of the `DriverManager` class.
- Create a `CallableStatement` object using the `prepareCall()` method of the `Connection` object. The `prepareCall()` method takes a string argument that specifies the SQL escape syntax for calling the stored procedure. The syntax is: `{call procedure_name[(?, ?, ...)]}`
- If the stored procedure has input parameters, use the `setXXX()` methods of the `CallableStatement` object to bind values to the parameters. The `setXXX()` methods take two arguments: the parameter index and the parameter value.
- If the stored procedure has output parameters, use the `registerOutParameter()` method of the `CallableStatement` object to bind the JDBC data type to the data type the stored procedure expects for the output values. The `registerOutParameter()` method takes two arguments: the parameter index and the JDBC data type.
- Execute the stored procedure using the `execute()` method of the `CallableStatement` object.
- If the stored procedure returns output values, use the `getXXX()` methods of the `CallableStatement` object to retrieve them. The `getXXX()` methods take one argument: the parameter index.

The following diagram illustrates the basic architecture of a stored procedure in JDBC using ASCII art:



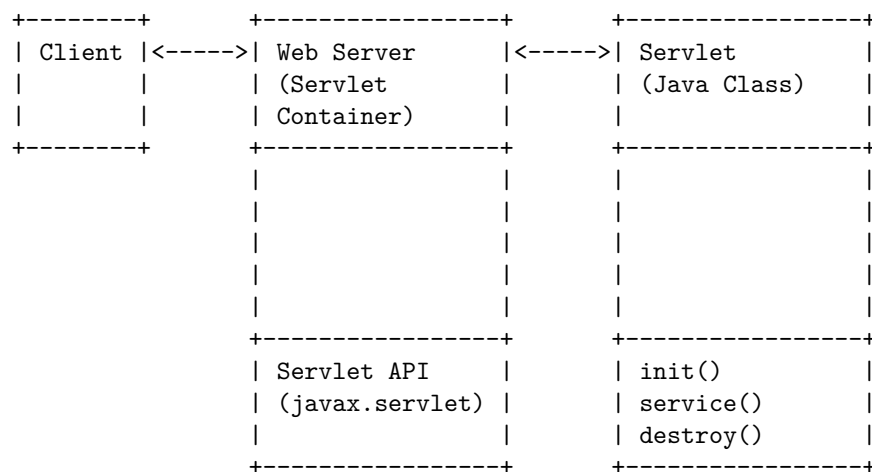




Unit 5 - Servlets

Servlets are Java classes that run on a web server and handle HTTP requests and responses. Servlets can be used to create dynamic web applications that interact with databases, process forms, generate HTML pages, and more.

The following diagram illustrates the basic architecture of a servlet:



The servlet architecture consists of the following components:

- **Client:** The client is the browser or any other application that sends HTTP requests to the web server and receives HTTP responses from the servlet. The client can also send parameters, cookies, headers, and other information along with the request.
- **Web Server:** The web server is the software that handles the incoming HTTP requests and delegates them to the servlet container. The web server also sends back the HTTP responses generated by the servlet to the client. The web server can be Apache Tomcat, Jetty, GlassFish, etc.
- **Servlet Container:** The servlet container is the component that manages the lifecycle and execution of the servlets. The servlet container loads, initializes, invokes, and destroys the servlets. The servlet container also

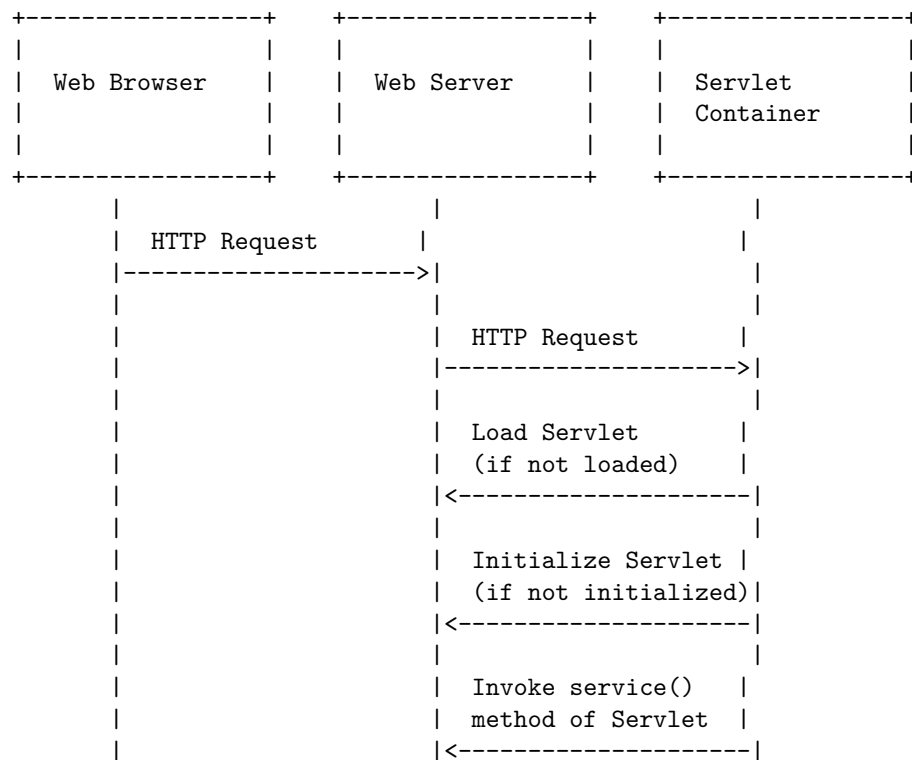
provides the servlet API, which is a set of interfaces and classes that define the communication between the servlet and the web server.

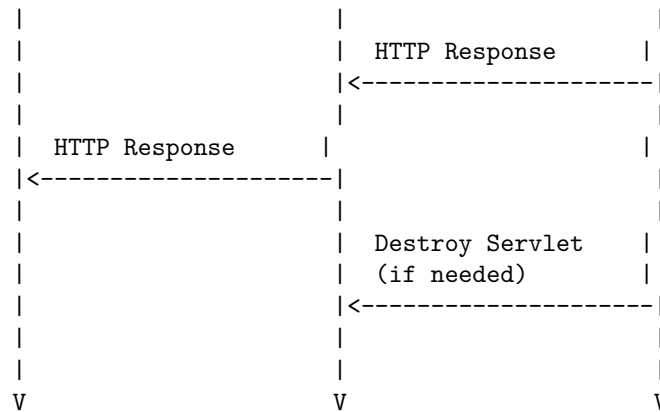
- **Servlet:** The servlet is the Java class that implements the `javax.servlet.Servlet` interface and overrides the `init()`, `service()`, and `destroy()` methods. The servlet can access the request and response objects, which contain the information about the HTTP request and response. The servlet can also perform business logic, access databases, generate dynamic content, set cookies and headers, and redirect or forward the request to other resources.

A servlet is a Java class that runs on a web server and handles HTTP requests and responses. Servlets can be used to create dynamic web applications that process user input, generate HTML pages, access databases, and perform other server-side tasks.

Servlets are managed by a servlet container, which is a component of a web server that provides the environment for servlets to run. The servlet container is responsible for loading, initializing, executing, and destroying servlets. It also handles the communication between servlets and web clients.

The following is a detailed ASCII diagram of the servlet overview and architecture:





Interface Servlet and the Servlet Life Cycle in Servlets

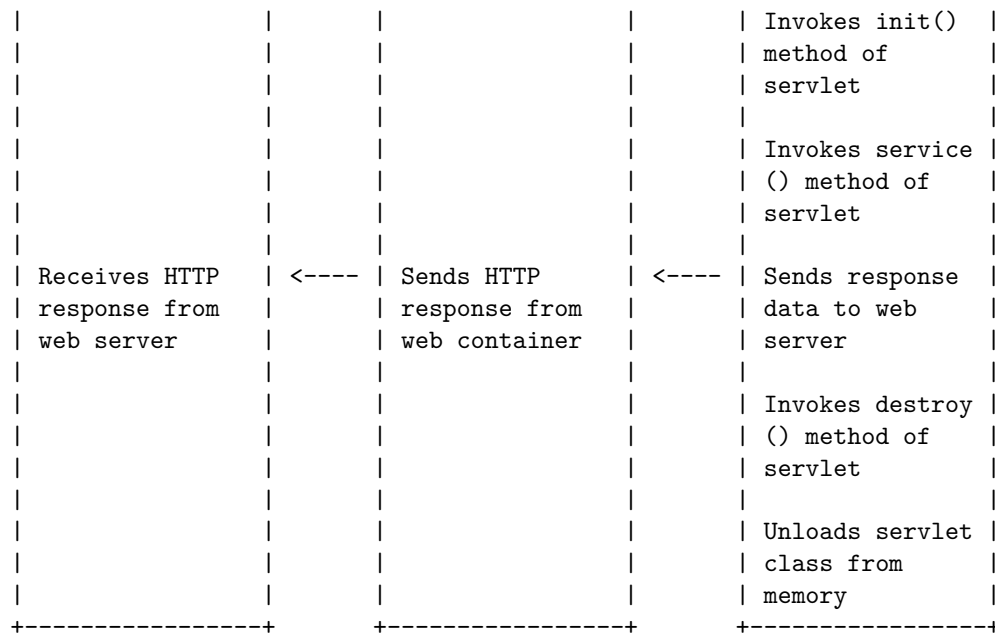
The Servlet interface defines the methods that a servlet must implement to interact with the web container and handle client requests. All servlets must implement this interface either directly or by extending a class that implements it .

The Servlet interface provides the following life cycle methods :

- **init()**: This method is invoked by the web container when the servlet is loaded into the memory. It is used to initialize the servlet with configuration parameters and resources. It is called only once during the servlet's lifetime.
- **service()**: This method is invoked by the web container to process a client request. It is called for each request that the servlet receives. It reads the request data, generates the response data, and sends the response back to the client.
- **destroy()**: This method is invoked by the web container when the servlet is unloaded from the memory. It is used to release any resources that the servlet has acquired. It is called only once at the end of the servlet's lifetime.

The following diagram illustrates the basic architecture of a servlet and its life cycle using ASCII art:



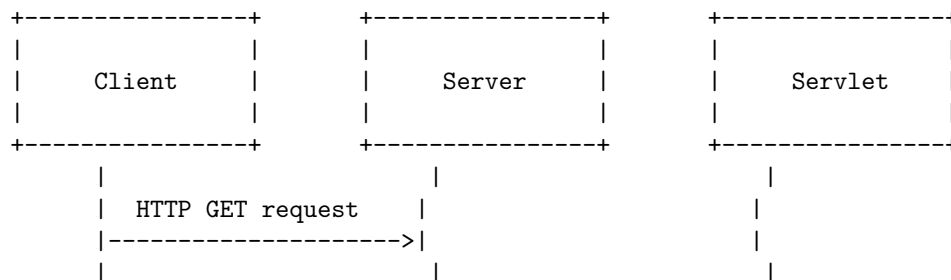


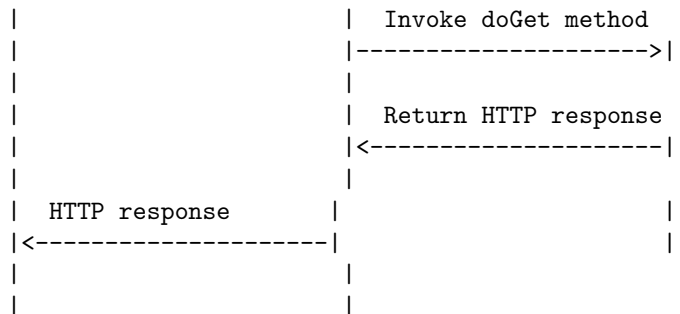
Handling HTTP get Requests in Servlets

HTTP GET requests are used to retrieve information from a web server, such as a web page, an image, or a file. A servlet is a Java class that runs on a web server and can handle HTTP requests and generate HTTP responses.

To handle HTTP GET requests in a servlet, you need to extend the `HttpServlet` class and override the `doGet` method. The `doGet` method takes two parameters: an `HttpServletRequest` object and an `HttpServletResponse` object. The `HttpServletRequest` object contains information about the request, such as the URL, the headers, the parameters, and the cookies. The `HttpServletResponse` object is used to send information back to the client, such as the status code, the headers, the content type, and the body.

The following diagram illustrates the basic architecture of a servlet for handling HTTP GET requests:





The steps involved in handling HTTP GET requests in a servlet are:

1. The client sends an HTTP GET request to the server, specifying the URL of the servlet.
2. The server receives the request and invokes the service method of the servlet. The service method determines the HTTP method of the request and calls the corresponding method of the servlet. In this case, it calls the `doGet` method.
3. The `doGet` method of the servlet receives the `HttpServletRequest` and `HttpServletResponse` objects as parameters. It can use the `HttpServletRequest` object to access the information about the request, such as the parameters, the headers, and the cookies. It can use the `HttpServletResponse` object to set the information about the response, such as the status code, the headers, the content type, and the body. It can also use the `PrintWriter` object obtained from the `getWriter` method of the `HttpServletResponse` object to write the content of the response.
4. The `doGet` method of the servlet returns the HTTP response to the server, which then sends it back to the client.
5. The client receives the HTTP response and displays the content of the response.

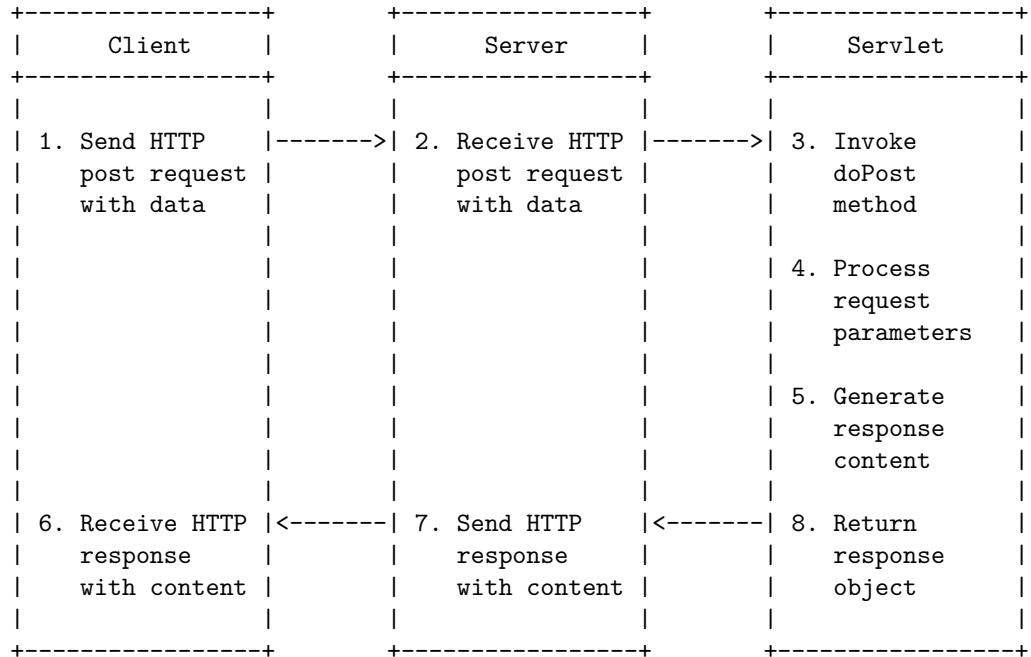
Handling HTTP post Requests in Servlets

The HTTP post method is used to send data to the server in the body of the request. The data is usually encoded in a key-value format, such as `name=John&age=25`. The post method is suitable for sending large amounts of data or sensitive data that should not be exposed in the URL.

To handle HTTP post requests in servlets, you need to extend the `HttpServlet` class and override the `doPost` method. The `doPost` method takes two parameters: a `HttpServletRequest` object and a `HttpServletResponse` object. The `HttpServletRequest` object contains the information about the request, such as the headers, the parameters, the cookies, etc. The `HttpServletResponse` object is used to send the response back to the client, such as the status code, the headers, the content, etc.

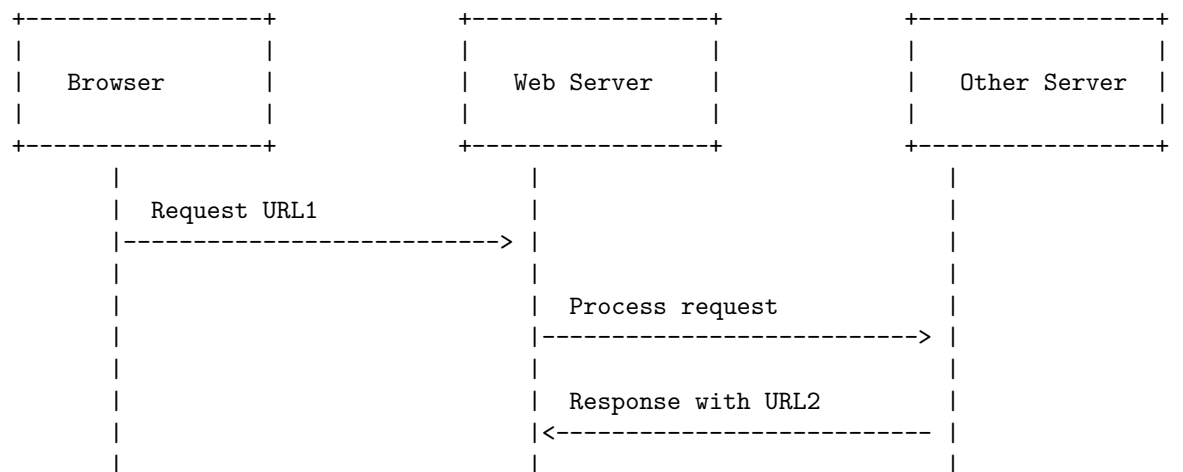
The following diagram illustrates the basic architecture of handling HTTP post

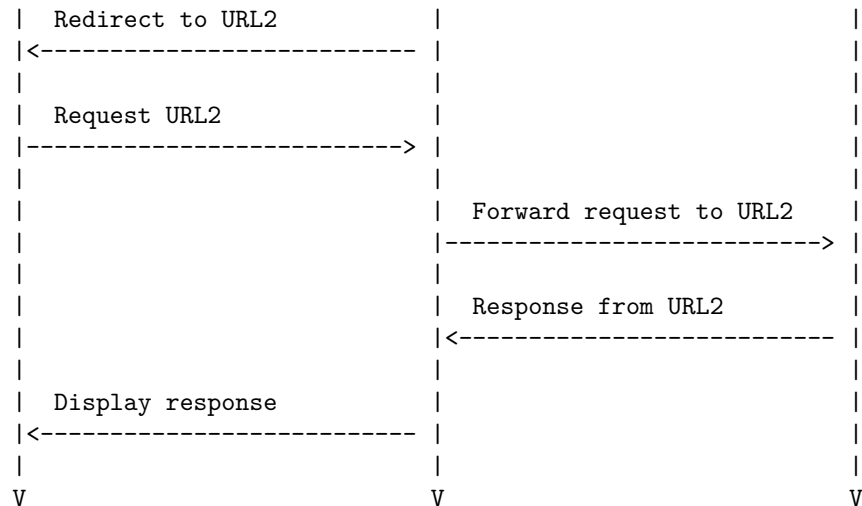
requests in servlets using ASCII art:



Redirecting requests to other resources in servlets is a technique that allows a servlet to send a response to another resource, such as another servlet, a JSP page, or an HTML file. This can be done by using the `sendRedirect()` method of the `HttpServletResponse` interface, which takes a URL as an argument and instructs the browser to make a new request to that URL.

The following diagram illustrates the basic architecture of a redirecting request in servlets:

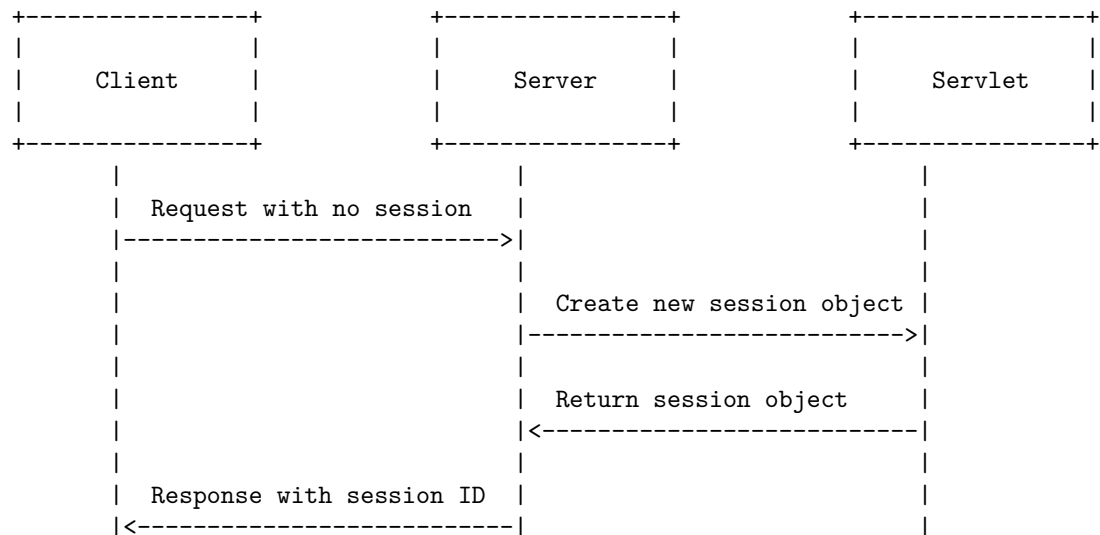


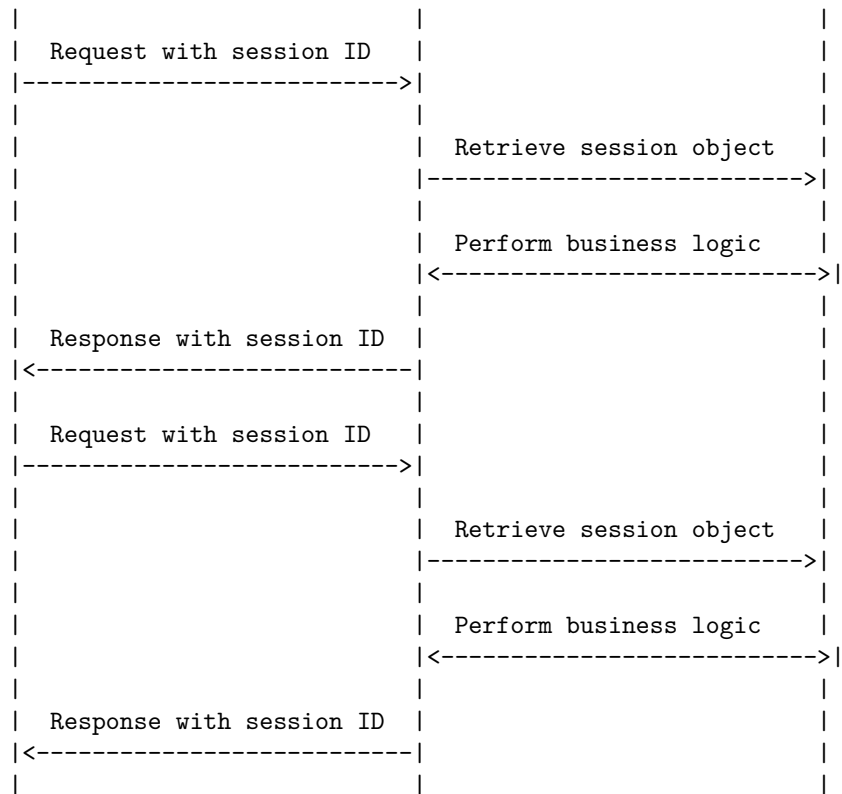


Session tracking is a mechanism that servlets use to maintain state about a series of requests from the same user across some period of time. Sessions are shared among the servlets accessed by a client. There are four techniques used in session tracking: cookies, hidden form fields, URL rewriting and HttpSession. HttpSession is an interface that provides a way to identify a user across more than one page request or visit to a website and to store information about that user.

The following diagram illustrates the basic architecture of a session tracking using HttpSession in servlets:

Session Tracking in Servlets





: <https://www.cs.fsu.edu/~jtbauer/cis3931/tutorial/servlets/client-state/session-tracking.html>
<https://www.javatpoint.com/session-tracking-in-servlets>
<https://www.c-sharpcorner.com/article/session-tracking-using-the-http-session-interface-in-servlets/>

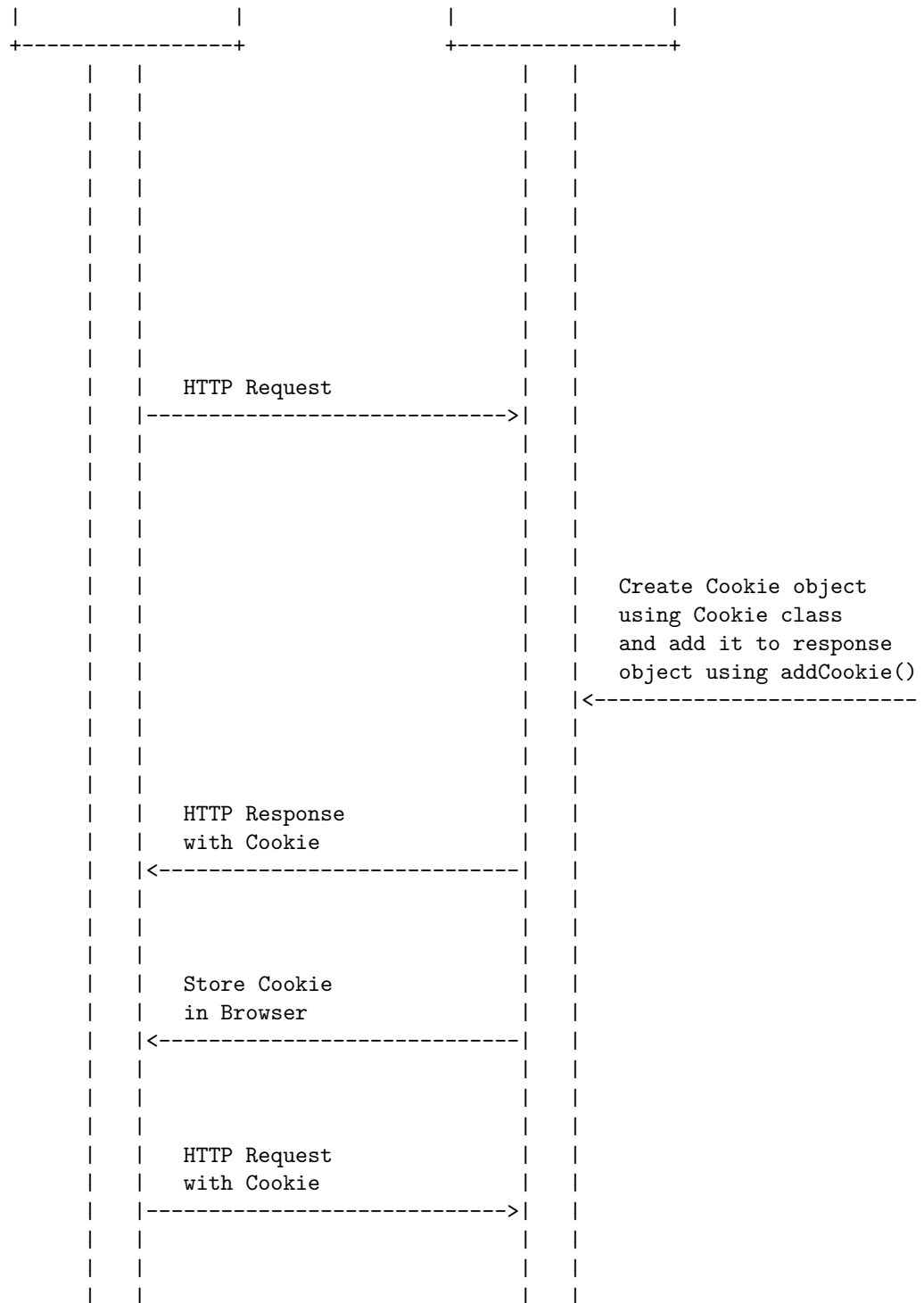
A cookie is a small piece of information that is persisted between the multiple client requests. A cookie has a name, a single value, and optional attributes such as a comment, path and domain qualifiers, a maximum age, and a version number.

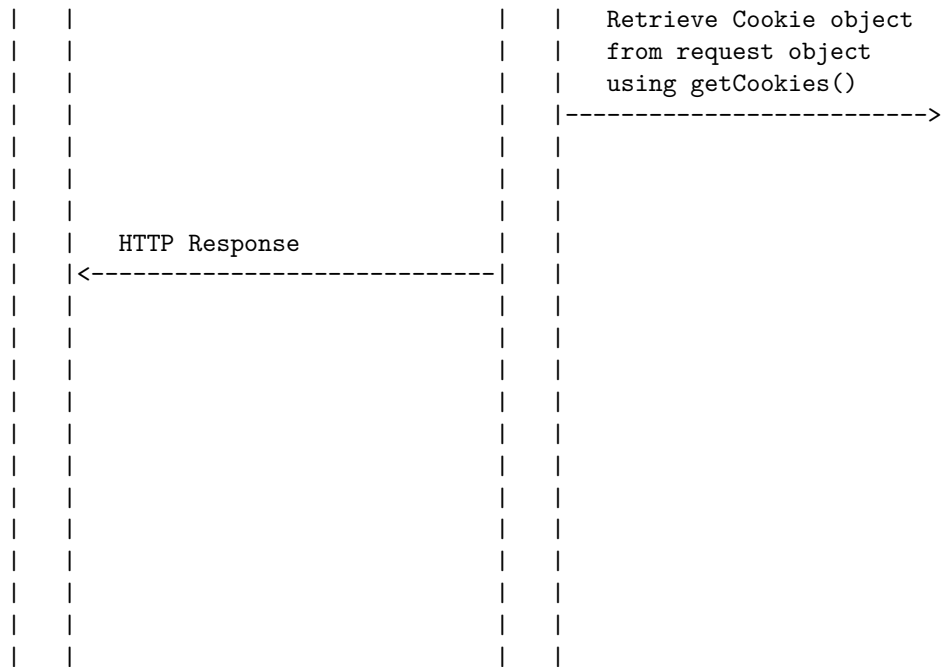
Cookies are created using Cookie class present in Servlet API. Cookies are added to response object using the addCookie() method. This method sends cookie information over the HTTP response stream. getCookies() method is used to access the cookies that are added to response object.

The following diagram illustrates the basic architecture of a cookie in servlet:

Cookies in Servlets

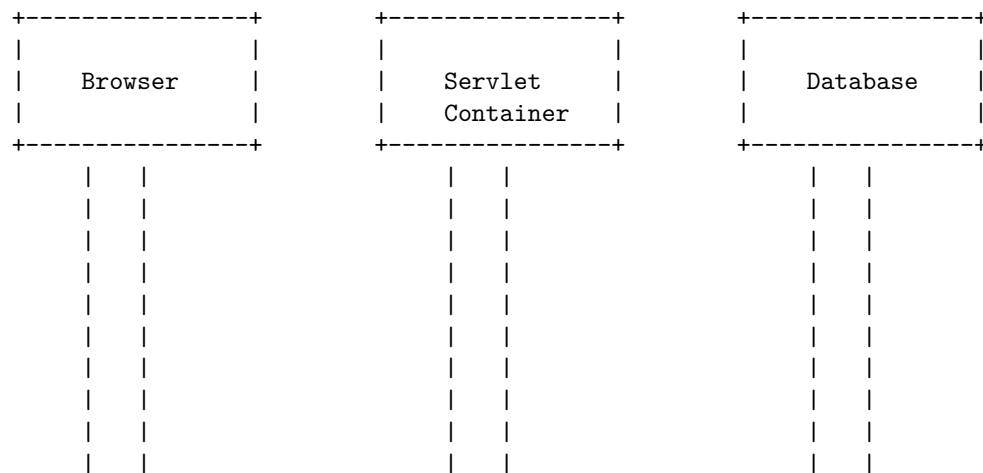


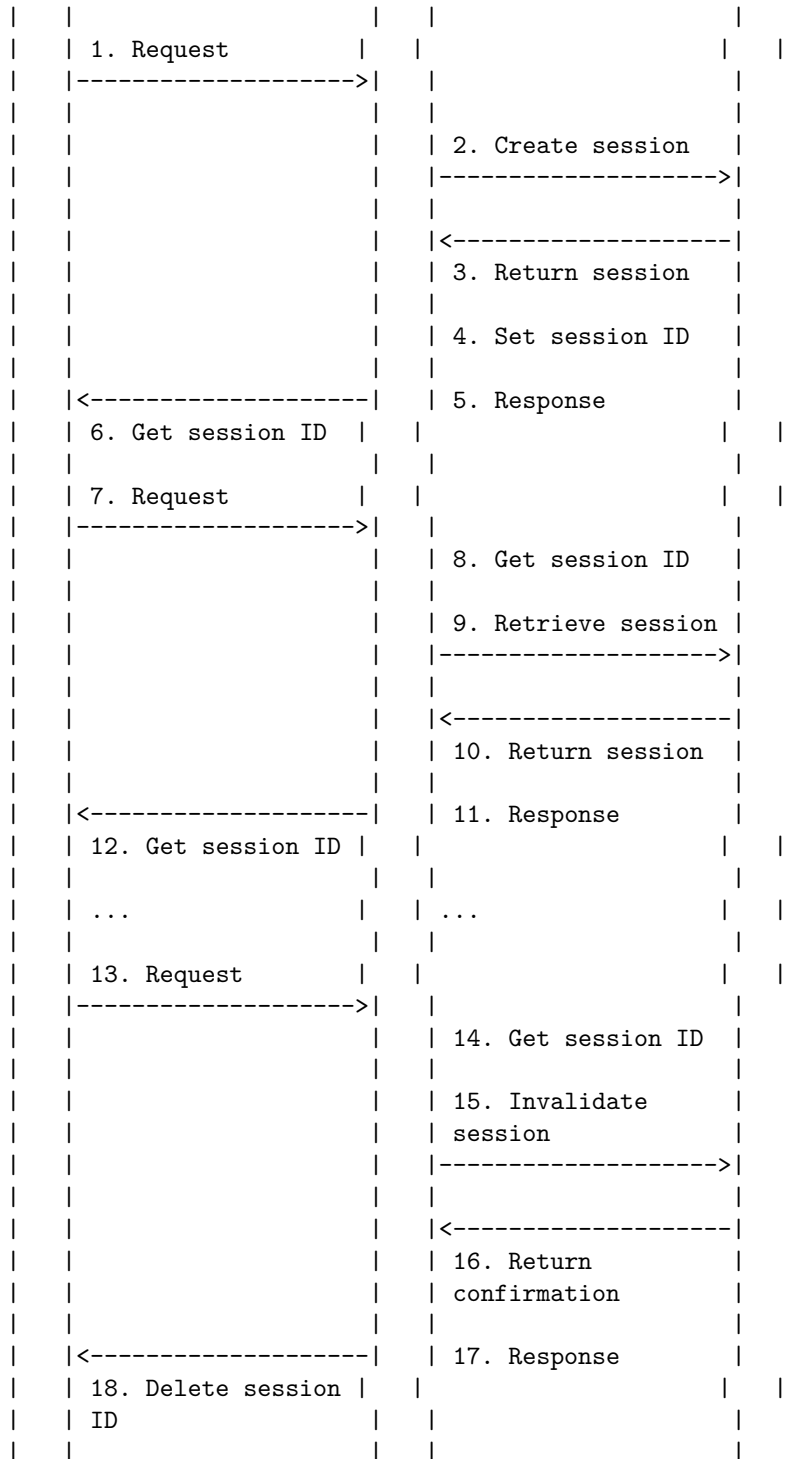




Session tracking is the process of remembering and documenting customer conversions over time. Session tracking allows the server to keep track of successive requests made by the same client. The session is created between an HTTP client and an HTTP server by the servlet container using HttpSession. The session object will be available to all of the servlets and JSP's that the user accesses until the session is closed due to timeout or error.

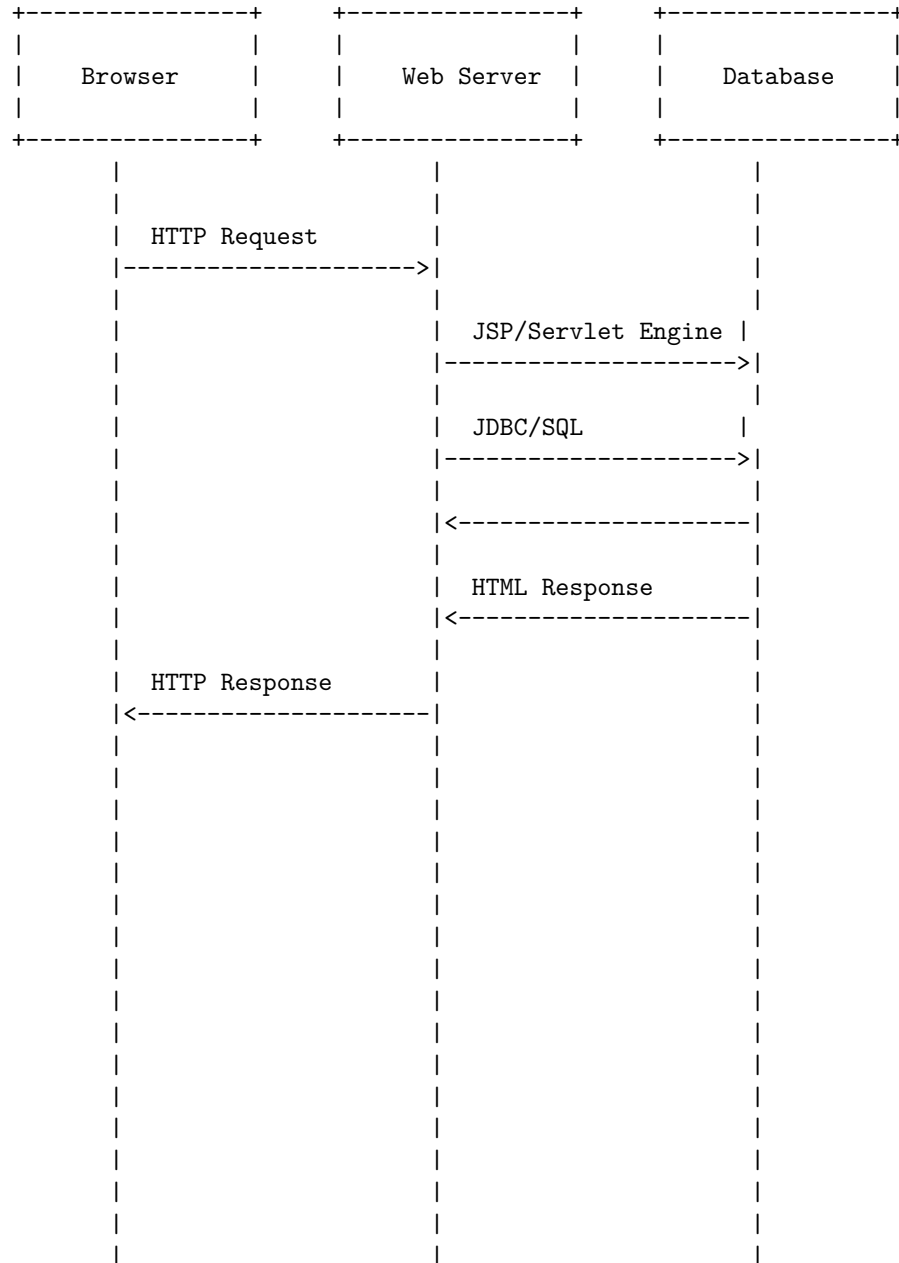
The following diagram illustrates the basic architecture of a session tracking with Http Session in Servlets:





Java Server Pages (JSP) are a technology that allows dynamic content injection into static web pages using Java and Java Servlets. JSP pages can be used in combination with servlets that handle the business logic, the model supported by Java servlet template engines .

The basic architecture of JSP in servlets is as follows:



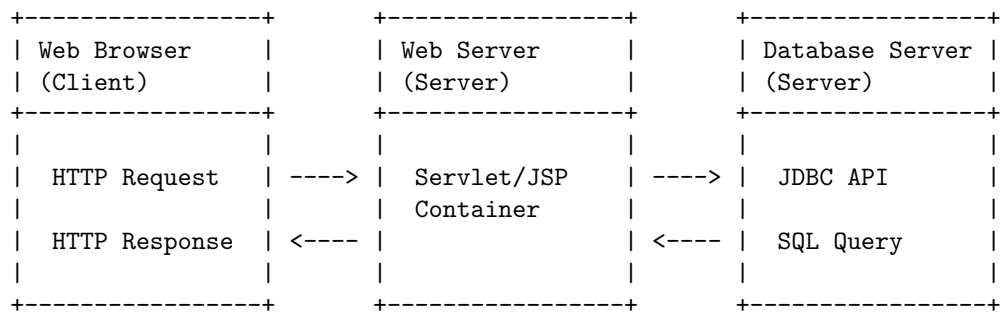
The browser sends an HTTP request to the web server, which then forwards it to the JSP/servlet engine. The JSP/servlet engine processes the request and generates dynamic content using Java code and optionally accesses the database using JDBC/SQL. The JSP/servlet engine then sends back an HTML response to the web server, which then forwards it to the browser. The browser displays the HTML response to the user.

Introduction to JSP in Servlets

JSP stands for Java Server Pages. It is a server-side technology that lets you create dynamic web applications that work on any platform. JSP is similar to HTML pages, but they also contain Java code executed on the server side. JSP is an extension to Servlet technology, which is another server-side technology that handles requests from web clients and produces responses. JSP provides more functionality than Servlet, such as expression language, JSTL, custom tags, etc. A JSP page consists of HTML tags and JSP tags.

Servlets are Java programs that run on a Java-enabled web server or application server. They are used to handle the request obtained from the web server, process the request, produce the response, then send the response back to the web server. Servlets work on the server-side and can use all Java APIs, including the JDBC API, which lets them connect to enterprise databases.

The following diagram illustrates the basic architecture of a JSP and Servlet application:



The steps involved in the diagram are:

- The web browser (client) sends an HTTP request to the web server.
- The web server receives the request and invokes the Servlet or JSP container, which is a component that manages the lifecycle and execution of Servlets and JSPs.
- The Servlet or JSP container executes the corresponding Servlet or JSP page, which may contain Java code to access the database server using the JDBC API.
- The Servlet or JSP page generates an HTTP response, which may contain HTML, CSS, JavaScript, or other content types, and sends it back to the

web server.

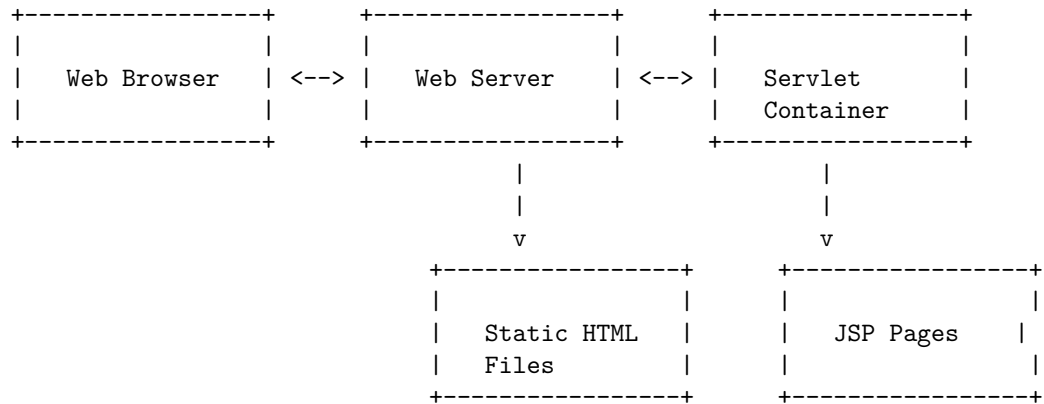
- The web server forwards the response to the web browser, which displays the content to the user.

Java Server Pages Overview in Servlets

JavaServer Pages (JSP) is a technology that allows developers to create dynamic web pages using Java and Java Servlets. JSP pages are compiled into Java servlets and run on the server. JSP uses a special syntax that embeds snippets of Java code within HTML, and these pages are stored as regular HTML files with a .jsp extension.

A servlet is a Java class that extends the `javax.servlet.http.HttpServlet` class and handles HTTP requests and responses. Servlets are under the control of another Java application called a Servlet Container, which is responsible for managing the servlet lifecycle and dispatching requests to the appropriate servlets.

The following diagram illustrates the basic architecture of a JSP and servlet application:



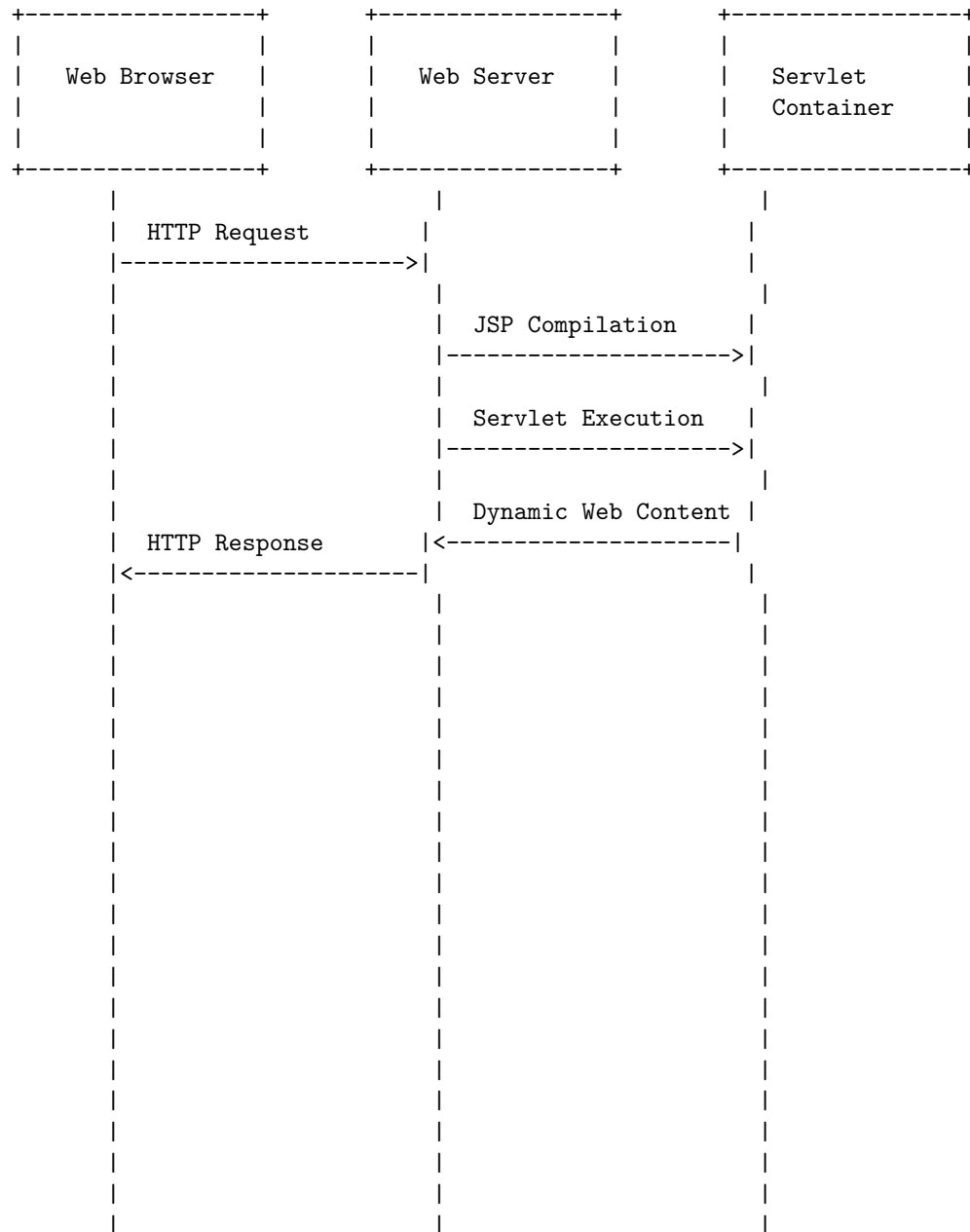
The web browser sends an HTTP request to the web server, which forwards it to the servlet container. The servlet container checks if the request is for a static HTML file or a JSP page. If it is for a static HTML file, the web server serves the file directly to the browser. If it is for a JSP page, the servlet container invokes the corresponding servlet that was generated from the JSP page. The servlet executes the Java code embedded in the JSP page, generates the dynamic HTML content, and sends it back to the web server, which delivers it to the browser.

A Java Server Page (JSP) is a web page that contains Java code embedded in HTML tags. The JSP is compiled and executed by a Java servlet container, which is a component of a web server that supports Java servlets. A servlet is a Java class that handles HTTP requests and generates dynamic web content.

A simple JSP example is a web page that displays the current date and time.

The JSP code uses a scriptlet tag (<% and %>) to insert Java code that calls the java.util.Date class and prints the date and time using the out object, which is an implicit JSP object that represents the output stream.

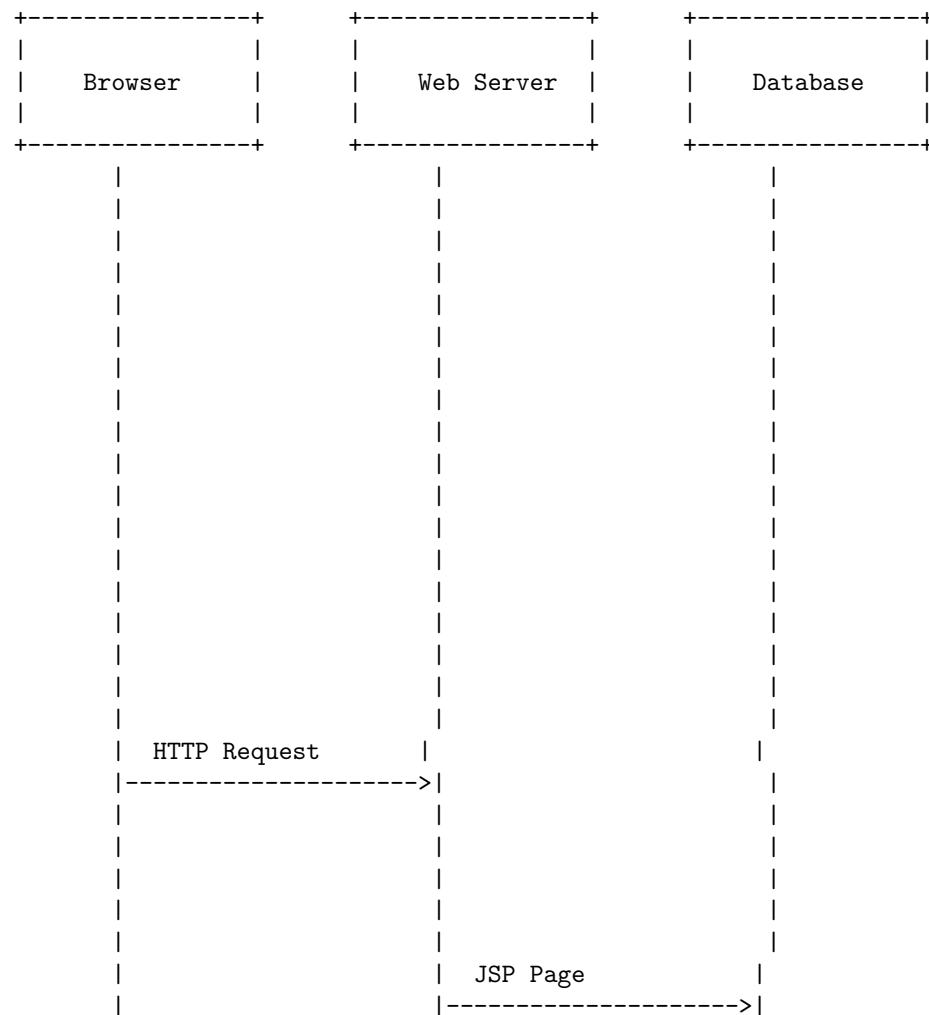
The following diagram illustrates the basic architecture of a JSP and servlet example:

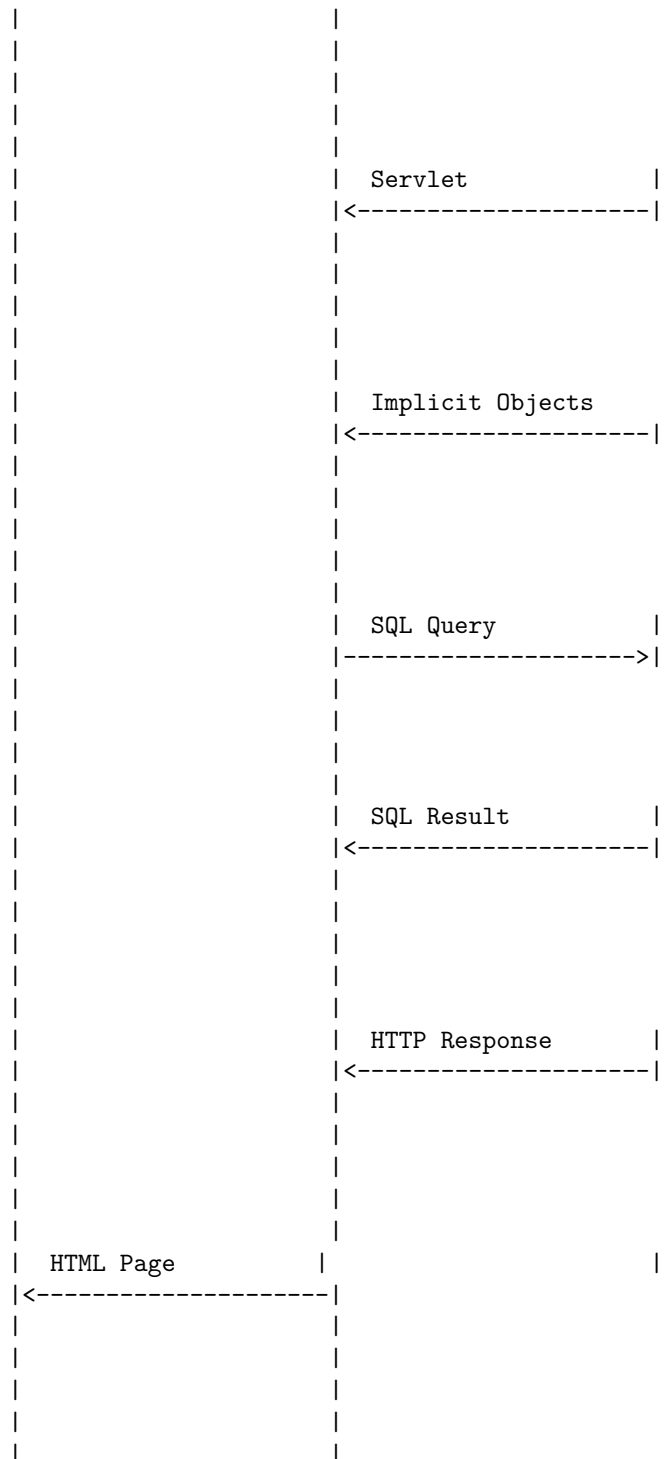


V

Implicit objects in servlets are Java objects that are created by the servlet container during the translation phase of JSP, when JSP is converted to servlet. These objects can be directly used in scriptlets that go in the service method. They are created by the container automatically, and they can be accessed using objects. There are 9 implicit objects in JSP: request, response, out, session, application, config, page, pageContext, and exception.

The following diagram illustrates the basic architecture of implicit objects in servlets using ASCII art:






```

+-----+ +-----+ +-----+
| Web Browser | <----> | Web Server | <----> | Servlet |
+-----+ +-----+ +-----+
          |           |
          |           |
          |           |
          |           |
          |           |
          |           |
          |           |
          |           |
          |           |
          |           |
          v           v
+-----+ +-----+
| HTML Page | | Script File |
+-----+ +-----+

```

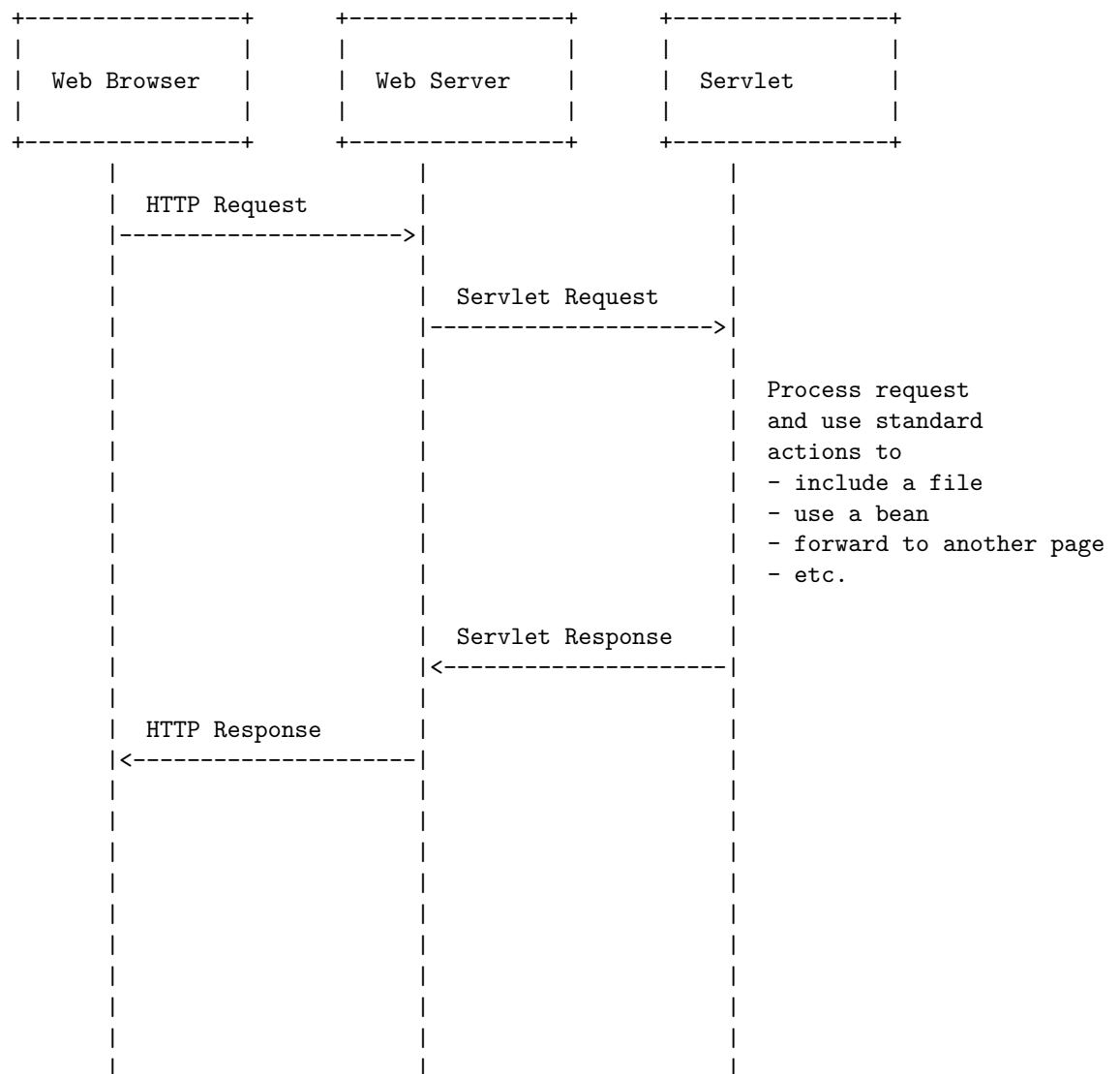
144

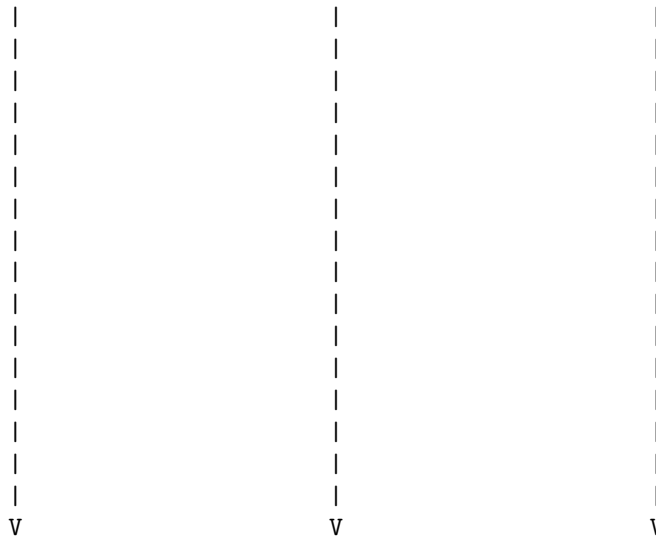
to the web browser, which can update the HTML page accordingly.

Standard actions in servlets are JSP elements that use XML syntax to control the behavior of the servlet engine. They can be used to dynamically insert a file, reuse a bean component, forward the user to another page, etc. There are 12 types of standard actions in JSP, each with a specific tag name and attributes.

The following diagram illustrates the basic architecture of a servlet that uses standard actions to process a request and generate a response:

Standard Actions in Servlets





Directives in Servlets are instructions that tell the container how to handle and process certain parts of the JSP code. They affect the overall structure of the servlet class that is generated from the JSP page. There are three types of directives in JSP: page, include and taglib.

The page directive defines attributes for the entire JSP page, such as the language, content type, error page, buffer size, etc. It has the following syntax:

```
<%@ page attribute="value" %>
```

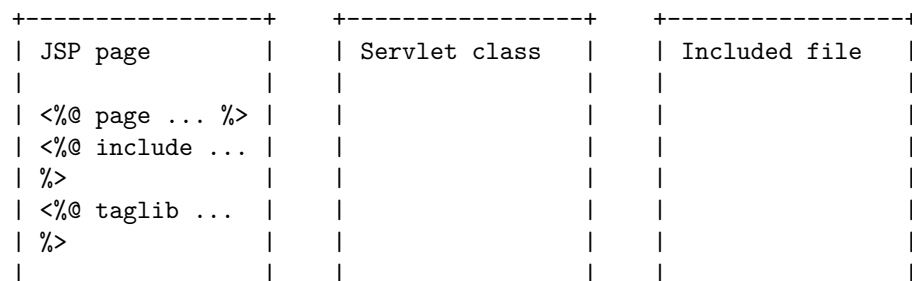
The include directive includes the content of another file at the translation time of the JSP page. It has the following syntax:

```
<%@ include file="filename" %>
```

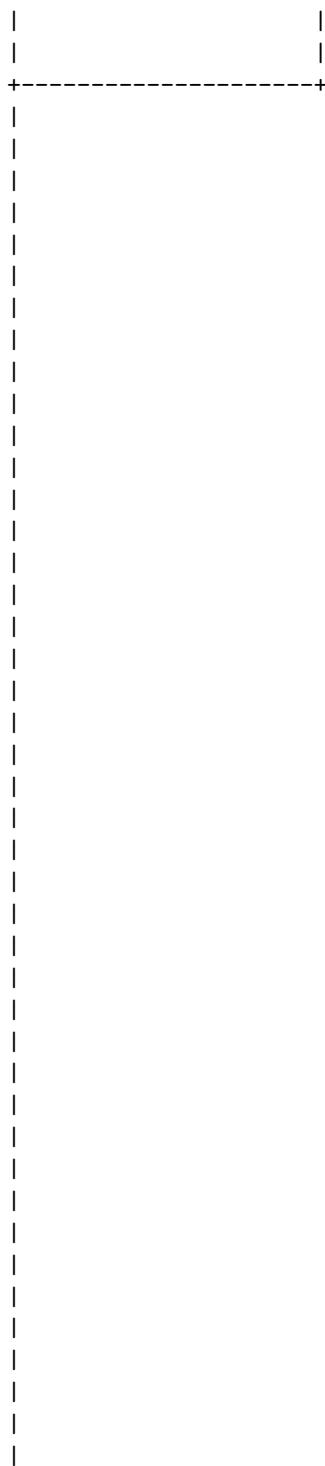
The taglib directive declares a custom tag library that can be used in the JSP page. It has the following syntax:

```
<%@ taglib uri="uri" prefix="prefix" %>
```

The following diagram illustrates the basic architecture of a servlet that is generated from a JSP page with directives:



<html> ... </html>		



|

Custom tag libraries in servlets are a way of creating reusable components that

can be used in JSP pages. Custom tag libraries are defined in a tag library descriptor (TLD) file, which specifies the name, attributes, and implementation class of each custom tag. Custom tag libraries can be used in JSP pages by declaring a taglib directive with the URI and prefix of the library.

The following diagram illustrates the basic architecture of a custom tag library in servlets:

